BioSpecInfo

Dossier completo

Samuele Pio Provenzano

Versione **bsi-v200** · commit `3bcf3b9`

Generato il 2026-10-14

Raccolta integrale dei documenti del progetto, nell'ordine di numerazione. Ogni documento è disponibile anche come PDF separato.

Indice

#	Documento
1	BioSpecInfo — Dossier Tecnico
2	Software Architecture Document (SAD) — BioSpecInfo
3	Verification & Validation (V&V) & Benchmark Report — BioSpecInfo
4	Security, Privacy & Compliance — BioSpecInfo
5	Open-Source Licensing & IP Compliance Matrix — BioSpecInfo
6	Architettura dell'Agente AI "Spectra" — BioSpecInfo
7	Accuratezza scientifica e provenienza dei dati — BioSpecInfo
8	Distinta dei componenti software (SBOM)
9	Matrice di tracciabilità — BioSpecInfo
10	Dichiarazione di conformità di rilascio — BioSpecInfo
11	Riferimento delle interfacce — BioSpecInfo
12	Modello dei dati — BioSpecInfo
13	Guida al deploy — BioSpecInfo
14	Specifiche funzionali — BioSpecInfo
15	Guida per chi la usa — BioSpecInfo
16	Documentazione dei test — BioSpecInfo

BioSpecInfo — Dossier Tecnico

Piattaforma web open-access per la chimica computazionale e la fruizione interattiva di dati chimici su qualsiasi dispositivo.

Autore	Samuele Pio Provenzano
Relatore tesi	Prof. Savino Longo — Università degli Studi di Bari Aldo Moro
Profilo	Cheminformatics Engineer / Scientific Software Developer
Tipo software	Progressive Web App (PWA) client-side, local-first, offline-first
Demo live	https://samupropio1-ship-it.github.io/BioSpecInfo-v11/
Repository	github.com/samupropio1-ship-it/BioSpecInfo-v11

> Questo dossier riassume l'applicazione ed è la copertina della documentazione tecnica completa (SAD, Validation & Benchmark, Security & Compliance, Licensing) allegata di seguito.

---

1. Executive summary

BioSpecInfo è una piattaforma di **chemioinformatica interattiva** che esegue l'intera logica scientifica **nel browser del client**, senza alcun server: analisi molecolare, predizione spettrale, modellistica 2D/3D e strumenti didattici, il tutto **gratuito, installabile come app e funzionante offline**.

Il valore distintivo è la **rara combinazione tra chimica avanzata e ingegneria del software moderna** (WebAssembly, WebGL, PWA), con predittori spettrali **validati quantitativamente** contro i database ufficiali SDBS e NIST.

---

2. Product factsheet

Value proposition — Una piattaforma web da **≈4,5 MB** (file principale) che esegue analisi molecolari, predizioni spettrali e visualizzazioni 3D **a costo server zero** e **100% offline-ready**.

Architettura tecnica	Contenuti & capacità
Client-side / Local-First (nessun backend)	Motore chemioinformatico **RDKit** (WebAssembly)
Single-file HTML5 / ES6, installabile come **PWA**	**Centro Spettri**: IR, $^{1}H/^{13}C$-NMR, MS + pattern isotopico, UV-Vis, Raman
Rendering **2D** (SVG/Canvas) e **3D** (three.js, 3Dmol.js)	**Chimica Organica**: 25 moduli interattivi
Service Worker offline-first, versionato	**Retrosintesi**: 46 strategie + meccanismi animati 2D/3D

Architettura tecnica	Contenuti & capacità
Zero dipendenze copyleft (solo MIT/BSD-3)	**Atlante Farmaci 3D**: 63 malattie (23 tumori), organi 3D, PubMed
Storage locale (localStorage, SQLite via sql.js)	Astrochimica 3D, Lab Quantistico, Accademia (quiz SM-2)

Validazione quantitativa (benchmark di tesi)

- **¹H-NMR** vs SDBS → **MAE 0,31 ppm** (89% dei segnali entro ±0,5 ppm)
- **IR (banda C=O)** vs NIST WebBook → **MAE 11 cm^{-1}** (90% entro ±30 cm^{-1})
- Prestazioni interattive **30–60 fps** da iPhone/iPad a Raspberry Pi 4

---

3. Capacità verificate (stato attuale del codice)

Centro Spettri (editor & analisi molecolare) — disegno da zero (tavola periodica completa, template, drag-to-bond) o caricamento via SMILES/libreria →

- Tutte le notazioni chimiche: formula, SMILES canonico, CXSMILES, SMARTS,

InChI, InChIKey, mol block, nome comune/IUPAC, CAS (con copia rapida).

- Spettri predetti con fonti bibliografiche; **pattern isotopico** (abbondanze

IUPAC) e **masse degli addotti** MS.

- Gruppi funzionali e proprietà (donatori/accettori H, aromatici, rotabili,

stereocentri) **evidenziabili su struttura 2D e 3D**.

- Drug-likeness: **Lipinski, Veber, Egan, Ghose, Muegge**, solubilità ESOL,

QED, **alert strutturali Brenk/PAINS**, radar fisico-chimico.

- Similarità di **Tanimoto** (fingerprint Morgan ECFP4), analisi elementare,

ricerca per sottostruttura SMARTS, export SVG/PNG/MOL.

Sintesi & Retrosintesi — 46 strategie con **player animato del meccanismo** in **2D e 3D**: frecce di *electron pushing*, intermedi di reazione, e modalità retrosintesi con **formazione dei sintoni**.

Chimica Organica — 25 moduli (fondamenti → eterocicli, pericicliche & Woodward–Hoffmann, organometallici & cross-coupling, radicali, spettroscopia, retrosintesi & gruppi protettivi).

Data Science — Atlante 3D dei Farmaci (13 zone del corpo, 63 malattie di cui 23 tumori, modelli 3D degli organi, link PubMed), laboratori ML, playground SQL.

Altro — Astrochimica 3D, Lab Quantistico, Accademia con ripetizione spaziata (SM-2), simulazioni interattive, database di elementi/farmaci/biomolecole.

> Metriche complessive dichiarate nella tesi (patrimonio dati storico): 324 reazioni organiche in grafica vettoriale SVG, 26 vie biosintetiche, 140+ farmaci, 80+ biomolecole, 118 elementi, 10 simulatori quantistici.

---

4. Stack tecnologico

`HTML5` · `ES6/ESNext` · `WebAssembly` (RDKit MinimalLib, sql.js/SQLite) · `WebGL` (three.js, 3Dmol.js) · `SVG` / `Canvas 2D` · `Service Worker` / `PWA` · integrazione API pubbliche (PubChem PUG-REST, NIST, SDBS, MassBank, nmrshiftdb2, NASA/ESA).

---

5. Dichiarazione dell'autore (self-attestation)

Il sottoscritto **Samuele Pio Provenzano** dichiara che:

1. Il software **BioSpecInfo** è **opera originale** dell'autore, ad eccezione

delle librerie open-source di terze parti (RDKit, 3Dmol.js, three.js, SmilesDrawer, sql.js) utilizzate secondo le rispettive licenze permissive e documentate nella *Open-Source Licensing & IP Matrix*.

1. I dati di validazione riportati (MAE 0,31 ppm per ^{1}H-NMR vs SDBS; MAE 11 cm^{-1}

per la banda IR C=O vs NIST) sono stati misurati dall'autore nell'ambito del lavoro di tesi; i predittori sono **euristici ad additività di gruppo**, idonei all'uso didattico e all'identificazione qualitativa dei gruppi funzionali, con i limiti dichiarati nel *Validation Report*.

1. L'architettura è **local-first**: nessun dato personale dell'utente è

trasmesso a server dell'autore (allineamento ai principi GDPR).

> Questa è una **dichiarazione dell'autore**, non una certificazione rilasciata da un ente terzo.

---

6. Documentazione allegata

1. **Software Architecture Document (SAD)** — architettura, componenti, tecnologie.
2. **Verification & Validation & Benchmark Report** — metodo di test, benchmark, limiti.
3. **Security, Privacy & Compliance** — local-first, GDPR, OWASP, GAMP 5 (allineamento).
4. **Open-Source Licensing & IP Matrix** — dipendenze, licenze, proprietà intellettuale.

Licenza del software: proprietaria — "All rights reserved" (vedi `LICENSE`).

Software Architecture Document (SAD) — BioSpecInfo

Campo	Valore
Progetto	BioSpecInfo
Autore	Samuele Pio Provenzano
Relatore tesi	Prof. Savino Longo — Università degli Studi di Bari Aldo Moro
Tipo	Progressive Web App (PWA) client-side, local-first, offline-first
Piattaforma	Browser moderni (Chromium, Firefox, Safari/WebKit) · Desktop e Mobile
Repository	`samupropio1-ship-it/BioSpecInfo-v11`

1. Panoramica del sistema

BioSpecInfo è una piattaforma web **monolitica e modulare** che esegue l'intera logica applicativa nel browser del client, senza alcun server applicativo. È distribuita come **Progressive Web App** installabile (manifest `display: standalone`) e funziona **offline** grazie a un Service Worker con precache delle risorse pesanti.

Pattern architetturale: Single-Page/Single-File Application + Sub-App satellite in iframe, con un *core* chemioinformatico basato su WebAssembly disaccoppiato dalla logica di presentazione.

1.1 Principi guida

- **Local-First:** dati e calcoli risiedono sul dispositivo; nessuna

trasmissione obbligatoria verso terzi.

- **Zero-backend:** nessun database server, nessun costo di hosting dinamico

(pubblicabile su hosting statico, es. GitHub Pages).

- **Progressive enhancement:** le funzioni online (ricerca PubChem, modelli 3D

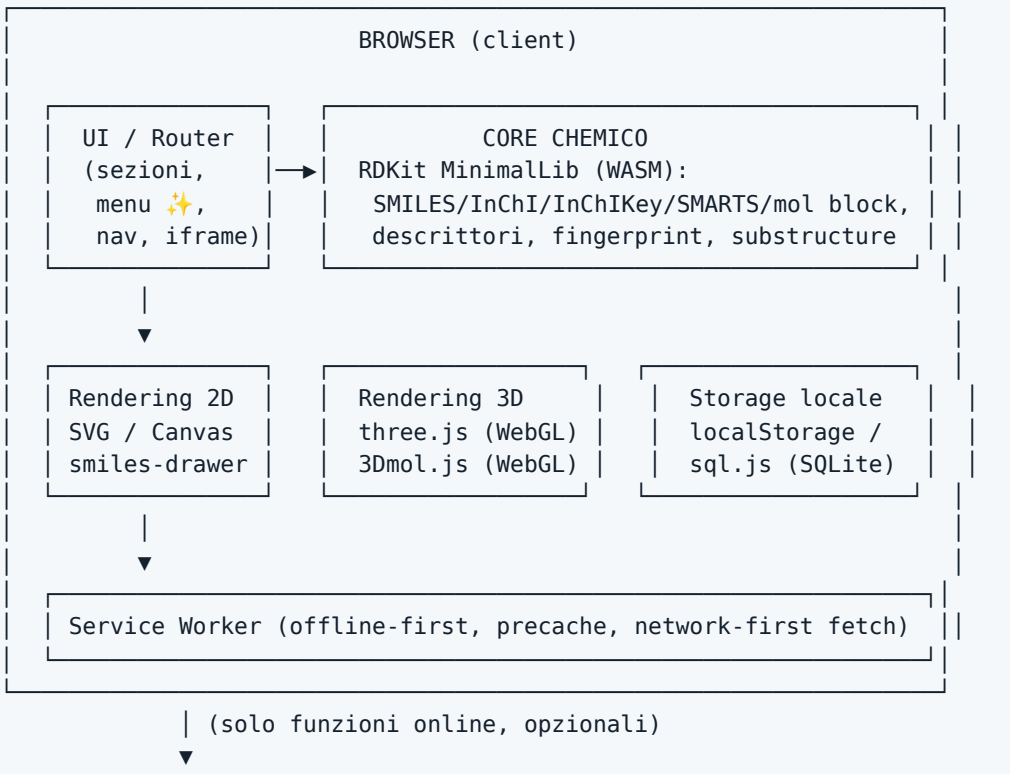
reali, immagini astronomiche) degradano con eleganza in assenza di rete.

2. Struttura dei file

File	Ruolo
`index.html`	Applicazione principale (core chimico, Centro Spettri, Data Science, menu ✨). ~4,5 MB, oltre 43.000 righe.
`chimorga.html`	Corso di Chimica Organica (25 moduli).

File	Ruolo
`astro.html`	Astrochimica 3D (three.js).
`pro.html`, `rdkit_lab.html`, `accademia.html`, `simulazioni.html`, `guidaret.html`, `file_manager.html`, `Biochimica_Guida_Definitiva.html`, `sr_completo.html`, `sr_essenziale.html`, `download.html`, `changelog_tesi.html`	Sotto-applicazioni tematiche caricate in iframe con cache-busting `?v=timestamp`.
`manifest.json`	Configurazione PWA (icone, tema, standalone).
`sw.js`	Service Worker: strategia *network-first* con fallback su cache; precache delle librerie e degli asset pesanti; versione cache incrementata a ogni release.
`RDKit_minimal.js` / `RDKit_minimal.wasm`	Motore chemioinformatico (RDKit MinimalLib).
`3Dmol-min.js`	Viewer 3D di macromolecole/molecole.
`three.min.js`, `three_bloom.js`, `gltf_loader.js`	Rendering 3D WebGL (anatomia, astrochimica, meccanismi).
`smiles-drawer.min.js`	Depiction 2D di SMILES.
`lib/sql-wasm.js` / `lib/sql-wasm.wasm`	SQLite in WebAssembly (playground SQL).
`models/`, `textures/`	Asset 3D (organi, ecorché, pianeti) e texture.

3. Architettura a componenti



3.1 Module Router / UI

Navigazione tra sezioni tramite classi CSS (`.section.on`) e un menu ✨ (FAB) che apre gli strumenti speciali (Centro Spettri, Retrosintesi, Lab Quantistico, Data Science, ecc.). Le sotto-app sono isolate in `iframe` con versionamento.

3.2 WebAssembly Engine (RDKit MinimalLib)

Caricato on-demand (`window.bsiLoadRDKit`) con `locateFile` sui file `.wasm` locali e CDN come sola riserva. Espone:

- **Identificatori:** SMILES canonico/isomerico, CXSMILES, InChI, InChIKey,

SMARTS, mol block MDL V2000.

- **Descrittori:** peso molecolare, massa esatta monoisotopica, LogP

(Wildman–Crippen), TPSA, HBD/HBA, legami rotabili, anelli aromatici/alifatici, frazione Csp^3, rifrattività molare, stereocentri.

- **Fingerprint & similarità:** Morgan ECFP4 (1024 bit) e indice di Tanimoto.
- **Substructure matching** via SMARTS (`get_qmol` + `get_substruct_matches`).
- **Depiction 2D** vettoriale (`get_svg`, `get_svg_with_highlights`) e

generazione di coordinate 2D (mol block) reimportabili nell'editor.

3.3 Rendering

- **2D:** SVG proprietario (meccanismi di reazione con frecce di Bézier),

Canvas 2D (editor molecolare, viewer stereochimico CIP), smiles-drawer.

- **3D:** `three.js` (WebGL) per anatomia (Atlante Farmaci), astrochimica e

meccanismi 3D; `3Dmol.js` per i modelli molecolari da SDF PubChem.

3.4 Storage Layer

- `localStorage` (con **guardia di sicurezza** e fallback in memoria per

modalità incognito / storage bloccato).

- `sql.js` (SQLite compilato in WASM) per il playground SQL della sezione Data

Science.

---

4. Funzionalità implementate (stato attuale)

4.1 Centro Spettri (editor & analisi molecolare)

Disegno da zero (tavola periodica completa, template di anelli/gruppi, drag-to-bond con snap) o caricamento via SMILES / libreria (63 molecole) → analisi completa:

- **Tutte le notazioni chimiche** (formula, SMILES canonico, CXSMILES, SMARTS,

InChI, InChIKey, mol block, nome comune/IUPAC, CAS) con copia rapida.

- **Spettri predetti** in micro-sezioni: IR, ^{1}H-NMR, ^{13}C-NMR, MS con **pattern

isotopico **(abbondanze IUPAC) e** masse degli addotti** ([M+H]$^{+}$, [M+Na]$^{+}$…), UV-Vis, Raman — ciascuno con fonte bibliografica.

- **Gruppi funzionali** (~30, via SMARTS) e **proprietà** (donatori/accettori H,

aromatici, rotabili, stereocentri) **evidenziabili su struttura 2D e 3D**.

- **Drug-likeness / ADMET:** regole Lipinski, Veber, Egan, Ghose, Muegge;

solubilità stimata ESOL; QED (quando esposto dall'engine); **alert strutturali Brenk/PAINS**; profilo radar fisico-chimico.

- **Analisi elementare** (% in peso) e **similarità di Tanimoto** con la libreria.
- **Modelli 3D** (ball&stick, spacefill, wireframe) da PubChem; export SVG/PNG/MOL.

4.2 Sintesi & Retrosintesi

46 strategie di disconnessione con **player animato del meccanismo** in **2D e 3D**: frecce curve di *electron pushing*, intermedi di reazione, e modalità retrosintesi con **formazione dei sintoni**.

4.3 Chimica Organica (`chimorga.html`)

25 moduli: fondamenti, stereochimica, alcheni/alchini, aromatici e SEA, SN/E, carbonili, ammine, carboidrati, amminoacidi, lipidi, nucleotidi, sintesi animate, reazioni nominali, eterocicli, pericicliche & Woodward–Hoffmann, organometallici & cross-coupling, radicali, spettroscopia, retrosintesi & gruppi protettivi.

4.4 Data Science

Atlante 3D dei Farmaci (13 zone del corpo, 63 malattie di cui 23 tumori, modelli 3D degli organi, link PubMed), laboratori ML interattivi, playground SQL (sql.js), roadmap corsi.

4.5 Altre sezioni

Astrochimica 3D, Lab Quantistico, Accademia (quiz gamificati con ripetizione spaziata SM-2), simulazioni interattive, guide, database di elementi/farmaci/ biomolecole.

---

5. Roadmap architetturale (NON ancora implementata)

Le seguenti voci fanno parte della visione *enterprise/HPC* e richiedono binari, modelli o pipeline di build esterni non ospitabili in una PWA single-file:

- Parallelismo HPC via Web Workers + `SharedArrayBuffer` / `Atomics`.
- Accelerazione WebGPU per algebra densa/sparsa e inferenza GNN.
- Predittori ML in ONNX Runtime WebGPU (GNN per NMR tipo ShiftML2/DeepNMR;

hERG, BBB, CYP450, LogS).

- Docking molecolare in-browser (AutoDock Vina/Webina, formato PDBQT).
- Chimica quantistica semi-empirica (xTB/GFN2-xTB in WASM).
- Motore di retrosintesi computazionale basato su regole SMIRKS.
- Database SQLite-WASM su scala 1.000.000 di strutture e batch SDF via Web Worker.
- Visualizzazione immersiva WebXR (VR/AR).
- Packaging nativo (TWA Android, iOS, Electron/Tauri) e pipeline CI/CD.

---

6. Requisiti e vincoli

- **Browser:** supporto a WebAssembly e WebGL; JavaScript abilitato.
- **Rete:** non richiesta per il nucleo; consigliata per ricerca PubChem e

modelli 3D reali.

- **Prestazioni:** target 30–60 fps sui rendering interattivi; i calcoli pesanti

sono delegati a RDKit WASM e mantenuti reattivi.

Verification & Validation (V&V) & Benchmark Report — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Autore	Samuele Pio Provenzano
Scopo	Documentare metodo di verifica, dati di validazione dei predittori spettrali, prestazioni cross-device e limiti dichiarati del modello.

1. Scopo e strategia di V&V

- **Verifica (Verification):** il software è costruito correttamente (sintassi, assenza di errori runtime, comportamento atteso dei componenti).
- **Validazione (Validation):** i risultati scientifici (predittori spettrali, descrittori) sono confrontati con banche dati sperimentali di riferimento.

2. Verifica del software (QA)

Tecnica	Descrizione	Stato
Controllo sintattico	`node --check` sui blocchi JavaScript critici a ogni modifica.	✅ In uso
Test end-to-end (E2E)	Automazione con **Playwright** su Chromium headless: apertura sezioni, disegno/caricamento molecole, verifica dei pannelli (notazioni, spettri, gruppi funzionali, ADMET, similarità), animazioni 2D/3D. Conteggio degli errori JavaScript di console/pagina (criterio di accettazione: **0 errori** non di rete).	✅ In uso
Verifica su server HTTP	RDKit WASM richiede contesto HTTP (CORS): i test girano su server locale per riprodurre l'ambiente reale.	✅ In uso
Test manuale cross-device	Prova su dispositivi reali (vedi §4).	✅ In uso
Suite di unit test automatizzata (Vitest/Jest)	—	🔜 Roadmap
Copertura ≥ 90%	—	🔜 Roadmap

> Ogni release è verificata E2E con Playwright con criterio "zero errori JS" prima del merge sul ramo principale.

3. Validazione scientifica dei predittori spettrali

I predittori spettrali di BioSpecInfo usano **modelli euristici ad additività di gruppo / regole strutturali** (tabelle di riferimento di letteratura), scelti per garantire **risposta istantanea in-browser** su qualsiasi dispositivo.

3.1 Benchmark ¹H-NMR vs SDBS

Metrica	Valore riportato
Database di riferimento	**SDBS** (Spectral Database for Organic Compounds, AIST)
Errore Assoluto Medio (MAE)	**0,31 ppm**
Segnali entro ±0,5 ppm	**89%**

3.2 Benchmark IR (banda C=O) vs NIST

Metrica	Valore riportato
Database di riferimento	**NIST Chemistry WebBook**
Errore Assoluto Medio (MAE)	**11 cm^{-1}**
Bande entro ±30 cm^{-1}	**90%**

3.3 Proprietà fisico-chimiche

Descrittori (peso molecolare, formula, LogP, TPSA, HBD/HBA, anelli, stereocentri) calcolati tramite **RDKit MinimalLib** e confrontabili con **PubChem** e **DrugBank**. Esempio di verifica riproducibile: *colesterolo → $C_{27}H_{46}O$, 4 anelli, 8 stereocentri*.

> **Fonte dei dati di benchmark:** misurazioni condotte dall'autore nell'ambito della tesi di laurea. I valori vanno intesi come validazione a scopo **didattico e di identificazione qualitativa** dei gruppi funzionali, non come sostituto di calcoli quantomeccanici di riferimento.

---

4. Prestazioni cross-device

Prove di fluidità dei rendering interattivi (viewer 3D, animazioni, editor) condotte su un ventaglio di dispositivi:

Dispositivo	Esito
iPhone / iPad	30–60 fps
Laptop Windows / macOS	60 fps
Raspberry Pi 4	30–60 fps

Peso del file principale `index.html`: **≈ 4,5 MB** (l'intero pacchetto, comprensivo dei moduli WASM e degli asset 3D, è superiore ma resta interamente statico e cache-abile offline).

5. Limiti dichiarati del modello

In coerenza con il principio di trasparenza scientifica:

1. **Modello ad additività.** I predittori NMR/IR sommano contributi di gruppo e

non modellano pienamente gli effetti dell'intorno chimico esteso (coniugazione lunga, effetti anisotropici, legami a idrogeno intramolecolari). Scarti maggiori sono attesi, ad es., su **carbonili fortemente coniugati** o su casi limite come l'**acido formico**.

1. **Spettri qualitativi/didattici.** Gli spettri simulati servono a riconoscere

i gruppi funzionali e a comprendere le relazioni struttura–spettro, non a sostituire dati sperimentali certificati.

1. **Dipendenza da rete per alcune funzioni.** Nome IUPAC/comune, numero CAS,

pittogrammi GHS e modelli 3D reali provengono da PubChem: in assenza di rete l'app mostra un fallback e resta pienamente funzionale sui dati locali.

1. **Precisione a doppia via.** Per l'uso rigoroso si rimanda ai link diretti

verso i database ufficiali (NIST, SDBS, MassBank, nmrshiftdb2) integrati nell'interfaccia.

---

6. Criteri di accettazione di una release

- ✅ `node --check` supera senza errori sui moduli modificati.
- ✅ Test E2E Playwright: 0 errori JS non di rete.
- ✅ Nessuna regressione visibile nelle sezioni toccate.
- ✅ Versione della cache del Service Worker incrementata.
- ✅ `tools/verifica-farmaci.js`: nessun difetto; ogni deviazione registrata con il motivo.
- ✅ `verifica_guida`: ciò che la documentazione promette esiste nel codice.

---

7. Batteria di verifica — stato alla versione `bsi-v200`

La verifica non è più solo end-to-end funzionale: comprende banchi dedicati alle proprietà che un test funzionale non osserva (stabilità nel tempo, degrado di rete, accuratezza dei dati).

7.1 Composizione

Famiglia	Banchi	Oggetto
Dati scientifici	`verifica-farmaci`, `test_spettri`, `test_assi`, `test_assi_canvas`, `test_costanti`, `audit_dati`	struttura vs peso molecolare, riconoscimento gruppi funzionali, convenzioni degli assi, costanti fisiche

Famiglia	Banchi	Oggetto
Agente AI	`test_ko`, `test_404`, `test_503`, `test_firma`, `test_attesa`, `test_attesalunga`, `test_tetto`, `test_nucleo`, `browser_ko`, `browser_prova`	modello ritirato, CORS, sovraccarico, firme di ragionamento, tetto di token, attese prolungate, memoria dei fornitori irraggiungibili
Stabilità	`audit_stabilita`, `audit_promesse`, `audit_quota`, `test_filemanager`, `test_doppioinvio`, `test_sw`, `test_visore3d`	sessioni lunghe, promesse rifiutate, memoria esaurita, archivio non disponibile, invii sovrapposti, rete degradata, contesti WebGL
Interfaccia	`browser_reset`, `browser_proxy`, `browser_proxyui`, `browser_rdkit`, `browser_lab`, `browser_frontiera`, `test_aggiorna`, `test_guidaproxy`	pannelli, proxy, laboratorio RDKit, aggiornamenti
Coerenza	`verifica_guida`	corrispondenza documentazione ⟷ codice, marcatori di conflitto su 58 file

7.2 Proprietà verificate che un test funzionale non osserva

Proprietà	Metodo	Risultato misurato
Assenza di perdite di memoria	95 sezioni aperte per 5 giri, conteggio nodi DOM per giro	costruzione +26 876 nodi al primo giro, **+0 nei quattro successivi**
Sopravvivenza a memoria esaurita	`localStorage.setItem` forzato a lanciare	10 pagine su 10 restano operative
Degrado di rete (≠ assenza di rete)	richieste sospese 20 s, intercettazione a livello di contesto	risposta dalla cache in **3 507 ms** (soglia 3 500 ms)
Riproducibilità degli spettri	doppio disegno, confronto byte a byte	identici
Contesti WebGL	conteggio costruzioni del visore 3D su 10 molecole	da 7 a **1**

7.3 Note di metodo

Un banco di prova che non misura nulla **passa**, ed è il modo più insidioso di ottenere una falsa garanzia. Durante lo sviluppo della batteria sono stati individuati e corretti diversi casi di misura vacua, fra cui:

- `page.route` di Playwright **non** intercetta le richieste originate dal

Service Worker: serve `context.route`. Con la prima, la prova «rete degradata» misurava una rete perfettamente funzionante e concludeva in 16 ms.

- Sondare l'esaurimento di `localStorage` con una scrittura di un byte non prova

nulla: dopo la saturazione un byte trova sempre posto.

- Le tacche di un asse su `<canvas>` non sono leggibili come testo: vanno

raccolte intercettando `fillText`, e la riga più in basso è il *titolo* dell'asse, non le tacche.

Da qui la regola adottata: **ogni banco che simula una condizione deve contare quante volte la simulazione è realmente scattata**, e fallire se il conteggio è zero.

Security, Privacy & Compliance — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Autore	Samuele Pio Provenzano
Modello	Client-side / Local-First PWA

1. Privacy & protezione dei dati (GDPR)

1.1 Local-First per progettazione

- **Nessun account, nessun login, nessun tracciamento pubblicitario.**
- Tutti i dati dell'utente (molecole disegnate, preferenze, cronologia dei quiz,

progressi di studio) sono conservati **esclusivamente in locale** sul dispositivo (`localStorage` / IndexedDB / `sql.js`), senza trasmissione a server dell'autore.

- **Non esiste un backend applicativo** che raccolga o elabori dati personali.

1.2 Trattamento dei dati e allineamento GDPR

- **Minimizzazione dei dati:** l'app non richiede dati personali per funzionare.
- **Assenza di trasferimento a terzi dell'autore:** i dati non lasciano il

dispositivo per infrastrutture controllate dall'autore.

- **Chiamate esterne opzionali e trasparenti:** solo su azione dell'utente

(ricerca di una molecola, apertura di un modello 3D) l'app interpella API pubbliche di terzi (PubChem, NASA/ESA, database spettrali). Tali richieste contengono **solo l'identificativo chimico** (es. SMILES/nome) e **nessun dato personale**. L'uso di questi servizi è soggetto alle rispettive privacy policy.

- **Diritto all'oblio locale:** l'utente può cancellare i dati svuotando lo

storage del sito dal browser.

> **Nota.** Questo allineamento descrive le proprietà di design local-first che favoriscono la conformità GDPR. Non costituisce una certificazione legale; per contesti regolamentati si raccomanda una valutazione DPIA dedicata.

2. Sicurezza applicativa (OWASP)

2.1 Superficie di attacco

Essendo un'applicazione **statica senza backend**, sono assenti per costruzione intere classi di vulnerabilità server-side (SQL injection su DB server, RCE lato server, autenticazione/sessione server, IDOR su API proprietarie).

2.2 Sanitizzazione degli input

- Il parsing di stringhe **SMILES / SMARTS** e di file molecolari è delegato a

RDKit MinimalLib (WASM), che valida le strutture e rifiuta input malformati restituendo molecole non valide anziché eseguirli.

- Le stringhe provenienti da input o da API esterne, quando inserite nel DOM,

vanno trattate come non fidate. Il valore testuale (nomi, notazioni) è mostrato come contenuto testuale/`textContent` o con escaping dei caratteri `& < >`.

2.3 Robustezza del runtime

- **Guardia su `localStorage`:** ogni accesso allo storage è protetto da

`try/catch` con fallback in memoria, per evitare che la modalità incognito o lo storage bloccato interrompa l'esecuzione (regressione storica risolta).

- **Degradazione controllata:** i pannelli non restano mai vuoti in silenzio; gli

errori mostrano un messaggio e, dove applicabile, un pulsante "Riprova".

- **Isolamento delle sotto-app** in `iframe`.

2.4 Content Security Policy (raccomandazione di deploy)

Poiché l'app è pubblicabile su hosting statico, si raccomanda di servirla con header di sicurezza a livello di hosting/CDN:

- `Content-Security-Policy` restrittiva (script/style self + WASM), `frame-ancestors`,

`X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`.

- `Cross-Origin-Opener-Policy` / `Cross-Origin-Embedder-Policy` se in futuro si

abiliteranno `SharedArrayBuffer` e i Web Worker paralleli (roadmap HPC).

---

3. Integrità e riproducibilità (allineamento GAMP 5)

Per l'uso in contesti farmaceutici/regolamentati, l'architettura è **compatibile** con i principi GAMP 5:

- **Determinismo:** i calcoli chemioinformatici (RDKit) sono deterministici e

riproducibili a parità di input.

- **Tracciabilità delle versioni:** versionamento del codice su Git, cache del Service Worker versionata, changelog.
- **Trasparenza dei metodi:** i predittori euristici sono documentati con le loro fonti e i loro limiti (vedi *Validation Report*).

> Anche qui: si tratta di **allineamento ai principi**, non di una qualifica formale (IQ/OQ/PQ), che richiederebbe un processo di validazione dedicato presso l'organizzazione utilizzatrice.

---

4. Riepilogo

Aspetto	Stato
Dati personali trasmessi a server dell'autore	**Nessuno**
Backend / database server	**Assente**
Validazione input molecolari	RDKit WASM
Escaping output nel DOM	Sì (contenuto non fidato)
Guardia storage / degradazione errori	Sì
CSP / header di sicurezza	Raccomandati a livello di hosting
Funzionamento offline	Sì (Service Worker)

Open-Source Licensing & IP Compliance Matrix — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Autore / titolare del codice proprietario	Samuele Pio Provenzano

1. Matrice delle dipendenze di terze parti

Tutte le librerie di terze parti incluse sono **open-source con licenze permissive** (MIT / BSD-3-Clause). **Nessuna dipendenza copyleft virale (GPL / AGPL / LGPL)** è utilizzata, quindi non vi sono obblighi di ridistribuzione del codice proprietario.

Componente	File nel repo	Uso	Licenza
RDKit (MinimalLib / rdkit-js)	`RDKit_minimal.js`, `RDKit_minimal.wasm`	Motore chemioinformatico (SMILES/InChI/InChIKey/SMARTS, descrittori, fingerprint, depiction 2D)	**BSD-3-Clause**
3Dmol.js	`3Dmol-min.js`	Viewer 3D di molecole/macromolecole (WebGL)	**BSD-3-Clause**
three.js	`three.min.js`, `three_bloom.js`, `gltf_loader.js`	Rendering 3D WebGL (anatomia, astrochimica, meccanismi)	**MIT**
SmilesDrawer	`smiles-drawer.min.js`	Depiction 2D di stringhe SMILES	**MIT**
sql.js (SQLite compilato in WASM)	`lib/sql-wasm.js`, `lib/sql-wasm.wasm`	Playground SQL client-side	**MIT**
PDF.js	`vendor/pdfjs/`	Apertura dei documenti PDF nel lettore di spettri: ogni pagina disegnata e il suo strato di testo	**Apache-2.0**

> Le versioni esatte sono quelle dei file inclusi nel repository. Ogni licenza permissiva richiede la conservazione dell'avviso di copyright originale, che va mantenuto nei file di libreria o in un file `THIRD_PARTY_NOTICES`.

2. Servizi/API esterni (non ridistribuiti)

Questi servizi sono interpellati via rete a runtime (dati non inclusi nel pacchetto): il loro uso è soggetto ai rispettivi termini.

Servizio	Uso
PubChem PUG-REST (NIH/NCBI)	Nome IUPAC/comune, CAS, formula, modelli 3D SDF, pittogrammi GHS/CLP
NIST WebBook · SDBS · MassBank · nmrshiftdb2 · SpectraBase	Link a spettri sperimentali di riferimento
NASA / ESA	Immagini e dati astronomici (sezione Astrochimica)

---

3. Proprietà intellettuale del codice proprietario

- Il codice applicativo di BioSpecInfo (logica dell'interfaccia, engine

vettoriale SVG dei meccanismi, viewer stereochimico CIP in Canvas, predittori spettrali euristici, contenuti didattici, database interni) è **opera originale dell'autore**, salvo le librerie di terze parti sopra elencate.

- **Nessun codice sotto licenza copyleft** è incorporato nel sorgente

proprietario: l'autore mantiene piena libertà di licenza sul proprio codice.

3.1 Licenza del progetto

Il repository include un file `LICENSE` **proprietario — "All rights reserved"**: il codice è visibile e valutabile (studio personale e valutazione professionale/accademica), ma **copia, modifica, ridistribuzione e uso commerciale richiedono l'autorizzazione scritta dell'Autore**. Questa scelta protegge il progetto da riusi non autorizzati pur mantenendolo mostrabile a recruiter e aziende. I componenti di terze parti restano soggetti alle loro licenze permissive.

---

4. Riepilogo compliance

Voce	Esito
Dipendenze copyleft virali (GPL/AGPL)	**Assenti**
Licenze delle dipendenze	Permissive (MIT / BSD-3-Clause)
Obbligo di conservare gli avvisi di copyright	Sì (nei file di libreria / NOTICES)
Libertà di licenza sul codice proprietario	Piena
File `LICENSE` del progetto	✅ Presente — proprietaria «All rights reserved»

Architettura dell'Agente AI "Spectra" — BioSpecInfo

Campo	Valore
Progetto	BioSpecInfo — componente Spectra (copilota AI agentico)
Autore	Samuele Pio Provenzano
Relatore tesi	Prof. Savino Longo — Università degli Studi di Bari Aldo Moro
Componente	`bsi-ai-hub.js` — 6.254 righe, nessuna dipendenza runtime
Tipo	Agente conversazionale multi-provider con esecuzione di strumenti lato client
Repository	`samupropio1-ship-it/BioSpecInfo-v11`
Versione documentata	Service Worker `bsi-v200`

1. Sintesi esecutiva

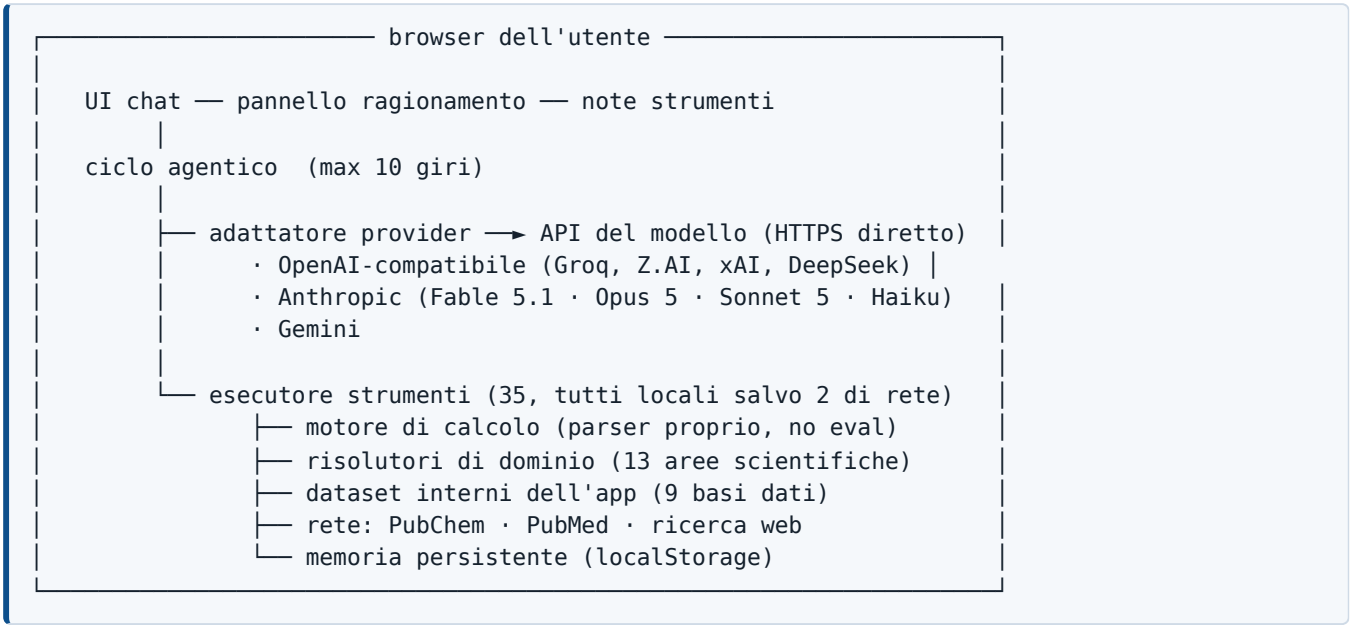
Spectra è un **agente** — non un semplice wrapper su un modello linguistico. La differenza è misurabile: il modello non risponde a memoria sui dati quantitativi, ma **invoca strumenti deterministici** che calcolano il risultato, e il ciclo di controllo gli permette di **concatenare fino a 10 chiamate** verificando ogni passaggio prima di rispondere.

L'architettura affronta tre problemi che distinguono un agente utilizzabile da una demo:

1. **Fondatezza (*grounding*)** — un dato chimico inventato può essere pericoloso. 35 strumenti coprono calcolo, banche dati pubbliche e i dataset interni dell'applicazione; il prompt di sistema vieta esplicitamente di citare valori numerici a memoria.
1. **Trasparenza** — il ragionamento del modello è mostrato in tempo reale e conservato accanto alla risposta, insieme all'elenco degli strumenti effettivamente invocati. L'operato dell'agente è verificabile a posteriori.
1. **Portabilità fra fornitori** — un unico registro di strumenti viene tradotto nei tre formati di *function calling* oggi in uso (OpenAI-compatibile, Anthropic, Gemini), così l'agente funziona identico su dieci configurazioni di modello, quattro delle quali gratuite.

2. Architettura

2.1 Vista d'insieme



Non esiste un backend. Le chiavi API restano nel `localStorage` del browser e viaggiano solo verso il fornitore scelto: nessun dato transita da server di BioSpecInfo, che non esistono.

2.2 Ciclo agentico

Ad ogni giro il ciclo: invia la cronologia (ultimi 40 turni) più il prompt di sistema e lo schema dei 35 strumenti → riceve la risposta in *streaming* → se contiene chiamate a strumenti le esegue **tutte** → ricostruisce il turno assistente nel formato nativo del fornitore → ripete.

Tre condizioni di uscita: nessuna chiamata a strumenti (risposta finale), `MAX_ROUNDS = 10` esaurito, o interruzione dell'utente.

2.3 Registro strumenti neutrale rispetto al fornitore

Gli strumenti sono definiti una sola volta in JSON-Schema e tradotti a runtime:

Famiglia	Formato richiesto
Anthropic	`{name, description, input_schema}`
Gemini	`[{functionDeclarations: [...]}]`
OpenAI-compatibile	`{type:"function", function:{...}}`

Lo stesso vale per i messaggi: i turni con chiamate a strumenti vengono serializzati nella forma nativa di ciascun fornitore e conservati in un campo `_native`, così una conversazione resta coerente anche cambiando modello.

2.4 Nessun nome di modello cablato

Un nome di modello scritto nel codice è una bomba a orologeria: funziona finché il fornitore non ritira quel modello, e allora l'applicazione si ferma con un 404 che l'utente non può risolvere. È successo due volte in questo progetto:

Fornitore	Errore	Quando
Google	`models/gemini-1.5-flash is not found for API version v1beta`	ritiro da `v1beta`
Groq	`The model llama-3.3-70b-versatile does not exist or you do not have access to it`	deprecato il 17/06/2026

Il problema è strutturale su alcuni servizi: gli id dei modelli gratuiti cambiano di continuo, per costruzione. (È il motivo per cui OpenRouter è stato poi tolto dall'elenco, vedi 2.4.)

La correzione non è scrivere il nome nuovo — sarebbe la stessa bomba con la miccia più lunga — ma **togliere il nome** e chiederlo all'API, che sa quali modelli esistono *per quella chiave*. Il che risolve anche l'ambiguità del messaggio di Groq: «non esiste» **oppure** «non ci hai accesso» sono casi diversi, e un elenco per chiave li distingue.

Ogni configurazione con `modelliCandidati` parte da `model: null` e viene risolta a runtime. Le due famiglie hanno strategie diverse perché i loro nomi lo sono:

Gemini — punteggio per versione. I nomi seguono uno schema regolare (`gemini-<major>.<minor>-<famiglia>`), quindi si può ordinare: versione più recente, famiglia `flash`, con penalità per varianti sperimentali, `preview` e `-lite`; scartati i modelli senza `streamGenerateContent` e quelli non conversazionali (embedding, immagini, audio nativo, Live API). Quando uscirà Gemini 3.0 verrà scelto da solo.

Famiglia OpenAI — i candidati in ordine di preferenza. Qui i nomi non sono confrontabili fra loro (`openai/gpt-oss-120b` contro `qwen/qwen3.6-27b`), e un punteggio automatico sceglierebbe male. Si interroga `GET /models` e si prende **il primo candidato ancora esistente**: l'ordine della lista *è* la preferenza, e l'elenco serve a saltare quelli spariti. Solo se nessun candidato sopravvive si ricorre a un punteggio generico sui modelli disponibili — che scarta trascrizione, sintesi vocale, embedding e classificatori, e a parità d'altro preferisce il modello più grande.

Tre livelli di riserva in entrambi i casi: elenco dei modelli → sonda dei candidati → primo candidato, lasciando parlare l'errore vero della chiamata di generazione invece di inventarne uno.

La scelta resta in cache sette giorni per fornitore, legata a un'impronta FNV-1a a 32 bit della chiave (chiavi diverse vedono cataloghi diversi; la chiave in chiaro non viene mai duplicata). Se il modello viene ritirato *mentre* è in cache, il `404` — o un `400` il cui testo parla di modello, come fanno alcuni fornitori — invalida la cache, ririsolve e ritenta **una volta sola**: un flag impedisce il ciclo infinito quando anche il modello nuovo fallisce.

Anthropic resta l'eccezione voluta: i suoi modelli sono a pagamento, scelti esplicitamente dall'utente e con deprecazioni annunciate con largo anticipo.

I quattro servizi gratuiti, scelti uno per uno

La qualità gratuita non è tutta uguale, e la differenza conta: un modello da 8 miliardi di parametri non regge un problema di chimica fisica in più passaggi. L'elenco è stato **ridotto**, non allungato — due servizi sono stati tolti perché peggioravano la risposta invece di migliorarla.

Servizio	Modelli	Il ruolo
Groq	GPT-OSS 120B, Qwen3	Il cavallo da tiro: 131K di contesto, ~30 richieste/minuto
Google Gemini	Gemini Flash	Fino a 1M di contesto: documenti e PDF interi
Z.AI GLM	GLM-4.7-Flash	Gratuito **senza scadenza**: la riserva che resta quando i crediti finiscono

Tolti. *GitHub Models*: c'era, ed è stato tolto perché GitHub lo ha **ritirato del tutto il 30 luglio 2026** — playground, catalogo e API di inferenza spenti per tutti. Era stato aggiunto due giorni prima sulla base di pagine che descrivevano il servizio ancora attivo: la lezione è che verificare *che una cosa esista* non è verificare *che sia ancora viva*, e la ricerca da fare è «<nome> retired / shutdown / deprecated <anno>». Il guasto non si è fermato alla voce in tendina: il rango che gli era stato assegnato lo rendeva anche la scelta automatica della Modalità Nucleo.

Mistral: qualità media e piano gratuito che richiede il consenso all'uso dei dati per l'addestramento — un prezzo che non vale la pena pagare per materiale di tesi non pubblicato, quando esistono cinque alternative senza quella clausola. *OpenRouter*: modelli gratuiti piccoli e id che cambiano di continuo, inadatti a un agente che concatena dieci chiamate a strumenti. *Cerebras*: dall'agosto 2026 niente più piano senza carta, e contesto gratuito limitato a 8K — Spectra manda 32 definizioni di strumenti oltre alla cronologia, non ci sta.

Nessuno dei cinque eguaglia un modello di frontiera a pagamento sui problemi più difficili; Z.AI GLM ci va vicino, al prezzo di tetti di richieste bassi. Per questo la scelta resta dell'utente, invece di imporre un solo servizio.

La frontiera, a pagamento

I gratuiti bastano per studiare; per un problema difficile no. Il criterio di scelta è stato **GPQA Diamond** — domande di livello dottorato in fisica, chimica e biologia — perché è l'unico banco che misura ciò che quest'app chiede davvero.

Servizio	Perché c'è
GPT-5.6	94,6% su GPQA Diamond: il punteggio più alto oggi disponibile. ~4 $/M in ingresso
Gemini 3 Pro	Oltre 1M di contesto, e **la stessa chiave** del Gemini gratuito. ~2 $/M
DeepSeek V4	Ragionamento di fascia alta a ~0,66 $/M: il miglior rapporto qualità/prezzo
Grok 4.6	Primo su *tool calling* agentico e con il minor tasso di allucinazione — esattamente il profilo che serve a un agente che concatena strumenti. ~2 $/M

Servizio	Perché c'è
Claude (Fable 5.1, Opus 5, Sonnet 5)	Una sola chiave per tutti e tre; gli unici con il sandbox Python in Modalità Nucleo

Due configurazioni possono usare **la stessa chiave**: i quattro Claude sono un solo account Anthropic, e Gemini Flash e Gemini 3 Pro una sola chiave di AI Studio. `chiaveCondivisaCon` le fa puntare alla stessa voce, così l'utente la incolla una volta. Chi la cancella la cancella per tutti i gemelli — lasciarne una la farebbe riapparire sull'altro.

Il caso dei due Gemini ha richiesto di generalizzare il punteggio: condividono famiglia, endpoint e chiave ma vogliono modelli **opposti**. La preferenza è ora dichiarata in configurazione (`/flash/` contro `/pro/`) e vince sul resto del punteggio. Senza, la configurazione gratuita avrebbe scelto `gemini-3-pro`: una generazione più avanti, quindi con punteggio più alto, ma **a pagamento** — un modello che la chiave gratuita non può usare.

Nello stesso passaggio è emersa una seconda cosa: il numero minore nel nome è **opzionale**. Dalla generazione 3 Google lo ha tolto (`gemini-3-pro`, non `gemini-3.0-pro`), e la regex che pretendeva il punto valutava i modelli *più nuovi* come alias generici — quindi perdevano contro i vecchi, e l'aggiornamento automatico descritto in 2.4 non sarebbe mai scattato.

L'audit dei fornitori, e perché va rifatto

Il sistema si ripara da solo sui **nomi dei modelli** — interroga il catalogo e sceglie fra ciò che esiste. Non si ripara su un **servizio morto**: quello va verificato a mano, e il ritiro di GitHub Models ha mostrato cosa costa non farlo.

Verifica di settembre 2026, fornitore per fornitore:

Fornitore	Esito
Groq	Attivo. `llama-3.3-70b-versatile` **non è più servito** da agosto 2026 e `llama-3.1-8b-instant` è deprecato: tolti dai candidati, restano i sostituti indicati da Groq
Google Gemini	Attivo
NVIDIA NIM	Rimosso 09/2026: attivo, ma senza header CORS non è chiamabile da una pagina web
Z.AI	Attivo, GLM-4.7-Flash e 4.5-Flash ancora gratuiti
OpenAI, DeepSeek, Anthropic	Attivi
xAI	Attivo, ma i candidati erano di **due generazioni** precedenti: il vertice è `grok-4.6` dal 12/08/2026

Due trappole registrate perché non si ripetano. La prima: non mettere un modello a pagamento fra i candidati di una configurazione gratuita — vale per `glm-5.x` su Z.AI come valeva per `gemini-3-pro`, e la configurazione sceglierebbe qualcosa che la chiave gratuita non può usare. La seconda: un fornitore morto con un rango alto non rompe solo la voce in tendina, **diventa la scelta automatica** della Modalità Nucleo.

Quando un fornitore non risponde

Un `fetch` che fallisce **prima** di ricevere una risposta significa una di tre cose: rete assente, il servizio non accetta chiamate dirette dal browser (CORS), oppure il servizio non esiste più. Il browser, per ragioni di sicurezza, non permette di distinguere quale — l'errore è volutamente generico.

Prima questo fermava il turno. Ma se le tre cause non si distinguono, la risposta giusta è la stessa per tutte e tre: **passare a un altro fornitore** per cui l'utente ha una chiave, esattamente come per la quota esaurita. È il comportamento che avrebbe mascherato del tutto il ritiro di GitHub Models, invece di lasciare ogni richiesta bloccata su un servizio spento.

Restano fuori dalla riserva gli errori che si ripeterebbero identici ovunque — una chiave sbagliata, una richiesta malformata: provarli su tutti i fornitori per poi riportare l'ultimo errore a caso nasconderebbe la causa vera.

Togliere un servizio dall'elenco è un'operazione che rompe le cose, e merita una nota: chi lo aveva selezionato ha quel nome salvato in `localStorage`, e senza validazione si ritroverebbe `PROVIDERS[undefined]` — Spectra non si aprirebbe più. `getSavedProvider()` valida sempre contro il registro e ripiega su un servizio esistente.

2.5 Il proxy opzionale — Spectra senza chiave

La chiave API vive nel browser dell'utente: senza backend non c'e' alternativa, ed e' il limite dichiarato apertamente in fondo a questo documento. Ogni visitatore deve procurarsi la propria chiave gratuita e incollarla.

`proxy/spectra-proxy.js` e' la via d'uscita, e sta **fuori** dalla pagina: un Worker Cloudflare (piano gratuito) che tiene le chiavi come segreti lato server. Quando e' configurato, il browser non contiene nessuna chiave e chi apre BioSpecInfo usa Spectra senza inserire niente.

Non e' un semplice inoltro:

- **Il percorso passa tale e quale.** La rotta e' `/<fornitore>/<percorso>`, e

il resto viene inoltrato come arriva: il proxy non conosce i formati delle API e continua a funzionare quando Spectra cambia modello o parametri.

- **Piu' chiavi per fornitore, con subentro automatico.** Ogni segreto e' una

lista separata da virgole; su `429` (quota) o `401/403` (chiave revocata) si passa alla successiva nella stessa richiesta. Su `400` no: e' un errore nella richiesta e si ripeterebbe identico.

- **Lo streaming non viene bufferizzato**: il corpo della risposta passa

direttamente, altrimenti le risposte comparirebbero tutte insieme alla fine.

- **Non e' un relay aperto.** Credenziali che arrivano dal client

(`Authorization`, `x-api-key`, `?key=`) vengono scartate e sostituite: nessuno puo' usare il proxy per far viaggiare una chiave propria.

- **Tre freni** contro l'abuso: elenco di origini ammesse, limite per IP e

tetto giornaliero.

Lato Spectra l'innesto e' in un punto solo. `GET /stato` dice quali fornitori il proxy copre davvero, cosi' non viene offerto come "senza chiave" un modello per cui manca il segreto; `buildRequest` instrada al proxy **e omette del tutto l'autenticazione**; `chiaveDaUsare()` restituisce un segnaposto perche' i controlli "manca la chiave" non blocchino l'invio — segnaposto che non lascia mai il browser.

Le due strade convivono: senza `PROXY_URL` tutto funziona come prima, e per i fornitori non coperti dal proxy Spectra continua a usare la chiave locale.

2.6 Ricominciare da capo

Tutto ciò che Spectra ricorda vive in `localStorage`: chat, cronologia, chiavi API, memoria persistente, schede di ripasso. Un pulsante "cancella cronologia" che togliesse solo le conversazioni non farebbe ripartire da capo — alla riapertura si ritroverebbe lo stesso stato.

Il punto delicato è che le chiavi hanno **due formati**: la mappa attuale (`bsi_api_keys`, una chiave per fornitore) e il formato singolo delle versioni precedenti (`bsi_api_key`). `getKeysMap()` migra automaticamente il secondo nel primo, quindi cancellare solo la mappa **fa riapparire la chiave vecchia** al primo accesso successivo. Vanno tolti entrambi — e con loro le cache dei modelli risolti, che senza chiave non hanno più significato.

Per questo l'elenco di ciò che l'applicazione scrive sta in un punto solo, `DATI_CANCELLABILI`, diviso in cinque gruppi con etichetta e descrizione. Il pannello di conferma si costruisce da lì: mostra quante voci esistono davvero per ciascun gruppo, disattiva quelli già vuoti e richiede un secondo clic esplicito. Spuntati di partenza solo chat e chiavi; memoria, ripasso e progressi del resto dell'app restano da scegliere a mano, perché cancellano lavoro che non c'entra con il ripartire da capo della conversazione.

Restano fuori di proposito la licenza PRO, il periodo di prova e l'identità del dispositivo: sono dati che l'utente non si aspetta di perdere svuotando una chat, e una licenza non si cancella per sbaglio. Il pannello lo dichiara.

Dopo la cancellazione viene azzerato anche lo stato in memoria del processo — elenco dei fornitori coperti dal proxy, modello risolto per ciascun provider — altrimenti resterebbe valido fino al ricaricamento della pagina, e Spectra continuerebbe a usare un modello scelto con una chiave che non esiste più.

2.7 Far reggere un carico vero alle chiavi gratuite

I piani gratuiti hanno due limiti diversi, e vanno affrontati in modo diverso.

Il tetto per richiesta. GitHub Models accetta 8.000 token in ingresso. Il costo fisso di Spectra è ~8.150 — 2.009 di prompt di sistema più **6.128 di sole definizioni degli strumenti** — quindi quel servizio non partirebbe nemmeno: fallirebbe prima che l'utente scriva una parola. La misura è stata fatta prima di scrivere il rimedio, e ha cambiato la soluzione.

Il rimedio non è tagliare la cronologia — è tagliare **il costo fisso**. Gli strumenti si pagano ad ogni turno, la conversazione è il contenuto: fra i due, è il costo fisso a doversi stringere per primo. `adattaAlBudget()` seleziona i **strumenti pertinenti** alla domanda invece di tutti e 32,

poi accorcia la cronologia con quel che resta.

La pertinenza si misura su una mappa di parole chiave tenuta **fuori** dal registro degli strumenti: le parole con cui la gente *chiede* una cosa non sono quelle con cui lo strumento è *documentato*. La descrizione di `astrofisica` parla di Wien e Stefan-Boltzmann; l'utente scrive «che temperatura ha questa nebulosa». Confrontando la domanda con la sola descrizione, su una domanda di astrochimica veniva scelto `farmacocinetica` — misurato, non ipotizzato. Il confronto è per radice (primi cinque caratteri) perché l'italiano flette: «nebulosa» e «nebulose» devono coincidere.

I limiti di frequenza. Un `429` sui piani gratuiti è quasi sempre il limite *al minuto*, non la quota esaurita: aspettare pochi secondi lo risolve. Il fornitore lo dice in `Retry-After`, oppure nel testo dell'errore («*Please try again in 7.5s*»); quando non lo dice si usa una crescita esponenziale con un po' di casualità, per non far ripartire tutte le schede aperte nello stesso istante. Se l'attesa richiesta è assurda non si aspetta affatto: meglio dire «quota finita» che lasciare l'utente davanti a una clessidra per un'ora.

La riserva fra fornitori. Quando una quota è davvero esaurita, il turno viene rifatto su un altro servizio per cui l'utente ha una chiave. È il motivo per cui più chiavi gratuite messe insieme reggono un carico che nessuna reggerebbe da sola.

Due vincoli non negoziabili:

- Si riparte dai messaggi **originali**, non dalla cronologia a metà. I turni

con chiamate a strumenti sono salvati nel formato nativo del fornitore precedente e non si possono passare a un altro. Si perde il lavoro del turno, si guadagna una risposta corretta.

- La riserva usa **solo servizi gratuiti**. Passare da solo a uno a pagamento

spenderebbe i soldi dell'utente senza che li abbia stanziati; se il servizio scelto era a pagamento resta comunque il primo tentativo.

E si passa oltre **soltanto** per quota esaurita. Un `401` o una richiesta malformata si ripeterebbero identici su ogni fornitore: provarli tutti per poi riportare l'ultimo errore a caso nasconderebbe la causa vera.

2.8 Modalità Nucleo, e dire cosa sta succedendo

La modalità. Un interruttore che cambia quattro cose insieme, perché nessuna delle quattro da sola sposterebbe l'ago: passa alla configurazione più capace fra quelle per cui esiste una chiave, porta i giri del ciclo agentico da 10 a 16 (una catena lunga di strumenti si esauriva a metà), chiede la profondità di ragionamento massima ai modelli che la espongono, e aggiunge al prompt l'istruzione di lavorare per passaggi verificati invece che per risposta rapida.

Il cambio di modello viene **dichiarato in chat**, e se il nuovo è a pagamento lo dice: passare di nascosto a un servizio a consumo significherebbe spendere i soldi dell'utente senza dirglielo. Per lo stesso motivo è un interruttore e non il comportamento predefinito — costa più quota, e su un servizio a pagamento costa denaro.

Dire cosa sta succedendo. Fra l'invio e la prima parola della risposta possono passare parecchi secondi: il modello ragiona, invoca strumenti, aspetta la rete. Prima l'utente vedeva una pausa muta e concludeva che si fosse bloccata.

L'intestazione ha ora un nucleo atomico i cui orbitali cambiano velocità e colore con lo stato — lento e ciano a riposo, rapido quando ragiona, ambra quando sta usando uno strumento — accompagnato da una riga che dice la stessa cosa a parole, per chi non guarda l'animazione. Gli stati si agganciano ai callback che il ciclo agentico già emetteva: non è stato aggiunto nessun meccanismo, solo reso visibile quello che c'era.

Tre difetti di impaginazione corretti nello stesso passaggio, tutti trovati misurando invece che guardando:

- Il campo di scrittura divideva la riga con quattro pulsanti e restava largo

156 px: il testo del segnaposto andava a capo e veniva **tagliato**. Ora il testo ha una riga propria e i comandi stanno sotto. La lunghezza del segnaposto è stata scelta provandola a 360 px di larghezza, non a occhio.

- Il titolo «Spectra — il copilota AI di BioSpecInfo» non entrava e si

troncava a metà. Resta il nome; il resto è diventato la riga di stato, che ora ha un uso.

- Gli orbitali del nucleo, ruotando, uscivano dal riquadro e finivano sopra il

nome.

2.9 Le capacità del fornitore, usate davvero

Tre cose che l'API di Anthropic offre e che non erano sfruttate. Verificate contro il riferimento ufficiale, non a memoria — due delle tre hanno un vincolo che a memoria non si sarebbe indovinato.

La cache del prompt. Il costo fisso di ogni richiesta è di circa 8.150 token: 2.000 di prompt di sistema più 6.128 di definizioni degli strumenti, identici ad ogni turno. Marcandoli si pagano una volta e poi si rileggono a una frazione del prezzo.

Il vincolo è che la cache funziona per *prefisso*: un solo byte diverso all'inizio la invalida tutta. Il prompt di Spectra però contiene anche la memoria dell'utente e il contesto della domanda, che cambiano ad ogni messaggio. Concatenandoli in un'unica stringa la cache **non avrebbe mai preso**. Ora il prompt viaggia in due blocchi — stabile e marcato, volatile e no — e il marcatore va anche sull'ultimo strumento, perché l'ordine di composizione è `tools` → `system` → `messages`.

Il sandbox Python, e perché esclude la ricerca web. `code_execution` dà al modello un ambiente con sympy, numpy, scipy e matplotlib: integrazione numerica, algebra simbolica, diagonalizzazione di matrici — cose che i 32 risolutori non coprono perché non si possono prevedere tutte.

Ma la ricerca web di nuova generazione **porta già dentro un ambiente di esecuzione** (filtra i risultati scrivendo codice), e dichiarare anche `code_execution` ne creerebbe un secondo, confondendo il modello. Sono alternative, non complementari. La scelta segue la

Modalità Nucleo: normalmente ricerca web e lettura di pagine, in Nucleo il sandbox — perché quando si chiede il massimo della potenza il problema è di calcolo, non di documentazione. Un test verifica che i due non compaiano mai insieme.

La lettura di pagine. `web_fetch` affianca la ricerca: se l'utente incolla il link di un articolo, Spectra lo legge invece di limitarsi a cercarne il titolo.

I risultati del sandbox arrivano come `bash_code_execution_tool_result` e `text_editor_code_execution_tool_result` — non come un generico `code_execution_tool_result` — e vanno conservati nel turno dell'assistente come i blocchi della ricerca web, altrimenti un turno messo in pausa dal server non si può riprendere.

---

3. I 35 strumenti

L'elenco è quello **misurato** in `window.BSI_AI_TOOLS` sull'applicazione in esecuzione, non trascritto a memoria: questo documento ne dichiarava 32 quando erano già 35, e tre — `analizza_molecola`, `disegna_molecola`, `mostra_spettri` — non comparivano affatto.

Area	Strumenti
Calcolo generale	`calcola`, `risolvi_equazione`, `analisi_dati`
Chimica generale	`bilancia_equazione`, `stechiometria`, `massa_molecolare`, `converti_unita`, `costante_fisica`
Chimica fisica	`termodinamica`, `equilibrio_acido_base`, `cinetica`, `gas_e_soluzioni`, `elettrochimica`
Struttura e spettri	`spettroscopia`, `quantistica_e_spettroscopia`, `cristallografia`, `analizza_molecola`, `disegna_molecola`, `mostra_spettri`
Scienze della vita	`biochimica`, `farmacocinetica`, `valuta_druglikeness`
Fisica	`astrofisica`, `nucleare`, `statistica_inferenziale`
Banche dati esterne	`cerca_pubchem` (NIH), `cerca_letteratura` (PubMed)
Dati interni	`cerca_nel_database` (9 dataset), `cerca_molecola`
Controllo app	`naviga_sezione` (95 sezioni), `apri_strumento` (12 laboratori), `stato_app`
Memoria	`ricorda`, `ricordi`
Animazioni	`apri_animazione` (6 meccanismi di reazione)

> **La ricerca web non è fra questi trentacinque**, ed è bene distinguerla. I trentacinque sono eseguiti **dall'applicazione**, nel browser dell'utente. Sui fornitori che li offrono, l'agente aggiunge allo schema due strumenti eseguiti **dal fornitore** — `web_search` e `web_fetch`, con un tetto di usi per turno. Contarli insieme agli altri li farebbe sembrare disponibili sempre, mentre dipendono dal fornitore scelto.

3.1 Dataset interni esposti

297 reazioni di sintesi · 118 elementi · 67 amminoacidi · 263 farmaci · 63 patologie · 46 strategie retrosintetiche · 36 interazioni farmacologiche · 29 vie metaboliche · 29 potenziali redox.

I farmaci e le strategie sono i due numeri che `tools/verifica-affermazioni.js` misura sull'applicazione in esecuzione a ogni batteria: qui erano rimasti a 143 e 39 mentre l'app ne contava 178 e 46.

Sono i dati curati dall'autore per l'applicazione: l'agente li consulta come fonte primaria e ha istruzione di **segnalare le discrepanze** fra database e propria memoria invece di appianarle silenziosamente.

---

4. Decisioni ingegneristiche rilevanti

Questa sezione documenta le scelte non ovvie e il motivo per cui sono state prese. È la parte del documento che descrive il ragionamento progettuale.

4.1 Motore di calcolo senza `eval()`

Problema. Servire calcoli arbitrari di chimica fisica richiede un valutatore di espressioni. `eval()` lo risolverebbe in poche righe.

Perché è stato escluso. L'agente legge contenuti esterni non fidati (risultati PubChem, abstract PubMed, pagine web). Un'iniezione in quei contenuti potrebbe indurre il modello a emettere un'espressione ostile; con `eval()` quella stringa verrebbe eseguita nel contesto della pagina, **dove risiede la chiave API dell'utente**.

Soluzione. Parser a discesa ricorsiva con insieme *chiuso* di 27 funzioni e costanti. Non ha accesso all'ambito globale: può solo fare aritmetica. Verificato con 10 tentativi di evasione (`window.localStorage`, `constructor`, `this`, `fetch(...)`, chiamate a funzioni non in whitelist), tutti respinti, anche eseguendo il test in un browser reale.

4.2 Conservazione dei blocchi di ragionamento

Sui modelli con ragionamento adattivo i blocchi `thinking` fanno parte del turno assistente e devono essere **rimandati indietro identici, firma crittografica inclusa**, nel giro successivo di *tool use*. Scartarli fa rifiutare la richiesta.

L'implementazione accumula `thinking_delta` e `signature_delta` dallo stream e li reinserisce **in testa** al turno ricostruito. Verificato in test d'integrazione: firma preservata, ordine corretto.

4.3 Ripresa dei turni sospesi (`pause_turn`)

La ricerca web è uno strumento eseguito sui server del fornitore. Quando il suo ciclo interno esaurisce le iterazioni disponibili, la risposta termina con `stop_reason: "pause_turn"`.

Il turno va ripreso **rimandando indietro il turno assistente così com'è, senza aggiungere alcun messaggio utente**: è il server a riconoscere il blocco `server_tool_use` in coda e a riprendere da lì. Aggiungere un "continua" spezzerebbe il meccanismo.

4.4 Configurazione per modello, non globale

I quattro livelli Claude non accettano gli stessi parametri:

Modello	`effort`	Ricerca web	Note
Fable 5.1	`xhigh`	sì (8)	fallback su rifiuto; ragionamento sempre attivo
Opus 5	`high`	sì (6)	predefinito
Sonnet 5	`high`	sì (5)	
Haiku 4.5	**non supportato**	**non supportata**	inviarli produce HTTP 400

Su tutti i modelli recenti `temperature`, `top_p` e `budget_tokens` sono stati rimossi dall'API e la loro presenza causa un errore: nessuno dei quattro li invia. Ogni parametro è quindi condizionato al fornitore, non impostato globalmente.

4.5 Ingresso multimodale e lettura dei materiali

L'agente accetta immagini, PDF e file di testo, anche molti insieme, trattati come pagine consecutive di un unico documento. Tre vincoli hanno guidato l'implementazione:

- **Le immagini vengono ridimensionate a 2000 px** e ricompresse in JPEG prima

dell'invio. Una foto da telefono passa da circa 1 MB a 440 KB: senza questo passaggio una singola immagine avvicinerebbe il tetto di richiesta e moltiplicherebbe il costo in token. 2000 px sono il minimo per leggere una grafia minuta — a 1600 px la scrittura piccola diventava illeggibile.

- **Il tetto di pagine dei PDF dipende dal modello**, non è una costante: 600

pagine per i modelli con finestra da 1M, 100 per quelli da 200K. Il conteggio avviene leggendo gli oggetti `/Type /Page` nei byte del file, senza librerie.

- **Nella cronologia va una miniatura da 160 px**, non l'immagine inviata.

Salvare quella grande (440 KB l'una) riempiva `localStorage` in una decina di foto, e a quota piena ogni scrittura successiva fallisce in silenzio.

Sei azioni rapide trasformano il materiale in un prodotto di studio (riassunto, mappa concettuale, schema, flashcard, domande d'esame, trascrizione) e due lo traducono. Per le trascrizioni l'agente ha istruzione di scrivere `[illeggibile]` invece di indovinare: un termine chimico inventato è peggio di una lacuna dichiarata.

4.6 Renderer di grafi proprio, senza CDN

Le mappe concettuali sono prodotte dal modello come diagrammi Mermaid e disegnate da un renderer scritto nell'applicazione (~190 righe): livelli per cammino più lungo dalle radici con interruzione dei cicli, riordino per baricentro dei predecessori su tre passate per ridurre gli incroci, output SVG con archi di Bézier.

La scelta di non caricare Mermaid da CDN nasce dalla natura offline-first dell'app: una mappa deve potersi disegnare anche senza rete. Le sintassi non coperte vengono riconosciute e lasciate come blocco di codice leggibile, invece di essere disegnate male.

4.7 Comando vocale con parola di attivazione

Ascolto continuo attivabile dall'utente: la frase «Hey Spectra» seguita dal comando lo invia automaticamente. Due dettagli non ovvi:

- il riconoscimento vocale del browser **termina da solo** dopo qualche secondo

di silenzio e va riavviato, altrimenti l'ascolto muore alla prima pausa;

- vengono esaminate **tutte le alternative di trascrizione**, non solo la

prima, e sono accettate le varianti fonetiche più comuni: il riconoscimento italiano rende "Spectra" in molti modi diversi.

Il rifiuto del permesso al microfono interrompe il ciclo invece di ritentare all'infinito. Il termine "spettroscopia", frequentissimo in questo dominio, è stato verificato come non attivante.

4.8 Guardia sui risultati non finiti

Un audit sistematico ha rivelato che, con input degeneri legittimi (lunghezza d'onda nulla, volume nullo, emivita nulla, transizione fra livelli identici), otto risolutori restituivano `ok: true` con `Infinity` o `NaN` fra i campi.

È il guasto più insidioso in questo contesto: un'eccezione si nota, un valore formalmente valido ma privo di senso viene riportato all'utente come corretto. La correzione è un controllo unico nel punto in cui transitano tutti i risultati: se un campo numerico non è finito, il risultato diventa un errore esplicito che nomina i campi e indica la causa probabile. Vale anche per gli strumenti aggiunti in futuro.

4.9 Degradazione controllata

- **Modelli senza *function calling* affidabile** (alcuni gratuiti): al primo

errore riconducibile agli strumenti, la richiesta viene ripetuta una volta in modalità solo testo, invece di fallire.

- **Timeout di inattività (45 s)**, azzerato ad ogni byte ricevuto: una risposta

lunga non viene troncata, ma una connessione morta viene segnalata subito con un messaggio esplicito.

- **Quota di `localStorage`**: lo storico è limitato (30 conversazioni, 100

messaggi) con potatura progressiva. Senza questo limite il superamento della quota faceva fallire *in silenzio* ogni scrittura successiva, **compreso il salvataggio della chiave API**.

- **Rifiuto dei classificatori** (`stop_reason: "refusal"`, HTTP 200): rilevato

e comunicato, invece di presentarsi come risposta vuota.

---

5. Verifica

La verifica è stata condotta su tre livelli, con esiti registrati.

5.1 Correttezza numerica — confronto con valori di riferimento

Ambito	Caso	Atteso	Ottenuto
Bilanciamento	`KMnO₄ + HCl → …`	2:16:2:2:8:5	✓ esatto
Massa molecolare	`K₄[Fe(CN)₆]` (annidata)	368,35	368,345
Massa molecolare	`CuSO₄·5H₂O` (idrato)	249,68	249,677
Equilibrio	acido acetico 0,1 M, K_a $1{,}8\cdot10^{-5}$	pH 2,874	2,875
Termodinamica	ΔG (N_2O_4/NO_2) a 298 K	+4,79 kJ/mol	✓
Cinetica	E_a da k(300 K) e k(320 K)	55,3 kJ/mol	55,33
Gas reali	van der Waals CO_2	scostamento −10 %	−10,13 %
Spettroscopia	M+2 del bromo	97,3 %	✓
Biochimica	glicilglicina	132,12 Da	✓
Farmacocinetica	t½ da V_d 50 L, Cl 5 L/h	6,93 h	✓
Astrofisica	picco di emissione solare (Wien)	501,5 nm	501,52
Astrofisica	velocità di fuga terrestre	11,19 km/s	11,186
Nucleare	energia di legame ^{56}Fe	8,79 MeV/nucleone	8,790
Statistica	t critico, 95 %, df = 4	2,7764	✓
Cristallografia	densità del rame (fcc)	8,96 g/cm³	8,935

I p-value non sono tabulati: sono calcolati con la funzione beta incompleta regolarizzata (frazione continua di Lentz) e la gamma incompleta.

5.2 Robustezza — casi che devono fallire

Verificato che vengano **respinti**: equazioni non bilanciabili, formule con parentesi non bilanciate o elementi inesistenti, specie assenti dall'equazione, tipi di database non validi, e i 10 tentativi di iniezione nel motore di calcolo.

5.3 Integrazione — browser reale

Test automatizzati con Chromium *headless* su tutte le 14 pagine dell'applicazione: nessun errore JavaScript, nessuna risorsa mancante. Percorse una per una le 95 sezioni di `index.html` e i 18 tab della sezione Astrochimica. Ispezionato il corpo delle richieste per i quattro livelli Claude, e simulato un ciclo completo con ricerca web, sospensione e ripresa del turno.

6. Riepilogo quantitativo

Metrica	Valore
Righe del componente	6.254
Strumenti	32
Configurazioni di modello	10 (3 gratuite)
Aree scientifiche coperte	13
Record nei dataset interni esposti	oltre 800
Giri massimi del ciclo agentico	10
Cronologia inviata al modello	40 turni
Dipendenze runtime	nessuna

---

7. Limiti dichiarati

Documentati per onestà tecnica; sono scelte consapevoli, non omissioni.

- **La chiave API risiede nel browser** — a meno di pubblicare il proxy.

Nella configurazione predefinita la chiave non lascia il dispositivo, ma è accessibile a chi ha accesso a quel browser, e ogni utente deve procurarsi la propria. Il Worker della sezione 2.5 toglie questo limite spostando le chiavi su un server, al prezzo di un componente da mantenere fuori dalla pagina: è una scelta di distribuzione, non un requisito.

- **Gli strumenti di rete dipendono dalla disponibilità dei servizi.** PubChem

e PubMed sono interrogati direttamente dal browser; in caso di irraggiungibilità lo strumento restituisce un errore esplicito e l'agente ha istruzione di dichiararlo, non di sostituirlo con una stima.

- **I risolutori applicano modelli semplificati** dove la letteratura ne

prevede di più raffinati (per esempio Woodward–Fieser per l'UV, o le rese in ATP con rapporti P/O medi). Ogni strumento dichiara il metodo usato nel proprio risultato.

- **La verifica è funzionale e numerica**, non formale: non esiste prova di

correttezza dei risolutori, ma un insieme di casi di riferimento tratti dalla letteratura didattica.

Documento redatto a corredo del dossier tecnico BioSpecInfo. Vedi anche: `01-Software-Architecture-Document.md`, `02-Verification-Validation-Report.md`, `03-Security-Privacy-Compliance.md`.

Accuratezza scientifica e provenienza dei dati — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Autore	Samuele Pio Provenzano
Versione descritta	`bsi-v200`
Scopo	Documentare come vengono generati i dati scientifici mostrati dall'applicazione, con quale metodo sono verificati, e quali sono i limiti dichiarati.

> **Perché questo documento esiste.** Un'applicazione didattica di chimica può essere ineccepibile dal punto di vista informatico e sbagliata dal punto di vista scientifico, e l'utente non ha modo di accorgersene: un peso molecolare plausibile, uno spettro dall'aspetto convincente e una struttura sbagliata si presentano tutti allo stesso modo. Questo documento descrive i controlli che rendono quell'errore **visibile e automatico**, e dichiara ciò che resta fuori dalla loro portata.

---

1. Principio: ogni dato scientifico deve poter essere contraddetto

Il criterio adottato in tutta l'applicazione è che un dato scientifico non si consideri verificato perché *sembra* giusto, ma perché **esiste un secondo dato indipendente che lo confermerebbe o lo smentirebbe**. Dove questo secondo dato non esiste, il limite viene dichiarato invece di essere colmato con una stima presentata come misura.

Tre applicazioni concrete di questo principio:

Ambito	Prima affermazione	Seconda affermazione indipendente	Controllo
Banca dati farmaci	struttura (SMILES)	peso molecolare di letteratura	il peso ricalcolato dalla struttura deve coincidere
Spettri IR / NMR	gruppi funzionali riconosciuti	struttura molecolare via RDKit	i gruppi si cercano sul grafo, non sul testo dello SMILES
Costanti fisiche	valore tabulato	simbolo e unità di misura	la ricerca rifiuta le corrispondenze ambigue invece di sceglierne una

---

2. Banca dati farmacologica — 263 voci

2.1 Il metodo di verifica

Ogni voce dichiara **due** proprietà della stessa molecola: la struttura, in notazione SMILES, e il peso molecolare. Sono affermazioni indipendenti — la prima descrive la connettività atomica, la seconda è un numero preso dalla letteratura — e quindi confrontabili:

```
peso calcolato da RDKit sulla struttura   ↔   peso dichiarato in tabella
```

Uno scarto superiore a **0,6 u** (tolleranza che copre gli arrotondamenti e le piccole differenze fra tabelle di masse atomiche, ma non un errore reale, che vale decine di unità) indica che almeno una delle due affermazioni è sbagliata.

Il controllo è automatizzato (`tools/verifica-farmaci.js`, eseguibile da chiunque disponga del repository) e non richiede giudizio umano: non chiede di fidarsi di nessuna fonte, confronta due numeri.

2.2 Cosa ha trovato

Applicato alle 143 voci preesistenti, il controllo ha individuato **89 errori**. La distribuzione è significativa: i **pesi molecolari erano corretti** — presi dalla letteratura — mentre le **strutture erano sbagliate**.

Farmaco	Struttura dichiarata
Ketamina	un pirrolidinone non correlato
Morfina	struttura non corrispondente (in due voci distinte)
Lorazepam	privo dell'ossidrile in posizione 3, che ne definisce l'identità
Atorvastatina	privo del gruppo anilidico
Testosterone	privo del doppio legame Δ^4 dell'anello A

Più **7 farmaci duplicati** in categorie diverse, con dati talvolta divergenti fra le due copie.

2.3 Lo scarto come strumento diagnostico

Il valore numerico della differenza non è solo un allarme: indica **quale** frammento manca o è di troppo.

Δ (u)	Interpretazione	Caso reale
28,05	C_2H_4	ketamina, fentanil, morfina
16,00	un atomo di ossigeno	tetraciclina (un ossidrile in più)
14,02	un gruppo CH_2	doxiciclina
2,02	un doppio legame mancante	testosterone (Δ^4)
0,98	citosina ⟷ uracile	**sofosbuvir**

L'ultimo caso merita una nota: lo scarto di 0,98 u corrisponde esattamente alla sostituzione di un gruppo −NH con un atomo di ossigeno, cioè alla differenza fra citosina e uracile. Il sofosbuvir è un analogo dell'**uridina**, e la struttura proposta conteneva la base sbagliata. Il numero ha indicato dove guardare.

2.4 Stato attuale e limiti dichiarati

Farmaci in banca dati	**263** (erano 233)
Con struttura leggibile	**242**
Con provenienza esterna verificata	**36**
Difetti	**0**
Voci senza struttura, con ragione dichiarata	**21**

La verifica è **conforme**: nessuna voce presenta una struttura che contraddica il proprio peso molecolare. Le 21 deviazioni sono voci **prive di struttura**, ciascuna registrata con il proprio motivo in `evidence/deviazioni-note.json` e riportata nel rapporto di verifica.

Difetto contro deviazione. Il controllo distingue due situazioni che non vanno confuse:

- una struttura che **contraddice** il proprio peso è un difetto, e fa fallire

la verifica;

- una struttura **assente** può essere una deviazione accettabile — ma solo se

registrata esplicitamente, con il motivo. Una voce priva di struttura e **non** registrata fa fallire la verifica come qualunque altro difetto.

Registrare una deviazione è quindi una decisione consapevole, tracciata in git e visibile nel rapporto, non un modo per silenziare un controllo.

E va anche **revocata** quando non serve più. Dalla versione `bsi-v188` il banco fallisce anche nel caso opposto: una voce elencata nel registro che **ha** una struttura verificata è un permesso rimasto acceso a vuoto, e domani coprirebbe in silenzio una struttura sbagliata messa al suo posto.

2.4-bis Quattro voci uscite dal registro

Delle sei voci senza struttura, quattro sono state chiuse riprendendo la struttura da **ChEMBL** — una fonte indipendente da questo progetto — e verificandola con lo stesso confronto struttura ⟷ peso molecolare che vale per tutte le altre.

Voce	ChEMBL	Formula	Peso dichiarato	Peso calcolato
Digossina	`CHEMBL1751`	$C_{41}H_{64}O_{14}$	780,94	780,95
Vincristina	`CHEMBL90555`	$C_{46}H_{56}N_4O_{10}$	824,96	824,97
Tacrolimus topico (Protopic)	`CHEMBL269732`	$C_{44}H_{69}NO_{12}$	804,02	804,03
Tacrolimus sistemico (Prograf)	`CHEMBL269732`	$C_{44}H_{69}NO_{12}$	804,02	804,03

Il motivo registrato per la digossina diceva «ogni struttura provata si discosta di 14-30 u dal peso di letteratura»: il peso dichiarato era giusto, erano le strutture provate a essere sbagliate. Che il confronto sia reale e non compiacente si vede aggiungendo un solo carbonio alla struttura corretta — il banco segnala **Δ 14,04** e fallisce.

Le due che restano non sono molecole singole, e nessuna notazione SMILES le rappresenta:

- **Ivermectina** è una miscela di omologhi: almeno 80 % di B1a ($C_{48}H_{74}O_{14}$) e non più del 20 % di B1b ($C_{47}H_{72}O_{14}$). Il peso dichiarato, 875,10, è quello del solo componente B1a. Mostrarne la struttura farebbe passare l'omologo maggioritario per l'intero farmaco.
- **Artemetere/Lumefantrina (Coartem)** è l'associazione di due principi attivi distinti.

In entrambi i casi non manca una struttura: non ce n'è una sola da mostrare.

2.4-ter Trentasei voci con la provenienza scritta nel dato (bsi-v188)

Il confronto struttura ⟷ peso è necessario ma non sufficiente, e si è visto dove cede. Le trentasei voci aggiunte in `bsi-v188` portano nel dato stesso l'identificativo ChEMBL e la formula del record (`chembl:`, `formula:`), e per ognuna la formula viene **ricostruita contando gli atomi del grafo** che RDKit legge dallo SMILES — idrogeni impliciti compresi — e confrontata con quella dichiarata da ChEMBL: **36 su 36 coincidono**. I dati grezzi stanno in `tools/dati/farmaci_v187.json`.

Dove il peso non vedeva. Sei di quelle voci esistevano già, scritte prima che la provenienza esterna fosse un requisito. Confrontando le due versioni con la **chiave InChI** che ChEMBL dichiara, tre si sono rivelate una molecola diversa:

Voce	chiave della versione precedente	chiave dichiarata da ChEMBL
Lenalidomide	`XKAYAFBLGLCWSY`	`GOTYRUGSSMKFNF`
Palbociclib	`PSRAOXPOEYRNJN`	`AHJRHEGDXFFMBM`
Aripiprazolo	`ZGXTVHHDFYTYTL`	`CEUORZQYGODEFX`

Erano **isomeri**: stessa formula bruta, stesso peso molecolare, posizione diversa di un azoto o di un carbonile. Il controllo struttura ⟷ peso — che esisteva, girava a ogni rilascio e passava — non poteva vederlo, perché un isomero pesa esattamente uguale. Serviva un testimone che guardasse la **connettività**, non la massa. Le tre voci sono state rimosse e sostituite da quelle verificate; le altre tre differivano solo nella scrittura canonica dello SMILES e descrivevano la stessa molecola.

Due strutture rifiutate. Nella stessa raccolta, l'**ivermectina** ha `structure_type NONE` nel proprio record ChEMBL — conferma esterna del limite già dichiarato in §2.4-bis — e la **semaglutide** è registrata come proteina, `structure_type SEQ`. Entrambe si sarebbero potute scrivere a memoria. È esattamente ciò che non si fa: una struttura senza fonte entra nel sito indistinguibile da una verificata, e da lì in poi nessun controllo le distingue.

Due nomi tenuti con il nome del record. La ricerca per nome di ChEMBL è a corrispondenza parziale: «morphine» ha restituito l'**apomorfina** e «levothyroxine» la **liotironina** (T3), trovata fra i sinonimi. Le voci sono state tenute con il nome che il record porta, non con quello della domanda: il contrario avrebbe messo una scheda clinica sbagliata sopra una struttura giusta — l'errore più difficile da trovare, perché ogni controllo automatico sulla struttura continuerebbe a passare.

Gruppo	Voci	Natura
D-01 — non sono molecole singole	2	ivermectina (miscela di omologhi B1a/B1b) e artemetere/lumefantrina (associazione di due principi attivi). Erano sei: le altre quattro sono state chiuse con la struttura da ChEMBL, vedi §2.4-bis.
D-02 — non rappresentabili in SMILES	19	anticorpi monoclonali, proteine e peptidi (trastuzumab, pembrolizumab, insulina, semaglutide, ciclosporina A…). L'assenza è corretta, non una lacuna.

> La scelta è deliberata: un dato sbagliato sostituito da un altro dato sbagliato non è un progresso, e l'assenza dichiarata è preferibile a una presenza non verificata.

---

3. Predizione spettrale

3.1 Riconoscimento strutturale, non testuale

I gruppi funzionali sono individuati con **pattern SMARTS applicati al grafo molecolare** tramite RDKit (`bsi-spettri.js`), non con espressioni regolari applicate alla stringa SMILES.

La differenza non è stilistica. Il metodo testuale precedente presentava due difetti strutturali:

1. **Esclusività.** I controlli erano organizzati in catene `else if`, per cui

veniva riconosciuto **un solo gruppo funzionale per molecola**.

1. **Cecità alla struttura.** Una regex legge caratteri, non legami.

Il caso emblematico è l'**acido acetilsalicilico** (`CC(=O)Oc1ccccc1C(=O)O`), che contiene sia un estere sia un acido carbossilico. Il riconoscimento testuale identificava solo l'estere; lo spettro IR risultante era privo della banda O–H allargata a ~3000 cm^{-1} e del C=O acido a ~1690 cm^{-1} — le due bande che identificano la molecola — e lo spettro ^1H NMR era privo del segnale del protone carbossilico a 11,6 ppm.

Il riconoscimento su grafo è **additivo per costruzione**: una molecola può presentare tutti i gruppi che possiede.

3.2 Convenzioni di rappresentazione

Le convenzioni degli assi non sono uniformi fra le tecniche, e sono verificate automaticamente (`test_assi.js`, `test_assi_canvas.js`) estraendo le etichette delle tacche dal disegno prodotto:

Tecnica	Ascissa	Verso	Ordinata
IR	numero d'onda (cm^{-1})	decrescente, 4000 → 400	**trasmittanza %**, bande verso il basso
^1H NMR	δ (ppm)	decrescente, 14 → 0	intensità, riferimento TMS a 0
UV-Vis	λ (nm)	crescente, 200 → 400	assorbanza
MS	m/z	crescente	abbondanza relativa

3.3 Determinismo

Nessuno spettro contiene componenti casuali. Le versioni precedenti aggiungevano rumore pseudo-casuale alla traccia a ogni ridisegno, con la conseguenza che la stessa molecola produceva ogni volta uno spettro diverso e due spettri non erano confrontabili. Un profilo calcolato deve essere **riproducibile**: dove serve un fondo realistico si usa una funzione deterministica del numero d'onda.

Analogamente, nel predittore NMR le altezze dei picchi erano generate casualmente. In uno spettro ^1H l'altezza **è** l'informazione quantitativa — l'integrazione — e inventarla è peggio che ometterla. Attualmente:

- dove il numero di protoni è noto, l'altezza è l'integrazione reale e

l'etichetta la riporta (es. `7,26 (4H)`);

- dove la predizione è un **intervallo** (es. «6,5–8,5 ppm»), si disegna la

fascia dell'intervallo con un segno sul valore tipico e altezza uniforme: il grafico indica *dove* cade il segnale senza affermare quanto sia intenso.

3.4 Effetti chimici modellati

Effetto	Implementazione
Coniugazione del carbonile	abbassamento di ~25 cm^{-1} applicato **al singolo carbonile**, non globalmente
Estere fenolico	innalzamento a ~1760 cm^{-1}
Intensità C–H	proporzionale al numero di gruppi CH_2/CH_3 presenti
Sostituzione dell'anello aromatico	bande "fuori dal piano" 900–690 cm^{-1} distinte per orto/meta/para/mono
Doppietto di Fermi delle aldeidi	bande a 2820 e 2720 cm^{-1}
Bande ammidiche I e II	1655 e 1550 cm^{-1}

Verifica: l'acido acetilsalicilico produce estere arilico a 1760 cm^{-1} e acido coniugato a 1690 cm^{-1} — i due valori che lo identificano — mentre acetofenone (chetone coniugato, 1690) e acetato di etile (estere non coniugato, 1735) confermano che l'effetto è applicato in modo selettivo e non indiscriminato.

3.5 Limiti dichiarati

> Gli spettri prodotti sono **predizioni basate su tabelle di gruppi funzionali**, non spettri misurati né simulazioni quantomeccaniche. Indicano dove cadono le bande dei gruppi presenti, con posizioni e intensità tabulate. **Non sostituiscono un database sperimentale** (SDBS, NIST) né una misura strumentale, e il grafico lo dichiara.

In particolare non sono modellati: accoppiamenti spin-spin del secondo ordine, effetti di solvente e concentrazione, bande di combinazione e sovratoni, polimorfismo allo stato solido.

3-bis. Chemioinformatica e modelli QSAR

3-bis.1 Da dove vengono i metodi

Nessuno dei metodi della sezione **Chemioinformatica** è inventato per l'occasione. Sono quelli in uso, con i riferimenti che li definiscono.

Metodo	Riferimento	Come è implementato qui
Coefficiente di Tanimoto	Rogers & Tanimoto, 1960	Bit in comune su bit in unione, con la convenzione 0/0 = 1 dichiarata nel codice
Impronte di Morgan / ECFP	Rogers & Hahn, *J. Chem. Inf. Model.* 50 (2010) 742	Da RDKit MinimalLib, raggio 2, 2048 bit
Chiavi MACCS	Durant *et al.*, *J. Chem. Inf. Comput. Sci.* 42 (2002) 1273	Da RDKit MinimalLib, 166 chiavi
Regola dei 5	Lipinski *et al.*, *Adv. Drug Deliv. Rev.* 23 (1997) 3	Sui descrittori RDKit, con il conteggio delle violazioni
Filtro di Veber	Veber *et al.*, *J. Med. Chem.* 45 (2002) 2615	Legami ruotabili ≤ 10, TPSA ≤ 140 Å²
QED	Bickerton *et al.*, *Nat. Chem.* 4 (2012) 90	**Ricalcolato**: MinimalLib espone 43 descrittori ma non il QED. I parametri delle funzioni desiderabilità sono quelli dell'articolo
Raggruppamento di Butina	Butina, *J. Chem. Inf. Comput. Sci.* 39 (1999) 747	Soglia sulla distanza di Tanimoto, capifila scelti per numero di vicini
Scheletri di Bemis–Murcko	Bemis & Murcko, *J. Med. Chem.* 39 (1996) 2887	Sistemi ad anello più i legami che li collegano
PAINS	Baell & Holloway, *J. Med. Chem.* 53 (2010) 2719	Sottoinsieme dei pattern SMARTS, con il motivo della segnalazione
Allarmi di Brenk	Brenk *et al.*, *ChemMedChem* 3 (2008) 435	Idem
SALI	Guha & Van Drie, *J. Chem. Inf. Model.* 48 (2008) 646	Δattività / (1 − Tanimoto), sulle coppie sopra la soglia di similarità

3-bis.2 Le quattro cose che rendono un QSAR onesto

Un modello QSAR è facilissimo da far sembrare buono. Le quattro precauzioni qui sotto sono quelle che distinguono un numero da una misura, e sono tutte verificate dal banco `test_cheminfo`.

Divisione per scheletro. Con una divisione casuale, analoghi stretti della stessa serie finiscono da entrambe le parti: il modello ritrova ciò che ha già visto. La divisione per scheletro raggruppa le molecole per scheletro di Bemis–Murcko e assegna **scheletri interi** a una sola parte. L'R^2 che ne esce è più basso — ed è quello che sopravvive alla molecola nuova.

Modello nullo per rimescolamento. Lo stesso modello viene riaddestrato otto volte su etichette mescolate. Se il punteggio vero non supera il migliore dei sosia, il modello non ha imparato nulla di trasferibile, e il pannello lo dichiara con quelle parole. È il controllo che manca più spesso: senza di esso, un R^2 di 0,4 su trenta molecole è indistinguibile dal caso.

Dominio di applicabilità. Una predizione su una molecola lontana da tutto ciò che il modello ha visto è un'estrapolazione, non una predizione. La distanza dal vicino più prossimo nell'insieme di addestramento è mostrata accanto a ogni valore predetto.

Validazione incrociata raggruppata per scheletro. Una divisione sola, su un insieme piccolo, è un numero rumoroso: su 28 molecole al 25 % l'insieme di prova ne contiene sette, e spostarne una muove l'R^2 di decimi. Le pieghe raggruppano per scheletro — nessuno scheletro sta in due pieghe — e il risultato viene dato come **media ± deviazione**, non come cifra singola.

> **Quanto conti, misurato sull'esempio dei 28 inibitori.** La divisione singola dà R^2 **0,861**. La validazione a cinque pieghe dà **0,605 ± 0,452**, con le singole pieghe a 0,861 · 0,538 · 0,795 · **−0,150** · 0,980. La divisione singola aveva pescato la piega fortunata. Le due cifre insieme dicono quello che nessuna delle due dice da sola: il modello funziona, ma dipende sensibilmente da quali molecole gli capitano in addestramento — e su un insieme di ventotto molecole è esattamente ciò che ci si deve aspettare.

3-bis.3 Limiti dichiarati

Limite	Conseguenza
Gli insiemi di esempio sono **piccoli** (28 e 40 molecole)	Servono a mostrare il metodo, non a produrre un modello utilizzabile. Su insiemi così, il modello nullo è la sola difesa seria — ed è il motivo per cui c'è
Il modello è **lineare nel kernel** (kernel ridge, kernel di Tanimoto) o logistico	Nessuna rete neurale, nessun gradient boosting: sono metodi che su poche centinaia di molecole non danno un vantaggio dimostrabile, e il costo sarebbe un modello che non si può ispezionare
I **PAINS non sono una condanna**	Un pattern PAINS segnala che il composto è stato spesso un falso positivo in saggi di fluorescenza, non che sia inattivo. Il pannello lo scrive accanto a ogni segnalazione
La **PCA è sui descrittori**, non sulle impronte	Sulle impronte binarie la PCA è poco informativa; i carichi sui descrittori si leggono, e sono mostrati

Limite	Conseguenza
Non c'è **ricerca di sottostruttura su larga scala**	Il banco di lavoro è dimensionato per gli insiemi che si incollano a mano, non per una libreria da milioni di composti

---

3-bis.4 Due superfici, due ruoli — e un numero che era falso

L'applicazione ha **due** luoghi in cui si fa chemioinformatica, e servono a cose diverse. La distinzione è scritta qui perché un valutatore che apra prima la pagina sbagliata si farebbe l'idea sbagliata.

	`rdkit_lab.html` — «RDKit Lab»	Sezione Chemioinformatica
A chi serve	Chi studia: una molecola alla volta, per vedere cosa RDKit sa dirne	Chi lavora: un insieme di molecole, per ricavarne un modello difendibile
Ingresso	Uno SMILES	Un insieme incollato, con l'attività misurata
QSAR	Stime **empiriche** dai descrittori, dichiarate tali nel pannello	Modello **addestrato** sui dati forniti, con divisione per scheletro, validazione incrociata e controllo nullo
Uscita	Quello che si vede a schermo	CSV e rapporto di metodo

> **Un numero che era falso, e come se n'è accorto il banco.** Il pannello «Pharma Pro» del laboratorio confrontava la molecola con otto farmaci di riferimento e mostrava una percentuale di similarità. Quella percentuale non veniva da un fingerprint: veniva da **otto bit di descrittori a soglia** — «ha anelli aromatici», «HBA > 4», «peso fra 200 e 500» — di cui si calcolava il Tanimoto. Misurato: **caffeina contro metformina dava 0,75**, mentre su fingerprint di Morgan vale **0,024**. Due molecole senza frammenti in comune, mostrate al 75 %, su una pagina che si chiama «RDKit Lab» e dove RDKit era già caricato.
>
> Accanto a quello, tre delle otto impronte di riferimento erano **scritte a mano** e avevano il bit «aromatico» sbagliato (caffeina, che ha due anelli aromatici, morfina e amoxicillina che ne hanno uno), e lo **SMILES dell'omeprazolo non era omeprazolo**: `COc1ccc2[nH]c(=S)cc2c1OC` è una struttura che RDKit rifiuta.
>
> Corretto tutto: il confronto usa il fingerprint di Morgan con il Tanimoto del motore condiviso, le impronte si calcolano invece di essere battute a tastiera, e l'omeprazolo è l'omeprazolo (massa monoisotopica 345,11, tre anelli aromatici — verificate). Le due funzioni della similarità finta sono state **rimosse**, non scavalcate: lasciarle in giro le avrebbe rimesse in uso alla prima modifica, e il banco controlla che non esistano più.

Una sola implementazione di Tanimoto. Il laboratorio ne aveva una propria, che concordava con quella del motore sui casi normali ma rispondeva 0 dove il motore risponde 1 (il caso di due impronte vuote). Ora la pagina carica `bsi-cheminfo.js` e chiama quello: `test_cheminfo` confronta le due strade su tutte e 28 le coppie di riferimento e pretende scarto **esattamente zero**.

3-ter. Il testo alternativo delle figure non si inventa

La guida di biochimica porta **98 figure**. Sostituendola, il banco di accessibilità l'ha bocciata: nessuna aveva un attributo `alt`.

La via comoda era `alt=""`, e il banco l'avrebbe accettata — il commento nel suo codice dice, correttamente, che per un'immagine **decorativa** è la cosa giusta: dice allo screen reader di saltarla. Ma queste sono figure di biochimica, cioè informazione. Marcarle come decorative avrebbe fatto passare il controllo dicendo a chi usa uno screen reader di ignorare 98 diagrammi: un verde vuoto, della stessa famiglia del «contrasto 0» che non copriva i pannelli nuovi (§3-bis) e del 100 % di copertura che sommava gli intervalli sbagliati.

La via onesta era a disposizione: ogni immagine sta dentro un `<figure>` con la sua `<figcaption>`, e **le didascalie le ha scritte l'autore del documento**.

Figure	62, per 98 immagini
Figure con una sola immagine	26 — l'alt è la didascalia, meno l'etichetta «Figura N.»
Figure con due pannelli	36
Didascalie di coppia divise fra i due pannelli	**33 su 36**
Coppie con didascalia condivisa	3 — non si dividono in modo inequivocabile

Le 36 didascalie di coppia distinguono tutte i due pannelli con «a sinistra» e «a destra»: dove la divisione è inequivocabile, ogni immagine riceve la propria metà della frase. Dove non lo è, le due immagini condividono la didascalia intera — verboso da ascoltare, ma **non può essere sbagliato**, e una descrizione sbagliata di una figura è peggio di una verbosa.

> **Perché non tentare una divisione più aggressiva.** Si potevano spremere anche le 3 restanti con espressioni regolari più elastiche. Una divisione che sbaglia produce la frase «questa immagine mostra X» accanto a un'immagine che mostra Y, e lo screen reader la legge con la stessa sicurezza di quelle giuste. Il ripiego è rumoroso; l'errore è invisibile.

Misurato dopo l'intervento: **98 su 98 con testo alternativo, nessuno vuoto**, contrasto **0** su 1 897 elementi di testo.

3-quater. Da prototipo a strumento di lavoro

Le funzioni descritte in §3-bis bastavano a dire qualcosa di difendibile su un insieme di molecole. Non bastavano a **lavorarci**: un gruppo di chemioinformatica, il primo giorno, chiede di cercare una sottostruttura su tutto l'insieme, di vedere la tabella SAR, e di sapere che cosa succede all'attività se sostituisce un cloro con un metile.

3-quater.1 Che cosa la libreria aveva, e che cosa non ha

Prima di progettare si è **sondata** MinimalLib 2025.03.4 all'esecuzione, invece di assumere. Sondare per primo ha cambiato il progetto: tre delle capacità qui sotto non erano state usate perché non si sapeva che ci fossero.

C'è	A cosa serve
`SubstructLibrary`	Ricerca per sottostruttura con pattern fingerprint: scarta in blocco le molecole che non possono corrispondere, invece di provare l'isomorfismo su ognuna
`get_rxn` · `Reaction.run_reactants`	Trasformazioni chimiche. È ciò che permette di **tagliare** un legame, e da lì vengono coppie corrispondenti e tabella SAR
`generate_aligned_coords`	Allineare le molecole a un nucleo comune
`get_molblock` · `get_v3Kmolblock`	Esportazione **SDF**, il formato con cui si scambiano insiemi fra gruppi e programmi

Non c'è	Conseguenza dichiarata
`cleanup`, `neutralize`	La standardizzazione si ferma al frammento maggiore e allo SMILES canonico: **le cariche non vengono neutralizzate**
`canonical_tautomer`	Due tautomeri scritti in modo diverso restano due voci distinte
`FragmentOnBonds`, `RWMol`	Nessuna modifica diretta delle molecole: il taglio passa per una reazione
Sottostruttura massima comune (MCS)	Il nucleo di una tabella SAR va **fornito**, non viene dedotto. Il pannello offre otto nuclei pronti e il campo libero

3-quater.2 Il taglio, e perché ne servono due regole

Una reazione SMARTS spezza un legame e marca i due capi con un atomo fittizio. Verificato all'esecuzione: `CCOc1ccccc1` dà tre tagli con la regola fine (CH_3–CH_2, CH_2–O, O–arile) e due con quella grossa; il benzene e il metano non ne danno nessuno.

Regola	Taglia	Quando serve
fine (predefinita)	Qualunque legame singolo aciclico, **compresi i sostituenti terminali** Cl, CH_3, OH, F	Tabella SAR e trasformazioni fini (Cl → CH_3)
grossa (stile BRICS)	Esclude gli atomi terminali	Insiemi grandi, dove i tagli fini sono troppi; produce parti variabili più grandi («clorofenile → fenile»)

> **La regola fine è nata da un difetto.** La prima stesura usava solo quella grossa, e la tabella SAR della serie di prova **perdeva il cloro sull'anello**: trovava il sostituente sull'azoto e mancava proprio la colonna che interessa, perché il cloro è terminale. La regola fine è un soprainsieme della grossa: ogni taglio che la grossa trova, lo trova anche la fine.

3-quater.3 Coppie molecolari corrispondenti

Due molecole formano una coppia quando, tagliando un legame in ciascuna, restano con lo **stesso contesto** e due parti variabili diverse. La differenza di attività è attribuita a quella sostituzione e a nient'altro — ed è il motivo per cui questa lettura batte una correlazione su tutto l'insieme, che media su molecole diverse per dieci cose.

Verificata su una serie **costruita con effetti noti**: se l'analisi non li ritrova non è utilizzabile su dati veri.

Trasformazione	Attesa	Trovata	Coppie concordanti
clorofenile → fenile	−1,0	**−1,0**	2 su 2
metilfenile → fenile	−0,5	**−0,5**	2 su 2
metile → cloro	+0,5	**+0,5**	2 su 2
etilammide → propilammide	+0,2	**+0,2**	3 su 3

Sui 28 inibitori dell'esempio, presi da ChEMBL: **449 coppie, 98 trasformazioni** viste almeno due volte, 199 viste una sola, in 133 ms. In cima alla lista **CF_3 → SO_2NH_2 con Δ mediano −2,65** su due coppie concordanti.

Due scelte dichiarate:

- la **direzione è normalizzata** sull'ordine alfabetico delle due parti.

Senza di essa lo stesso effetto comparirebbe due volte, con mediane opposte, e nessuna delle due avrebbe il numero di coppie giusto;

- le trasformazioni viste **meno di due volte** sono escluse dalla tabella e

contate a parte. Una trasformazione vista una volta con Δ = +3 non è una scoperta: è un aneddoto, e metterla in cima farebbe prendere decisioni su di essa.

> **La colonna «concordi» è quella che decide se fidarsi.** Una mediana di +1,0 su tre coppie di cui una a −2,0 non è la stessa cosa di +1,0 su tre coppie tutte positive, e la mediana da sola non lo dice.

3-quater.4 La tabella SAR, e come si trova la posizione

Il punto difficile non è trovare i sostituenti: è sapere in quale **posizione** sta ognuno. Senza MCS e senza modifica delle molecole, la soluzione sfrutta una proprietà del taglio: **il frammento che contiene il nucleo porta l'atomo fittizio esattamente nel punto di attacco**. Facendo corrispondere il nucleo a quel frammento si scopre su quale suo atomo è appeso il fittizio, e quindi quale posizione occupa il sostituente.

Ne viene anche un filtro necessario e gratuito: se il fittizio **non** è attaccato a un atomo del nucleo, il taglio è avvenuto dentro un sostituente e quel pezzo non è il sostituente intero. Si scarta.

Sui 28 inibitori, con nucleo benzamidico: **20 molecole con il nucleo, 2 posizioni, 8 escluse**. Le escluse non compaiono in tabella: metterle con le celle vuote le farebbe sembrare parte della serie con sostituenti non trovati.

> **Un difetto di denominazione, e come si è manifestato.** `scaffoldMurcko()` restituiva il suo risultato in un campo chiamato `smiles`, ma quel valore **non è uno SMILES**: è un'impronta canonica dello scheletro, costruita perché due molecole con lo stesso scheletro producano la stessa stringa. Serve a raggruppare — pieghe, divisione per scaffold — non a interrogare. Il pannello SAR lo ha preso per uno SMILES e proponeva `6,6,6,6,7,…|0-13:1,…` come nucleo; la decomposizione rispondeva «nucleo non interpretabile».
>
> Il campo si chiama ora `chiave` e dichiara `eUnoSmiles: false`; `smiles` resta come alias. Il banco fissa che due scheletri uguali diano la stessa chiave, che scheletri diversi diano chiavi diverse, e che la chiave **non si rilegga come SMILES**. Il nome sbagliato di un campo è un difetto come un altro.

3-quater.5 Il rigore che un valutatore pretende

Cinque cose che un gruppo di modellistica mette in ogni rapporto, e la cui assenza è la prima obiezione in riunione.

	Metodo	Perché
Confronto fra modelli	Riferimento banale (predire la media), kNN su Tanimoto, regressione kernel — tutti sulle **stesse pieghe** raggruppate per scheletro	Confrontare un modello valutato su una divisione con un altro valutato su un'altra non è un confronto. E il riferimento banale sta in cima all'elenco, perché è il numero che ogni altro deve battere
Validazione annidata	Gli iperparametri (k, lambda) si scelgono su pieghe **interne all'addestramento**	Provarne dieci sull'insieme di prova e riportare il migliore gonfia il punteggio: quel numero non è una predizione, è il massimo di dieci tentativi
Intervalli conformi	Split-conformal, quantile $\lceil (n+1)(1-\alpha) \rceil$ dei residui assoluti su una porzione di calibrazione **divisa per scaffold**	«7,2» e «7,2 ± 0,3» sono due informazioni diverse, e solo la seconda dice se vale la pena sintetizzare. Una calibrazione su analoghi dell'addestramento darebbe intervalli troppo stretti proprio sulle molecole nuove
Arricchimento	EF a 1 %, 5 %, 10 % con il **massimo possibile** accanto; BEDROC (Truchon & Bayly, *J. Chem. Inf. Model.* 47 (2007) 488)	In uno screening non si guarda tutta la lista. Un ROC-AUC di 0,80 può nascondere una testa di lista senza un solo attivo, perché l'AUC premia anche l'ordine nella coda, che nessuno comprerà
Curva di apprendimento	Punteggio al variare della dimensione dell'addestramento, con l'insieme di prova **intero** a ogni punto	Risponde alla sola domanda che conta quando il modello è mediocre: serve più chimica o più dati?

Il verdetto del confronto è provato nei due versi, come il modello nullo. Su un segnale vero — attività proporzionale al numero di alogeni, su dieci nuclei distinti — la regressione kernel arriva a R^2 0,943 con margine 0,943 contro una dispersione di 0,035, e il verdetto è *supera*. Sulle **stesse molecole con etichette casuali** il migliore si ferma a −0,265, margine −0,034, e il verdetto è *non supera*. Una guardia che non scatta mai e una che scatta sempre sono lo stesso difetto.

L'arricchimento è verificato sui tre casi in cui il valore giusto si calcola a mano:

Ordinamento (2 attivi su 10)	AUC	EF@20 %	BEDROC
Perfetto: i due attivi in testa	1	**5** — ed è il massimo possibile	**1**
Pessimo: i due attivi in coda	0	0	**0**, non un negativo
Attivi ai ranghi 2 e 5	**0,75** = 12/16	**2,5** = 1/0,4	0,119

Il caso pessimo è quello che conta: senza di esso non si saprebbe se BEDROC tocca lo zero o scende sotto, e un BEDROC negativo è il segno che manca il termine additivo della formula.

3-quater.6 Limiti di questo blocco

Limite	Conseguenza
Le coppie crescono col **quadrato** delle molecole	Misurato: 6 molecole 45 ms, 40 molecole 227 ms con 1 255 coppie. Oltre 300 molecole il pannello rifiuta di partire e lo dice, invece di bloccare la pagina
Il taglio è **singolo**	Una coppia che differisce per due sostituzioni insieme non viene trovata. È il limite dello schema a taglio singolo, condiviso con gli strumenti standard
La stessa trasformazione compare in **più forme**	Tagli in posizioni diverse danno contesti diversi per la stessa modifica chimica (`*CC>>*CCC`, `*NCC>>*NCCC`, `*C(=O)NCC>>*C(=O)NCCC`). È ridondanza attesa nello schema a taglio singolo, non un errore
Le cariche **non** sono neutralizzate e i tautomeri non canonizzati	MinimalLib non lo permette (§3-quater.1). Un insieme che mescola forme neutre e ioniche della stessa molecola le conta come voci distinte
Il nucleo della tabella SAR va **fornito**	Nessun MCS disponibile. Il pannello non propone nulla automaticamente, perché una proposta sbagliata è peggio di nessuna proposta

3.6 Predizione NMR per atomo: la fonte e lo scarto misurato

Il predittore $^1H/^{13}C$ che assegna ogni segnale a un ATOMO (`bsi-nmr.js`) non contiene numeri. Le tabelle di stima stanno in **`bsi-pretsch.js`**, trascritte riga per riga da:

> E. Pretsch, P. Bühlmann, M. Badertscher, *Structure Determination of Organic Compounds — Tables of Spectral Data*, 4ª ed., Springer. §4.1 pp. 82-84 · §4.5 pp. 100-102 · §5.1 p. 170 · §5.2 pp. 178-179 · §5.3 p. 182 · §5.5 pp. 188-189

Stanno in un file separato perché si possano **controllare contro la pagina stampata** senza leggere il codice che le usa, e perché una riga sbagliata si corregga in un posto solo.

schema	formula	righe
benzeni monosostituiti, ^{13}C	$\delta = 128{,}5 + \Sigma\ Zi$	91
benzeni monosostituiti, ^{1}H	$\delta = 7{,}34 + \Sigma\ Zi$	66
etileni sostituiti, ^{1}H	$\delta = 5{,}25 + Zgem + Zcis + Ztrans$	42
alcani sostituiti, ^{1}H	$\delta = base(CH_3/CH_2/CH) + \Sigma Z\alpha + \Sigma Z\beta$	31
alchini terminali, ^{1}H	valore diretto per sostituente	30
alifatici, ^{13}C	$\delta = -2{,}3 + \Sigma\ Zi + \Sigma\ Sj$	24
correzioni steriche Sj	per grado del C osservato e dell'atomo α	4×4

Lo scarto è misurato, su molecole che non hanno scelto i parametri. Le molecole di prova sono divise in due insiemi: *taratura*, usate per scegliere valori e regole, e *validazione*, mai usate per quello. Il numero dichiarato nel pannello dell'applicazione è quello della validazione.

	scarto medio misurato
^{13}C, taratura (9 molecole)	0,71 ppm
^{13}C, validazione (22 molecole)	**0,92 ppm**
^{13}C, caso peggiore	4,9 ppm (cicloesanone)
^{1}H (14 molecole)	0,06 ppm
^{1}H, soli aromatici	0,03 ppm

Il banco (`test_nmr`, 56 controlli) pretende anche che la validazione resti **peggiore** della taratura: se diventassero uguali vorrebbe dire che una molecola è stata spostata da un insieme all'altro, e il numero non direbbe più niente.

Quello che resta sbagliato non è stato corretto a posteriori. Il difenile sbaglia il carbonio ipso di 4,6 ppm, perché l'incremento del fenile della tabella ($Z_1 = 8{,}1$) non lo descrive. Quel numero **non è stato ritoccato**: cambiare un valore trascritto perché una molecola di validazione non torna è esattamente il modo di rendere falso lo 0,92 ppm dichiarato sopra.

Limiti dichiarati. Non è un calcolo quantistico: non prevede gli effetti del solvente, non distingue conformeri (le correzioni conformazionali K della fonte sono trascritte ma valgono 0, perché la conformazione da uno SMILES non si ricava) e non fa NMR bidimensionale. Le costanti di accoppiamento vengono da una tabella di valori **tipici** per relazione geometrica, non calcolate. Gli **eteroaromatici sostituiti** non hanno, in questa fonte, una tabella di incrementi per posizione come l'hanno i benzeni: restano la previsione meno affidabile del modulo, e lo strumento di elucidazione lo dichiara abbassando la fiducia quando ne incontra uno.

3.7 NMR bidimensionale: COSY, HSQC, HMBC

`bsi-nmr2d.js` non contiene spostamenti: li chiede al predittore per atomo e costruisce le correlazioni **camminando il grafo**. È per questo che si poteva fare solo dopo l'arrivo delle tabelle di Pretsch: servono lo spostamento di *ogni* protone e di *ogni* carbonio, più l'indice dell'atomo che li porta.

mappa	regola topologica
HSQC	protone e carbonio a **un** legame (^{1}J); segno invertito sui CH_2
COSY	due protoni a **tre** legami (H–C–C–H), escluso fra equivalenti
HMBC	protone e carboni a **due o tre** legami (^{2}J, ^{3}J)

Gli idrogeni **scambiabili** (O–H, N–H) non danno macchie fuori diagonale: si scambiano col solvente troppo in fretta perché l'accoppiamento si veda. Sulla diagonale del COSY ci sono, perché sono protoni come gli altri.

Verificato contando i cammini (`test_nmr2d` , 40 controlli), nei due versi: l'esafluorobenzene non dà macchie HSQC, il benzene non ne dà fuori diagonale nel COSY, il metano non ne dà in HMBC.

Limiti dichiarati. Non è una simulazione dell'esperimento: non ci sono intensità calcolate, artefatti, accoppiamento residuo né dipendenza dal tempo di miscelamento. In un HMBC vero alcune correlazioni a due legami non si vedono; qui ci sono tutte. Gli spostamenti ereditano l'incertezza misurata del predittore (0,9 ppm sul ^{13}C, 0,06 sul ^{1}H).

3.8 Geometria 3D: costruita dal grafo, non scaricata

`bsi-geom3d.js` costruisce le coordinate con la **geometria delle distanze**: una matrice di limiti da legami, angoli di valenza, diagonali d'anello e contatti di van der Waals; posizioni iniziali casuali ma **riproducibili**, perché il generatore è seminato dallo SMILES; correzione iterativa delle violazioni. I sistemi aromatici si proiettano sul loro piano e le terne lineari si raddrizzano ruotando i rami come corpi rigidi, perché quelle due cose una distanza da sola non le impone.

Gli indici sono quelli del predittore NMR, e gli idrogeni si aggiungono in coda: è la condizione senza la quale il collegamento picco↔atomo↔3D illuminerebbe l'atomo sbagliato. Il visore 3D che pesca da PubChem non poteva essere usato per questo — la numerazione di un SDF non ha nessun rapporto con quella di RDKit.

Misurato contro i valori noti (`test_geom3d` , 29 controlli): C–C 1,54 Å, C=C 1,34, C≡C 1,20, aromatico 1,39, C–H 1,09; angolo tetraedrico 109,5°, aromatico 120,0°, alchino 180,0°. **Nei due versi**: benzene piano (0,00 Å), cicloesano **non** piano (0,34). Su molecole vere il residuo resta sotto 0,03 Å fino a una trentina di atomi e arriva a 0,31 Å su una molecola di 76.

Limiti dichiarati. Non è un campo di forze e non minimizza un'energia: minimizza violazioni geometriche. Non sceglie il conformero più stabile e non tratta la stereochimica — un centro R e il suo enantiomero escono uguali.

3.9 Dal documento allo svolgimento: che cosa è dedotto e che cosa è letto

`bsi-documento.js` apre il file; `bsi-quesito.js` riconosce i dati nel testo; `bsi-elucida.js` svolge. Sono tre passi separati **di proposito**, perché sbagliano in modi diversi e confonderli nasconde l'errore: se il riconoscimento legge «1715» come banda IR quando era una massa, lo svolgimento che segue è impeccabile e la conclusione è sbagliata. Per questo il pannello mostra **la tabella dei dati letti sopra lo svolgimento**, con scritto accanto a ognuno da dove viene.

Il riconoscimento è per sezione, non per numero. Una banda IR a 1738 e una massa a 150 sono entrambe numeri: distinguerli dal solo valore è impossibile. Si cerca l'etichetta della tecnica (IR, MS, ^{1}H NMR, ^{13}C NMR) e si legge quello che viene dopo, fino alla fine del capoverso o all'etichetta successiva.

Quello che non fa. Non c'è nessun modello linguistico: ci sono espressioni regolari e un motore di regole spettroscopiche. Se il testo scrive i dati in una forma non prevista, i dati **non** vengono letti — e il modulo lo dichiara invece di svolgere a metà. Lo svolgimento arriva fino a dove arrivano le regole: gruppi funzionali compatibili, frammenti, conteggi. **La struttura finale non viene proposta**, perché proporla vorrebbe dire indovinare; una struttura la si può scrivere e il programma la **confronta** con i dati, dicendo quali segnali tornano e quali no.

Non c'è riconoscimento ottico dei caratteri. Una pagina scansionata contiene pixel, non lettere: il testo non si legge, e il modulo lo dichiara invece di restituire una stringa vuota come se il documento fosse vuoto. Le pagine restano visibili, e da lì passa l'estrazione della traccia dai pixel.

3.10 I due punti in cui il predittore cede, misurati

Dichiarare «scarto medio 1,0 ppm» e fermarsi lì nasconde la cosa che serve davvero sapere: *dove* sbaglia. Sono due posti, e si conoscono per misura.

a) Gli eteroaromatici sostituiti — manca la tabella, non il metodo

Per i benzeni la fonte dà gli incrementi di sostituente per posizione; per furano, tiofene, pirrolo e piridina **non li dà**. Un eteroaromatico sostituito riceve quindi i valori del composto *non* sostituito, e i suoi carboni β risultano tutti uguali quando nella realtà non lo sono.

Misurato sul quesito del 23/04/2024 (struttura `O=CC(=Cc1ccco1)CSC`):

osservato	previsto	scarto	che carbonio è
192,04	192,0	0,04	CHO
150,77	150,0	0,77	C α del furano, sostituito
146,05	142,7	**3,35**	C α del furano
135,55	135,0	0,55	=C
135,05	135,0	0,05	=C
118,55	109,6	**8,95**	C β del furano
112,97	109,6	**3,37**	C β del furano

osservato	previsto	scarto	che carbonio è
26,67	34,0	**7,33**	CH_2 fra C=C e S
15,72	15,6	0,12	S–CH_3

I due carboni β escono **identici** (109,6) perché senza incrementi di posizione non c'è niente che li distingua. **Si è provato** a trasferire gli incrementi del benzene all'anello eteroaromatico: sul carbonio *ipso* funziona (151,6 contro 150,77 misurato), ma sui β **peggiora** — il C3 passerebbe da 3,4 a 5,7 ppm di scarto. Non è stato fatto: una generalizzazione che peggiora dove il problema sta non è una generalizzazione.

b) Un carbonio sp³ con DUE sostituenti in α

Gli incrementi sono ricavati da composti **mono**-sostituiti. Sommandone due sullo stesso carbonio si conta due volte un effetto che in realtà satura, e la previsione esce alta. Misurato due volte, stessa entità e stesso verso:

molecola	carbonio	osservato	previsto	scarto
acetato di benzile	OCH_2 (O estereo + anello)	66,3	73,8	**7,5**
il quesito qui sopra	CH_2 (C=C + S)	26,7	34,0	**7,3**

È il caso peggiore dichiarato del modulo, e l'acetato di benzile è entrato nell'insieme di validazione proprio per tenerlo misurato.

Lo strumento di confronto lo dice. Quando una struttura proposta contiene un carbonio del genere, la fiducia scende da «alta» a «media» e compare il motivo: *un segnale che non torna su quel carbonio può essere un limite del predittore, non una prova contro la struttura*. Senza quell'avviso, un punteggio basso su una struttura **giusta** verrebbe letto come una smentita — ed è esattamente quello che succedeva all'acetato di benzile, che prende 42 su 100 pur essendo la risposta.

3.11 Che cosa si è potuto consultare, e che cosa no

Una tabella di incrementi per gli eteroaromatici sostituiti chiuderebbe il punto debole descritto sopra. **Dalla macchina in cui questo progetto si compila non è raggiungibile nessuna fonte primaria**, e la cosa è stata verificata, non supposta:

via	esito
`curl` verso qualunque dominio	la politica di rete consente **solo** i registri di pacchetti (npm, PyPI, crates, Go) — lo dichiara il proxy stesso
NIST WebBook, SDBS (AIST), PubChem, SpectraBase	**403**, bloccati
CSIRO, RSC, ACS, Springer, Wikipedia, LibreTexts	**403**, bloccati
le tabelle di Hans Reich (`organicchemistrydata.org`, `chem.wisc.edu`)	**403**, bloccati

via	esito
NMRShiftDB2 (`nmrshiftdb.nmr.uni-koeln.de`)	**403**, bloccato
PubMed (via il suo servizio)	nessun risultato: indicizza il biomedico, non la chimica pura
npm e PyPI	nessun pacchetto ridistribuisce una banca dati NMR di riferimento: `nmr-predictor` la **scarica** da GitHub, che è bloccato; `nmr-processing` non la contiene

Quello che la ricerca testuale ha potuto accertare è *dove* sta il dato, e questo è un risultato utile anche senza averlo in mano:

> M. T. W. Hearn, «Carbon-13 chemical shifts in some substituted furans and thiophens», *Australian Journal of Chemistry* **29**(1), 107–113 (1976). DOI 10.1071/CH9760107

L'abstract dichiara che «gli effetti dei sostituenti in questi eterocicli **assomigliano** a quelli riportati per i benzeni sostituiti». *Assomigliano* non vuol dire *coincidono*: trasferire gli incrementi del benzene è stato provato (§3.10 a) e sui carboni β peggiora. La tabella vera serve ancora.

Perché non è stata scritta a memoria. Sarebbe stato facile: quei numeri sono noti. Ma un valore che nessuno può risalire a una fonte è esattamente ciò che questo documento esiste per impedire, e metterlo in una tabella accanto a novantuno righe trascritte da una pagina stampata renderebbe meno credibili anche quelle. **Il buco dichiarato vale più di un buco tappato male.**

Una riga di validazione con provenienza più debole

Per onestà va detto anche questo: l'acetato di benzile, entrato nell'insieme di validazione perché mostra il punto debole dei due sostituenti in α, porta valori **non verificati contro una fonte primaria** da questo ambiente. Il fenomeno che mostra è confermato in modo indipendente dal quesito d'esame del 23/04/2024 — dato di provenienza vera — ma il **caso peggiore dichiarato (7,5 ppm) poggia su quella riga**, ed è scritto nel banco a chiare lettere.

---

4. Costanti fisiche e dati tabulati

La ricerca delle costanti fisiche procede per livelli di specificità decrescente (corrispondenza esatta → simbolo → parole significative → sottostringa di almeno 4 caratteri) e **raccoglie tutte le candidate**: se ne restano più d'una, la richiesta viene rifiutata con l'elenco delle possibilità anziché restituire arbitrariamente la prima. Un valore fisico sbagliato restituito con sicurezza è peggio di una domanda di chiarimento.

Le molecole d'esame (`MOLECOLE_ESAME`) sono tabulate con valori verificati; il riconoscimento dei gruppi da SMILES usa un campo `consuma`, così che ogni gruppo rimuova la porzione di struttura che ha spiegato e non si producano conteggi sovrapposti.

---

5. Automazione della verifica

Tutti i controlli descritti sono eseguibili e ripetibili. Al momento della stesura la batteria comprende, fra gli altri:

Banco	Oggetto	Controlli
`audit_farmaci`	struttura ⟷ peso, duplicati, formula ricostruita contro ChEMBL, deviazioni registrate con la loro ragione	263 voci
`test_spettri`	riconoscimento gruppi su molecole di riferimento	36
`test_assi` / `test_assi_canvas`	convenzioni degli assi (SVG e canvas)	19
`test_costanti`	ricerca delle costanti fisiche e rifiuto delle ambiguità	45
`audit_dati`	coerenza dei dati tabulati	29
`verifica_guida`	corrispondenza fra ciò che la documentazione promette e ciò che il codice fa	71

L'ultimo banco merita una menzione: verifica che le affermazioni contenute nella documentazione utente corrispondano al comportamento reale del codice. Una promessa non mantenuta dalla documentazione è un difetto al pari di un errore di calcolo, e viene trattata come tale.

---

6. Dichiarazione di trasparenza

Questo documento descrive controlli **effettivamente implementati ed eseguibili**, con i risultati realmente ottenuti e i limiti dei predittori. Alla versione `bsi-v188` gli errori residui sulla banca dati farmaci sono **zero**: le 21 deviazioni che restano sono voci senza struttura, ciascuna con il proprio motivo registrato, non errori taciuti. Le percentuali di copertura e i conteggi riportati sono prodotti dagli strumenti citati e riproducibili eseguendoli.

Dove un dato non è verificabile con gli strumenti a disposizione, lo si dichiara anziché presentarlo come verificato.

Documento aggiornato alla versione `bsi-v200`.

Distinta dei componenti software (SBOM)

BioSpecInfo · documento generato da `tools/genera-sbom.js`.

Versione applicazione	`bsi-v200`
Commit	`3bcf3b9c4601ec1c16de9a50cad666894a86b466`
Generato (UTC)	`2026-10-14T09:17:06.973Z`
Formato macchina	`evidence/sbom.cdx.json` — CycloneDX 1.5

> Le impronte SHA-256 si riferiscono ai file effettivamente distribuiti in questo commit. Un revisore può ricalcolarle con `sha256sum <file>` e confrontarle, verificando di avere sotto mano lo stesso artefatto descritto.

1. Componenti di terze parti distribuiti con l'applicazione

Tutti i componenti sono **inclusi nel repository** e serviti dallo stesso dominio: l'applicazione non carica codice da CDN esterne in fase di esecuzione. È la condizione che le rende utilizzabile offline e che elimina la dipendenza dalla disponibilità di terzi.

Componente	Versione/File	Licenza	Dimensione	Ruolo
RDKit MinimalLib (JS glue) _{RDKit / Greg Landrum}	`RDKit_minimal.js`	BSD-3-Clause	125 kB	Chemioinformatica: interpretazione SMILES, SMARTS, descrittori, disegno 2D.
RDKit MinimalLib (WebAssembly) _{RDKit / Greg Landrum}	`RDKit_minimal.wasm`	BSD-3-Clause	6753 kB	Binario WebAssembly del motore chemioinformatico.
three.js _{mrdoob e collaboratori}	`three.min.js`	MIT	589 kB	Motore di rendering WebGL per le scene 3D.
three.js — UnrealBloomPass _{mrdoob e collaboratori}	`three_bloom.js`	MIT	25 kB	Effetto di post-produzione per le visualizzazioni astronomiche.
three.js — GLTFLoader _{mrdoob e collaboratori}	`gltf_loader.js`	MIT	94 kB	Caricamento dei modelli anatomici e molecolari in formato glTF.

Componente	Versione/File	Licenza	Dimensione	Ruolo
3Dmol.js _{University of Pittsburgh / David Koes}	`3Dmol-min.js`	BSD-3-Clause	502 kB	Visualizzazione molecolare 3D (PDB, SDF, superfici).
SmilesDrawer _{Daniel Probst}	`smiles-drawer.min.js`	MIT	246 kB	Disegno 2D delle strutture a partire da SMILES.
PDF.js (build legacy) _{Mozilla Foundation}	`vendor/pdfjs/pdf.min.mjs`	Apache-2.0	511 kB	Apertura dei documenti PDF nel lettore di spettri: ogni pagina disegnata e il suo strato di testo. E' la build legacy, perche' quella predefinita usa funzioni di linguaggio recentissime e si rompe sui browser che non le hanno ancora.
PDF.js — worker _{Mozilla Foundation}	`vendor/pdfjs/pdf.worker.min.mjs`	Apache-2.0	1290 kB	Il lavoro di decodifica di un PDF, fuori dal filo principale: una pagina complessa non blocca l'interfaccia.
sql.js (JS glue) _{sql.js / SQLite}	`lib/sql-wasm.js`	MIT	45 kB	Database SQLite in-browser per le esercitazioni sui dati.
SQLite (WebAssembly) _{SQLite Consortium}	`lib/sql-wasm.wasm`	Public Domain	644 kB	Motore SQLite compilato in WebAssembly.
Dataset dimuoni (CERN Open Data) _{CERN Open Data Portal}	`lib/dimuon.js`	CC0-1.0	48 kB	Dati sperimentali per l'esercitazione di fisica delle particelle.

Impronte

File	SHA-256
`RDKit_minimal.js`	`58d3c996ade7e0b0137d4f9363ff6c204b545689428fa0898eeaed579ef788d9`
`RDKit_minimal.wasm`	`e0967d44fed59e44a2d07bfc08f3a86821c54a14c95bb7e6e392fe867333b8c8`
`three.min.js`	`9274bbcec8d96168626c732b5d31c775aa8cfb7eaa0599bec0c175908a2c1ce2`
`three_bloom.js`	`42e0ff40b00ea1ba4fe53baeb3726834a3ac3db46e9cfb80856baebf01611090`
`gltf_loader.js`	`5c15967ba830918a9caea6338712c994c354bccd4edc4569bde411c3ec06a3e6`

File	SHA-256
`3Dmol-min.js`	`06b6d2fc7d418e8bef62a32cca44373ff21b2b787bd377613dd4039394a4e9ff`
`smiles-drawer.min.js`	`955285fac52a7017b3c3658e2cb84bef7f1f76732f3f7cd565d1bb0903232dc2`
`vendor/pdfjs/pdf.min.mjs`	`bccc24ea711db8e44503629519904a5292d73b9daaa214bbe7cdcc282b0f4259`
`vendor/pdfjs/pdf.worker.min.mjs`	`145d2dd3ab0c86151011dba95acfa2d5336e2accd59388ea43dbee0efddaaec6`
`lib/sql-wasm.js`	`77d6435bac506af0e3c59636dce9d22b1b14156348bc327f41a1577f3212360f`
`lib/sql-wasm.wasm`	`438c88f666dc054ce4e9395f80fe9db4218b1a3c379960454880f048a7898aed`
`lib/dimuon.js`	`61190d7bbec3ce4ea3add2503b9513ac27b227a3878dd229560a6cd29aabc36d`

2. Codice proprio del progetto

File	Dimensione	Ruolo
`index.html`	4694 kB	Applicazione principale: interfaccia, sezioni didattiche, dati chimici.
`bsi-ai-hub.js`	422 kB	Agente AI «Spectra»: ciclo agentico, strumenti, gestione dei fornitori.
`bsi-spettri.js`	35 kB	Motore di predizione spettrale IR/NMR su grafo molecolare.
`bsi-cheminfo.js`	125 kB	Motore di chemioinformatica: standardizzazione, impronte, raggruppamento, PCA, modelli QSAR con modello nullo.
`bsi-pretsch.js`	37 kB	Le tabelle di stima NMR trascritte intere da Pretsch–Bühlmann–Badertscher, 4ª ed.: benzeni ^{13}C e ^{1}H, etileni, alcani, alchini, alifatici ^{13}C, correzioni steriche, $^{1}J(C,H)$. Solo numeri, nessun codice.
`bsi-nmr.js`	57 kB	Predizione NMR ^{1}H e ^{13}C assegnata per atomo: applica le tabelle di bsi-pretsch.js camminando l'anello e misurando le distanze, con composto di riferimento ciclico ed equivalenza chimica per codice d'intorno.
`bsi-geom3d.js`	46 kB	Coordinate 3D da uno SMILES per geometria delle distanze, con gli stessi indici del predittore NMR: è la condizione del collegamento picco↔atomo↔3D.
`bsi-nmr2d.js`	21 kB	NMR bidimensionale: COSY, HSQC editato e HMBC, costruiti camminando il grafo sulle due predizioni per atomo.
`bsi-moldraw.js`	44 kB	Spettro NMR interattivo: zoom, tabella di assegnazione, collegamento picco↔atomo, cronologia SMILES/InChI, uscite CSV/JCAMP-DX/PNG.
`bsi-documento.js`	23 kB	Apertura di qualunque documento — PDF, immagini, testo, Word, OpenDocument, PowerPoint, Excel — con l'elenco esplicito di cio' che NON si e' potuto leggere.
`bsi-quesito.js`	19 kB	Riconoscimento dei dati spettroscopici nel testo di un quesito, con la provenienza di ogni dato, e svolgimento passo per passo tramite bsi-elucida.js.
`bsi-spettrolettore.js`	101 kB	Lettore di spettri: JCAMP-DX con compressione ASDF, ricerca dei picchi per prominenza, assegnazione delle bande IR, perdite neutre, apertura di documenti.

File	Dimensione	Ruolo
`bsi-mol3d.js`	42 kB	Visore 3D delle molecole: animazione di formazione, selezione degli atomi, misura di angoli, lunghezze e diedri.
`bsi-lingue.js`	162 kB	Motore di internazionalizzazione: quattordici lingue, verso di scrittura compreso.
`bsi-molingue.js`	54 kB	Linguaggi chimici: venticinque uscite da SMILES, con nomenclatura IUPAC su una classe dichiarata.
`bsi-pannelli-lingua.js`	66 kB	Pannelli delle lingue e dei linguaggi chimici.
`sw.js`	6 kB	Service Worker: funzionamento offline e strategia di rete.
`rdkit_lab.html`	260 kB	Laboratorio di chemioinformatica.
`astro.html`	2660 kB	Modulo di astrochimica.
`chimorga.html`	194 kB	Modulo di chimica organica.
`proxy/spectra-proxy.js`	10 kB	Proxy opzionale (Cloudflare Worker) per le chiamate all'AI.

Impronte

File	SHA-256
`index.html`	`4ebccfafaebfc254991ad74c19deb76545369905a07eb9a36f5f9190015f4a45`
`bsi-ai-hub.js`	`985b9fa3f6a3d330cc6f7e31456e16760d0a0d9482f760b04d83765a013f2476`
`bsi-spettri.js`	`be757f15a70f884e3374d401596e2dfd18bcc852cafbb4584920e2b2d24eb3c9`
`bsi-cheminfo.js`	`59a3111b3b9436add5376fde1be6f90ded3dc08c095cebdc04da50c7f8268bbc`
`bsi-pretsch.js`	`27f727240363579fba3dba50ab3b567fbcdb26a8a9a06c0e43968b195c479db3`
`bsi-nmr.js`	`881917dbb141dfcbab9a25b9c518eaa4a8c1c52dc1020cbac90985a1cf859b72`
`bsi-geom3d.js`	`a616b91efa8f790388c99f22cc9f2d48630cb760d4f7fa87f92b99b9db106054`
`bsi-nmr2d.js`	`a0b1db87f27ffa8c124d617e4903f61e35b89c30c35162527ad023325c03fa78`
`bsi-moldraw.js`	`45cacdca20c8091cbfb32e256c179de8c32c71068a5d1008fbef9be005b5df0e`
`bsi-documento.js`	`0e9ad6043a396652549b30879764cc401f8b7bd25689b5bb5f37cf26f7d7d8d9`
`bsi-quesito.js`	`6f686abcfdc3959d8c865a3e6ee7129224de52992ab1ba3da07372a986769cdf`
`bsi-spettrolettore.js`	`4bb07fb9166438212c6519f1bd0fe1e1fc92537ef7ff4cfc16b8bbf9923b7f69`
`bsi-mol3d.js`	`93fea4a6811ba4263de450db4e7ae1801d4fde196cf7683d0100846c51751893`
`bsi-lingue.js`	`2f4c8d97765e1cf8737fb5a101326ffb1b0deb81fccf7a3a526ff4506afbc6c9`
`bsi-molingue.js`	`e45fec1ee20e3e7b9a7bea3d6e20aa4e33e74ec43578bfa7bfb94646b7bc53fb`
`bsi-pannelli-lingua.js`	`4af94d70ef104cf21be18ff7fbc7a2248c56e376e2ae48e8cac83878f4554f93`
`sw.js`	`9c0b61e0a9071bff49016513e6d4cae94fcb60cf34950451ed846e536609e759`
`rdkit_lab.html`	`066a42092b829df2fa306b691b9d162b6d44edfdb5317ce4cc8a276bc34c80ad`
`astro.html`	`3376b2b7f26ed483f2d698c6692b31f21c9f245f87d1fb1884216fb8b1da489d`

File	SHA-256
`chimorga.html`	`047934abc73a79fbaf06ed7036b6e89a913ebdf5354fc52cecdbee7e8f94f51d`
`proxy/spectra-proxy.js`	`808c0a12891ae9c6f45cf0c676b033075089e064df9047be844f03badf940962`

3. Dipendenze di sviluppo

Usate **solo** per l'esecuzione dei banchi di prova. Non vengono distribuite con l'applicazione e non sono presenti in produzione.

Pacchetto	Versione	Licenza
`playwright-core`	1.62.1	Apache-2.0

4. Dipendenze di esecuzione

Nessuna. L'applicazione non ha backend, non installa pacchetti a runtime e non carica script da domini esterni. Le uniche chiamate di rete sono opzionali e dirette a servizi pubblici documentati:

Servizio	Scopo	Se non raggiungibile
PubChem PUG-REST	nome IUPAC, CAS, pittogrammi GHS, conformeri 3D	l'app usa i dati locali e lo dichiara
NASA / ESA	immagini e dati astronomici	il modulo mostra i dati tabulati
Fornitori AI (a scelta dell'utente)	risposte dell'assistente	l'assistente lo segnala e propone alternative

Nessun dato personale viene trasmesso: le chiavi API, se inserite, restano nel `localStorage` del dispositivo, oppure sul proxy dell'utente (vedi `proxy/`).

5. Compatibilità delle licenze

Tutte le licenze dei componenti distribuiti (MIT, BSD-3-Clause, CC0, Public Domain) sono **permissive** e reciprocamente compatibili. Nessun componente è soggetto a licenza copyleft forte (GPL/AGPL): non esistono vincoli di ridistribuzione del codice proprio del progetto.

Gli obblighi residui sono di **attribuzione**: le note di licenza dei progetti a monte sono riportate in `THIRD_PARTY_NOTICES.md`.

> **La licenza dell'applicazione è distinta da quelle dei componenti.** Il codice proprio di BioSpecInfo è **proprietario** — *All rights reserved* (vedi `LICENSE`): visibile e valutabile, ma copia, riuso e uso commerciale richiedono autorizzazione scritta dell'Autore. Le licenze permissive elencate sopra riguardano **soltanto** le librerie di terze parti incluse, e non si estendono all'applicazione.

Generato il 2026-10-14T09:17:06.973Z.

Matrice di tracciabilità — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Autore	Samuele Pio Provenzano
Versione descritta	`bsi-v200`
Scopo	Collegare ogni requisito dichiarato all'implementazione che lo realizza e al banco di prova che lo verifica.

> **Come leggere questa matrice.** Ogni riga è una catena chiusa: un requisito, il punto del codice che lo realizza, e il banco che ne verifica il funzionamento. Una riga senza banco è un requisito **non verificato** e viene dichiarata come tale, non nascosta: l'utilità di una matrice sta esattamente nel rendere visibili le lacune.
>
> Gli esiti dei banchi citati sono riportati per esteso, con l'uscita integrale, in `evidence/RAPPORTO-VERIFICA.md`, generato automaticamente.

1. Requisiti scientifici

ID	Requisito	Implementazione	Banco di verifica
SCI-01	La struttura di un farmaco deve venire da una fonte esterna e coincidere con quella fonte, non solo pesare giusto	`FARM_DATA` in `index.html` — **263 voci**, 36 con identificativo ChEMBL e formula dichiarati	`audit_farmaci` — la formula viene **ricostruita contando gli atomi del grafo** e confrontata con quella di ChEMBL: 36 su 36. Il solo peso non bastava: tre voci preesistenti (lenalidomide, palbociclib, aripiprazolo) erano **isomeri** — stesso peso, molecola diversa — e sono state sostituite. Due cricche: provenienza verificata ≥ 36, strutture leggibili ≥ 240
SCI-02	I gruppi funzionali devono essere riconosciuti sulla struttura, non sul testo dello SMILES	`bsi-spettri.js` §1, pattern SMARTS via RDKit	`test_spettri` — 8 molecole di riferimento, riconoscimento additivo
SCI-03	Una molecola con più gruppi funzionali deve mostrarli tutti	`bandeDaGruppi()`, costruzione additiva	`test_spettri` §1 — acido acetilsalicilico: estere **e** acido

ID	Requisito	Implementazione	Banco di verifica
SCI-04	Gli spettri devono rispettare le convenzioni della tecnica	`svgIR()`, `makeNMRsvg()`, `drawUVSpectrum()`, `drawMSSpectrum()`	`test_assi` (SVG), `test_assi_canvas` (canvas)
SCI-05	Uno spettro calcolato deve essere riproducibile	rimozione di ogni `Math.random()` dalle tracce	`test_assi` §3, `test_spettri` §8 — confronto byte a byte
SCI-06	L'altezza dei picchi ^{1}H deve rappresentare l'integrazione o non essere asserita	`makeNMRsvg()`; `_nmrSvg()` disegna fasce a altezza uniforme	`test_assi` §4
SCI-07	La coniugazione deve agire sul singolo carbonile	SMARTS `acidoConiug`, `estereConiug`, `chetoneConiug`, `aldeideConiug`	`test_spettri` §1 — aspirina 1760/1690; acetofenone 1690
SCI-08	Una costante fisica ambigua non deve essere risolta arbitrariamente	`costante_fisica` in `bsi-ai-hub.js` — raccoglie tutte le candidate	`test_costanti` — 45 controlli, compresi i rifiuti per ambiguità
SCI-09	I limiti dei predittori devono essere dichiarati all'utente	intestazione dei grafici; `docs/06` §3.5	`verifica_guida` — coerenza fra promesse e codice
SCI-10	La simmetria molecolare deve determinare correttamente le regole di selezione	sezione `ssimm` in `index.html`	`test_simmetria` — 26 controlli: modi normali, centro di inversione, esclusione mutua
SCI-12	Un cromatogramma deve discendere dai parametri, non essere disegnato	sezione `scroma`: σ = t_R/√N, R_s da distanza e larghezze	`audit_stabilita` (apertura e disegno), ispezione dei valori contro le definizioni
SCI-13	Una retta di taratura deve dichiarare l'incertezza dei suoi coefficienti e riconoscere i due modi in cui inganna	sezione `staratura`: minimi quadrati ordinari, test F retta⟷parabola, residuo cancellato	Verificato sui tre insiemi di esempio inclusi: lineare (F = 0,3), non lineare (F = 186,6 > 7,71), con punto anomalo (1 rilevato)
SCI-14	Un modello QSAR deve essere valutato su scheletri mai visti, non su analoghi dello stesso insieme	`bsi-cheminfo.js` §9 — `divisionePerScaffold()` accanto a `divisioneCasuale()`	`test_cheminfo` — la divisione per scheletro non lascia nessuno scheletro in comune fra addestramento e prova
SCI-15	Un punteggio di modello deve essere confrontato con il caso, non presentato da solo	`bsi-cheminfo.js` §9 — riaddestramento su etichette rimescolate, otto ripetizioni	`test_cheminfo` — verificato **nei due versi**: su un segnale apprendibile R^2 0,394 batte tutti gli otto sosia (margine 0,287); su etichette casuali R^2 −0,199 **non** li batte

ID	Requisito	Implementazione	Banco di verifica
SCI-16	Due scritture della stessa molecola non devono contarsi due volte né finire da parti opposte della divisione	`bsi-cheminfo.js` §1 — deduplicazione sullo SMILES canonico dopo il frammento maggiore	`test_cheminfo` — `OC(=O)C` e `CC(=O)O` collassano in una voce; i sali perdono il controione
SCI-17	La similarità fra molecole deve essere calcolata, non stimata	`bsi-cheminfo.js` §3 — Tanimoto e Dice su impronte Morgan, RDKit, pattern, MACCS	`test_cheminfo` — confronto con valori calcolabili a mano, compresa la convenzione 0/0 = 1
SCI-18	Una validazione su una divisione sola, su un insieme piccolo, è un numero rumoroso: va accompagnata dalla sua dispersione	`bsi-cheminfo.js` §9-bis — `validazioneIncrociata()` su pieghe **raggruppate per scheletro**	`test_cheminfo` — le pieghe coprono ogni molecola una volta sola, **nessuno scheletro sta in due pieghe**, e con lo stesso seme le pieghe sono le stesse
SCI-19	Un risultato che cambia a ogni esecuzione non è verificabile da nessuno	generatore mulberry32 seminato per divisioni, rimescolamenti e pieghe; il seme è scritto nel rapporto	`test_cheminfo` — due chiamate con lo stesso seme danno pieghe identiche; il rapporto di metodo dichiara il seme
SCI-20	I risultati devono poter uscire dallo strumento, e il metodo deve uscire con loro	`bsi-cheminfo.js` §9-ter — `esportaCsv()` e `rapportoMetodo()`, entrambi formati in memoria nel browser	`test_cheminfo` — colonne coerenti con l'intestazione, nome con virgola citato, residuo calcolato, cella vuota per l'attività mancante; il rapporto contiene seme, fingerprint, verdetto, limiti
SCI-21	Un numero presentato come similarità strutturale deve venire da un fingerprint, e una sola implementazione deve calcolarlo	`rdkit_lab.html` carica `bsi-cheminfo.js` e ne usa `tanimoto()` e `fingerprint()`; la similarità a otto bit è stata rimossa	`test_cheminfo` — la pagina e il motore danno lo stesso Tanimoto su tutte e **28 le coppie**, scarto **esattamente 0**; le funzioni della similarità finta non esistono più; ogni struttura di riferimento è valida e ha la **massa monoisotopica** del farmaco che dichiara
SCI-22	Le conversioni fra linguaggi molecolari devono essere verificate contro una fonte esterna o contro il calcolo a mano, e le coordinate scritte in un file devono essere vere e dichiarate	`bsi-molingue.js` — **venticinque uscite**, da SMILES a CML e alle sei impronte	`test_lingue` — 7 chiavi InChI **di letteratura**, giro completo SMILES → molfile → SMILES, 26 nomi a mano, **7 rifiuti pretesi**; composizione in massa contro il calcolo a mano (alanina C 40,44%, aspirina C 60,00%); XYZ e PDB con coordinate non nulle **e** dichiarate 2D

ID	Requisito	Implementazione	Banco di verifica
SCI-23	Angoli, lunghezze e diedri si misurano sulle coordinate, e su coordinate piatte non si mostrano	prodotto scalare per gli angoli, prodotto vettoriale per i diedri	`test_mol3d` — tetraedro **109,4712°**, acqua 104,47°, CO_2 180°, BF_3 120°, ammoniaca 106,13°; diedri 0°, 60°, 90°, 180°; C–C dell'etanolo 1,509 Å; e il rifiuto provato nei due versi
SCI-24	Uno spettro letto da un file deve essere decodificato secondo il formato, e i picchi trovati devono essere quelli che ci sono — né più né meno	`bsi-spettrolettore.js` — JCAMP-DX con ASDF completo (PMAI, DIF, DUP) e il controllo di integrità che il formato prevede; picchi per **prominenza**, rumore da MAD × 1,4826	`test_spettrolettore` — **61 controlli**. Su tre gaussiane costruite a 1715, 2950, 3400 cm^{-1} sotto rumore ne trova tre: 1716, 2952, 3402. **Nei due versi**: su rumore puro ne trova **zero**. ASDF verificato simbolo per simbolo (`abcdefghi` = −1…−9; `n` = −5, non −4)
SCI-25	Una banda deve essere dichiarata *compatibile con*, mai assegnata a una sola possibilità	`BANDE_IR` — 24 intervalli, `assegnaIR()` restituisce **tutte** le assegnazioni compatibili	`test_spettrolettore` — a 1715 cm^{-1} compaiono sia C=O chetonico sia C=O di acido carbossilico; la sezione dichiara di non dedurre la struttura
SCI-26	Un modello pubblicato va verificato contro il **suo** errore dichiarato, e un modello appreso contro il caso	`bsi-cheminfo.js` §10 — ESOL (Delaney 2004) con incertezza, quattro filtri di drug-likeness separati, foresta casuale con errore fuori sacco	`test_spettrolettore` §4-5 — ESOL contro quattro valori sperimentali, scarto medio **0,68** entro l'errore dichiarato di ~1 unità logaritmica; foresta R^2 fuori sacco **0,90** su una relazione non lineare e **−0,16** su puro rumore; deterministica a parità di seme e diversa con seme diverso
SCI-27	Una struttura senza fonte non deve entrare, nemmeno quando la si conosce	raccolta da ChEMBL, `tools/dati/farmaci_v187.json` con l'identificativo di ogni voce	**Due rifiuti**: ivermectina (`structure_type NONE`, miscela di omologhi) e semaglutide (proteina, `SEQ`). E due nomi tenuti con il nome del **record** e non della domanda: «morphine» → apomorfina, «levothyroxine» → liotironina

ID	Requisito	Implementazione	Banco di verifica
SCI-28	Uno spettro previsto deve dire QUALI atomi producono ogni segnale, e lo scarto va misurato su molecole che non hanno scelto i parametri	`bsi-pretsch.js` — le tabelle di stima trascritte intere dalla fonte (Pretsch 4ª ed.: 91 righe ^{13}C e 66 ^{1}H per i benzeni, 42 per gli etileni, 31 per gli alcani, 24 per gli alifatici ^{13}C, correzioni steriche 4×4); `bsi-nmr.js` — il ragionamento che le applica: posizione nell'anello ricavata camminando il ciclo, schema additivo con composto di riferimento ciclico, equivalenza chimica per codice d'intorno a gusci	`test_nmr` — **56 controlli**. Due insiemi separati: taratura **0,71 ppm** su 9 molecole, **validazione 0,92 ppm** su 22 mai usate per tarare, caso peggiore **4,9 ppm**, ^{1}H **0,06 ppm** (0,03 sui soli aromatici). Il banco pretende che la validazione resti **peggiore** della taratura: se diventassero uguali, qualcuno avrebbe spostato una molecola fra i due insiemi
SCI-29	Atomi chimicamente equivalenti devono dare UN segnale, non uno per atomo	codice d'intorno a gusci concentrici per distanza (principio dei codici HOSE), con i legami aromatici scritti come tali e non in forma di Kekulé	`test_nmr` — otto molecole di cui si sa quanti segnali danno: benzene **1** (non sei), toluene 5, p-xilene 3, naftalene 3, difenile 4, aspirina 9. Tutte giuste
SCI-30	Da un'immagine si può recuperare la FORMA di uno spettro, non la sua taratura	`bsi-digitalizza.js` — `bsi-spettrolettore.js` gli delega `daImmagine()`, una sola implementazione. Cornice degli assi riconosciuta prima di tutto, traccia a **subpixel** (baricentro pesato sulla scurezza), scelta del tratto per continuità **e** per scurezza, taratura dei due assi lineare o logaritmica, %T convertita in assorbanza con $A = -\log_{10} T$	`test_digitalizza` — **45 controlli** su sette figure costruite: scarto medio **2,2 cm⁻¹**, peggiore **7,3** (una colonna vale ~4 cm⁻¹). **Nei due versi**: da un foglio bianco e dall'icona dell'applicazione non esce nessuna curva, e il ripiego grezzo sulla stessa figura con cornice ed etichette trova **un** picco invece di tre. `test_spettrolettore` per l'instradamento
SCI-31	Una mappa NMR bidimensionale deve dire QUALI atomi producono ogni correlazione, e non deve collegare ciò che la topologia non collega	`bsi-nmr2d.js` — HSQC a un legame con segno invertito sui CH_2, COSY a tre legami fra protoni non equivalenti, HMBC a due e tre legami; gli scambiabili esclusi dalle macchie fuori diagonale	`test_nmr2d` — **40 controlli**, nei due versi: l'esafluorobenzene non dà macchie HSQC, il benzene **nessuna** macchia COSY fuori diagonale (i sei protoni sono equivalenti), il metano **nessuna** macchia HMBC pur avendo quattro protoni
SCI-32	Una geometria 3D costruita dal programma deve avere GLI STESSI indici del predittore, altrimenti il collegamento picco↔atomo illumina l'atomo sbagliato	`bsi-geom3d.js` — geometria delle distanze sul grafo di `bsi-nmr.js`, idrogeni aggiunti in coda, generatore seminato dallo SMILES	`test_geom3d` — **29 controlli**: ogni atomo pesante ha lo stesso indice e lo stesso elemento su cinque molecole; legami e angoli entro tolleranza dai valori di letteratura; **nei due versi** il benzene esce piano e il cicloesano no

ID	Requisito	Implementazione	Banco di verifica
SCI-33	Un documento va aperto per intero, e ciò che NON si è potuto leggere va dichiarato — un lettore che restituisce poco senza dirlo fa credere che il documento fosse quasi vuoto	`bsi-documento.js` — PDF pagina per pagina con lo strato di testo (PDF.js), immagini, testo, archivi Office aperti con `DecompressionStream` senza librerie	`test_documento` — **47 controlli**: PDF, Word e testo aperti davvero e il loro contenuto verificato; **nei due versi** un'immagine dichiara che non c'è riconoscimento ottico dei caratteri e non produce svolgimento, un formato sconosciuto viene rifiutato invece che indovinato, un file binario chiamato `.txt` viene riconosciuto come binario
SCI-34	I dati spettroscopici riconosciuti in un testo devono portare la PROVENIENZA, e lo svolgimento deve dichiarare che cosa gli manca	`bsi-quesito.js` — riconoscimento per sezione (IR, MS, ^{1}H, ^{13}C) con i confini fra sezioni, poi `bsi-elucida.js` per lo svolgimento passo per passo	`test_documento` — formula, bande, masse con intensità, segnali ^{1}H con integrazione e molteplicità, segnali ^{13}C; **nei due versi**: $CDCl_3$ non diventa il composto, una banda fuori da 400-4000 non è una banda, senza etichetta MS non ci sono masse, un capoverso nuovo chiude la sezione, un testo senza spettri non produce dati
SCI-35	Una figura con la cornice, le etichette e la griglia non deve essere digitalizzata come se fosse solo la curva: gli assi e i numeri sono pixel scuri come la traccia	`bsi-digitalizza.js` — la cornice si cerca fra le linee CONTINUE nei margini, e un bordo si distingue da una riga di griglia per la **spaziatura irregolare** rispetto alla famiglia **oppure** per il tratto più spesso e più nero; il ritaglio si sposta oltre lo spessore della linea; le colonne di griglia verticale si segnano come non misurate e si ricostruiscono dalle vicine	`test_digitalizza` §1-2 — su una figura con cornice **e** griglia (11 famiglie verticali, 7 orizzontali) trova tutti e quattro i lati e i picchi restano entro 2,2 cm^{-1}; **nei due versi**, su una figura con la sola griglia **non** inventa una cornice e lo dichiara, e una **linea di base piatta** non viene presa per un asse
SCI-36	Una curva digitalizzata deve poter uscire dall'applicazione, e il file esportato deve essere valido	`bsi-digitalizza.js` — uscita CSV con le unità in testa e JCAMP-DX 4.24 in forma (XY..XY) con FIRSTX/LASTX/NPOINTS e la provenienza scritta dentro il file	`test_digitalizza` §4 — il **giro completo**: il JCAMP esportato viene riletto da `leggi()` di `bsi-spettrolettore.js` e torna con lo stesso numero di punti, gli stessi estremi e le stesse unità

ID	Requisito	Implementazione	Banco di verifica
UI-14	Una stessa funzione non deve avere due porte d'ingresso. Gli spettri stavano in **nove sezioni** su due menù diversi; Spectra in tre posti; e nel pannello ✨ comparivano **due «Data Science», due «File Manager», due «Articoli», due «Aggiornamenti»**, perché il pannello ha un elenco cablato che apriva le stesse sezioni inserite dal modulo del menù — e nessuno confrontava le due liste fra loro	`bsi-centrospettri.js` — i pannelli vengono **spostati** (`appendChild` muove, non copia) in una sola sezione a schede; i vecchi pulsanti restano nascosti perché sono il bersaglio di `goSection()`, e la scheda **clicca il pulsante** invece di mostrare il pannello, così l'inizializzazione pigra passa per l'unica strada che già c'era. `bsi-menu.js` — la categoria «Utility» è nel **menù principale** con tutte le sue voci, e la regola è: se una cosa ha un pulsante **raggiungibile** nella barra non sta nel menù ✨, se non ce l'ha sta solo nel menù. «Raggiungibile» non si misura con `offsetParent` — nella barra un gruppo per volta è mostrato, e un pulsante perfettamente raggiungibile ha `offsetParent` nullo	`test_articoli` §5-6 — le **dieci** schede sono dentro il centro, e ogni identificatore esiste nel documento **una volta sola** (la prova che si è spostato e non copiato); `goSection()` su ognuna apre il centro sulla scheda giusta e ne mostra **solo** una. `test_menu` — Utility visibile nella barra con tutte e **quindici** le voci più le due delle lingue, **nessuna** voce del menù che ripeta una sezione della barra, **nessuna** etichetta doppia nel pannello, e su **due viewport**: schermo grande e telefono. **Nei due versi**: nessun id ripetuto, nessun pulsante doppio, Spectra e il lettore fuori dalla barra ma con il pulsante ancora nel documento perché `goSection()` lo usa
UI-15	Una tabella di dati non deve esistere due volte: due copie divergono, e chi legge ottiene una risposta diversa a seconda di dove entra	Le costanti di accoppiamento J esistevano in due tabelle indipendenti con numeri **diversi** (^{3}J orto aromatico 6–9 Hz contro 7–10 Hz). Resta quella dentro il Centro spettri, che è nella sezione con la sua lista di riferimenti; `_apriCostantiJ()` è l'unica strada e porta lì	`test_articoli` §6 — chiedere le costanti J apre il centro sulla scheda delle tabelle di riferimento, sulla sotto-scheda giusta, e la tabella ha le sue righe
ING-02	Un elenco di articoli scritto a mano è vecchio il giorno dopo, e un anno di premio scritto a memoria è un errore che non si vede	`bsi-articoli.js` legge **in diretta** da Crossref e dalla Fondazione Nobel; `bsi-articoli-nobel.js` contiene **solo spiegazioni**, abbinate all'argomento della motivazione ufficiale e non all'anno, così nessuna attribuzione viene dall'applicazione. Gli indirizzi non sono scritti: sono **funzioni pure** dell'identificatore (DOI, anno)	`test_articoli` §1-4 — i link provati come funzioni (stabili a parità di dato, diversi al variare); le spiegazioni abbinate a nove motivazioni di prova e **nessuna** inventata quando l'argomento non si riconosce; e la prova che conta: si **inietta** una risposta con un nome inesistente e si pretende che sullo schermo compaia **quel** nome. **Tre guasti distinti provati**: richiesta fallita, risposta vuota, risposta di forma non riconosciuta — e devono essere detti **diversi**

ID	Requisito	Implementazione	Banco di verifica
ING-01	Una stessa funzione non deve avere due implementazioni: lo stack React/FastAPI deve eseguire gli STESSI file dell'applicazione	`stack/api/worker/motore.mjs` — un processo Node carica i moduli dal repository con RDKit WebAssembly; nessun motore in Python	`test_stack` — **15 controlli**: confronta le impronte SHA-256 dichiarate da `GET /salute` con quelle dei file su disco, verifica che i numeri coincidano con quelli del browser, e che nessun file `.ts`/`.tsx` contenga una tabella di spostamenti

2. Requisiti dell'agente AI

ID	Requisito	Implementazione	Banco di verifica
AI-01	Un modello ritirato dal fornitore non deve bloccare l'assistente	`modelloSuggeritoDaErrore()`, lista nera `bsi_modelli_ko`	`test_404` — 36 controlli
AI-02	Un fornitore irraggiungibile dal browser deve essere ricordato e retrocesso	`bsi_prov_ko`, TTL 24 h, `providerUtilizzabili()`	`test_ko` (35), `browser_ko` (37)
AI-03	Un sovraccarico temporaneo deve produrre un nuovo tentativo, non un errore	`erroreTemporaneo()`, `TENTATIVI_TEMPORANEO`	`test_503` — 40 controlli
AI-04	Un'attesa imposta superiore a 10 s deve far cambiare fornitore se ve n'è uno pronto	`ATTESA_TROPPO_LUNGA_MS`	`test_attesalunga` — 14 controlli
AI-05	Il tetto di token del fornitore deve essere appreso dall'errore, non codificato	`tettoDaErrore()`, `MARGINE_TETTO`	`test_tetto` — 32 controlli
AI-06	Le firme di ragionamento opache devono essere restituite al fornitore	cattura e replay in `appendAgentTurn()`	`test_firma` — 31 controlli
AI-07	Due invii sovrapposti non devono corrompere la cronologia	`_turnoInCorso` in `send()`	`test_doppioinvio` — 7 controlli
AI-08	L'utente deve poter misurare quali fornitori raggiunge davvero	`provaTuttiIFornitori()`	`browser_prova` — 29 controlli
AI-09	I fornitori a pagamento non devono essere proposti come gratuiti	separazione per `PROVIDERS[id].free`	`browser_prova` §D2
AI-10	Deve esistere una via documentata per eliminare CORS e limiti	`proxy/spectra-proxy.js` + guida in-app	`test_guidaproxy` (12), `browser_proxy` (15), `browser_proxyui` (17)

3. Requisiti di stabilità

ID	Requisito	Implementazione	Banco di verifica
STA-01	Una sessione lunga non deve accumulare nodi DOM né timer	gestione del ciclo di vita delle sezioni	`audit_stabilita` §1 — 95 sezioni × 5 giri
STA-02	L'esaurimento di `localStorage` non deve rendere inutilizzabile l'app	scritture protette; avviso persistente nel File Manager	`audit_quota` (10), `test_filemanager` (15)
STA-03	Nessuna promessa rifiutata deve restare non gestita	`.catch()` sistematici	`audit_promesse` — 22 controlli, rete sana e rete morta
STA-04	Dati salvati corrotti non devono impedire l'avvio	`loadJSON()` con ripiego	`audit_stabilita` §4
STA-05	Una rete lenta non deve essere trattata come una rete assente	`ATTESA_RETE_MS = 3500` in `sw.js`	`test_sw` §3 e §3-bis — misurato 3 507 ms
STA-06	Un aggiornamento non deve distruggere il lavoro in corso	messaggio `BSI_SW_UPDATED`, decisione lato pagina	`test_sw` §4
STA-07	Sfogliare molecole non deve esaurire i contesti WebGL	riuso del visore 3D nei 5 punti di creazione	`test_visore3d` — da 7 costruzioni a 1
STA-08	L'archivio IndexedDB non disponibile deve essere dichiarato, non fatale	`dbStore()` su promessa; avviso persistente	`test_filemanager` §2

4. Requisiti di interfaccia e accessibilità

ID	Requisito	Implementazione	Banco di verifica
UI-01	Ogni pagina deve aprirsi senza errori JavaScript	—	`audit_stabilita` §5 — 14 pagine
UI-02	I grafici devono essere nitidi su schermi ad alta densità	`bsiNitido()` sui contesti canvas	`audit_grafici` — 40 canvas

ID	Requisito	Implementazione	Banco di verifica
UI-03	L'app deve funzionare a 390 px di larghezza, e il contenuto che non ci sta deve restare **raggiungibile**	griglie `auto-fit`, nessuna colonna fissa, e le tabelle larghe dentro un contenitore con `overflow-x:auto`	`audit_mobile` — **due misure distinte** su tutte le 95 sezioni a 390 px: lo `scrollWidth` del documento (la barra orizzontale che l'utente vede) **e** gli elementi più larghi della loro sezione **senza un antenato scorrevole**, cioè il contenuto ritagliato e irraggiungibile — una colonna scritta che non si può leggere e non dà nessun segno di esistere. **Nei due versi**: una tabella larga dentro un contenitore che scorre **non** viene segnalata, perché si legge scorrendola. `audit_stabilita` per gli errori in viewport telefono
UI-04	L'utente deve poter sapere quale versione sta usando e forzare l'aggiornamento	voce «Aggiornamenti» nel pannello ✨	`test_aggiorna` — 9 controlli
UI-05	La cancellazione dei dati deve essere selettiva e reversibile nelle scelte	`bsiCancellaDati()` per gruppi	`browser_reset` — 24 controlli
UI-08	Anche le superfici che compaiono solo dopo un'azione devono rispettare il contrasto WCAG AA	i sei pannelli della sezione Chemioinformatica	`test_cheminfo` — l'analisi viene eseguita, i pannelli aperti a turno, **460 elementi di testo misurati, 0 difetti**, con le formule WCAG riscritte dentro il banco
UI-09	Aprire una sezione non deve bloccare la pagina	primo `render()` di 3Dmol fuori dal click; le 296 figure di sintesi **e le 263 carte dei farmaci** disegnate a fette da 8 ms	`test_fluidita` — **95 sezioni attraversate**, cambio mediano 16 ms, peggiore 87 ms, **0 sezioni oltre 100 ms** (erano 3 con i farmaci nuovi, a 159 ms); e le prove contrarie: la tela WebGL compare comunque, le 296 figure esistono tutte poco dopo, la stampa non esce muta. Sul visore l'affermazione è **strutturale e non temporale**: subito dopo il ritorno del gestore del click il contesto WebGL non deve esistere, poco dopo deve — perché una soglia in millisecondi misurava SwiftShader e non il codice (14…192 ms a codice identico)

ID	Requisito	Implementazione	Banco di verifica
UI-10	L'interfaccia deve poter cambiare lingua fra molte, dichiarando per ognuna quanto è tradotta, portando con sé il verso di scrittura	`bsi-lingue.js` con **quattordici lingue**, selezionatore con ricerca, `dir="rtl"` per l'arabo	`test_lingue` — **170 elementi di scheletro su 170 in ognuna delle tredici lingue**, copertura calcolata applicando il dizionario e contando; verso di scrittura provato nei due versi; categorie della barra e segnaposto della ricerca compresi
UI-11	La molecola deve mostrare come sta insieme, e l'angolo lo deve scegliere chi guarda	`bsi-mol3d.js`: formazione dalla polvere con arrivo scaglionato, e selezione degli atomi con un clic	`test_mol3d` — i **pixel** della tela a 130 ms e a 2,3 s, l'ordine d'arrivo (scheletro prima degli idrogeni) misurato sull'avanzamento di ogni atomo, e la selezione di 1, 2, 3 e 4 atomi
UI-12	Chi ha un file deve poterlo aprire: un selettore che elenca le estensioni non restringe la scelta su un telefono, la **nasconde**	`bsi-spettrolettore.js` — un solo `<input type="file">` **senza** `accept`, un solo bottone, e `apriQualunque()` che instrada per tipo, per estensione dichiarata dal formato e **per contenuto**, annunciando la decisione in `#bsiSP-deciso`	`test_spettrolettore` §7 — un solo bottone visibile, `accept` **vuoto** (un `accept` non vuoto qui *è* il guasto), e l'instradamento verificato su PDF, `.docx`, `.txt`, `.jdx`, due colonne `.csv` e un'immagine; **nei due versi** un binario travestito da `.txt` non viene riversato come testo ma dichiarato binario
UI-13	La versione mostrata in intestazione deve essere quella che sta girando	`index.html` — il distintivo si riempie da `window.BSI_APP_VERSION`, non è più una stringa scritta a mano	`test_spettrolettore` §8 — il testo del distintivo è confrontato con `BSI_APP_VERSION` misurata nella pagina in esecuzione. Serviva: per circa ottanta versioni l'intestazione ha scritto `v13l` quando l'applicazione aveva già superato la versione 190, e nessun controllo la guardava

5. Requisiti di sicurezza e riservatezza

ID	Requisito	Implementazione	Banco di verifica
SEC-01	Nessuna chiave API deve essere presente nel repository	chiavi solo in `localStorage` o nei segreti del Worker	`tools/verifica-sicurezza.js` — 368 file tracciati, 8 forme di credenziale
SEC-02	Nessun dato personale deve lasciare il dispositivo senza azione esplicita	architettura local-first, telemetria disattivata	`audit_rete` — **verifica diretta**: un valore spia seminato in 71 depositi dei dati utente, l'applicazione usata per 95 sezioni su 6 pagine, e URL, intestazioni e corpo di ogni richiesta ispezionati. Più `verifica-sicurezza` sui due meccanismi di uscita
SEC-03	Le password non devono comparire in chiaro nel sorgente	SHA-256 in `file_manager.html`	`tools/verifica-sicurezza.js`

ID	Requisito	Implementazione	Banco di verifica
SEC-07	Nessuna credenziale deve essere MAI entrata nel repository, nemmeno in un commit poi corretto	nessuna chiave è mai stata committata; le chiavi stanno in `localStorage` o nei segreti del Worker	`audit_storia` — **1 551 versioni distinte di file di testo su 390 commit**, 9 schemi provati nei due versi; fallisce su un clone superficiale, perché misurerebbe meno superficie
SEC-04	Nessun marcatore di conflitto deve raggiungere la pubblicazione	—	`verifica_guida` §12 — 58 file di testo

> **SEC-03 — limite dichiarato.** GitHub Pages serve file statici: qualunque controllo di accesso lato pagina è aggirabile da chi legge il sorgente. La protezione del File Manager è un deterrente, **non** un controllo di sicurezza, e la documentazione lo afferma. Inoltre la password in chiaro è rimasta nella cronologia git fino alla sua rimozione, e togliere un segreto dai file non lo toglie dalla storia: `git log -p` lo restituisce a chiunque. L'unico rimedio effettivo era cambiarla, ed **è stato fatto** alla versione `bsi-v188`. Quella vecchia resta nella cronologia e non apre più niente.

---

6. Requisiti non ancora verificati automaticamente

Dichiarati per completezza. Sono coperti da ispezione documentale o da prova manuale, e la loro automazione è in programma.

ID	Requisito	Copertura attuale
UI-06	Conformità WCAG 2.1 AA completa	la parte meccanica è automatizzata su **13 pagine e 95 sezioni** (`tools/verifica-accessibilita.js`, **38 407** elementi di testo: erano 19 751 prima che il testo su gradiente entrasse nella misura, e 37 409 prima della sezione «Lettore spettri»). **Dei 1 069 difetti di contrasto emersi non ne resta nessuno: 0 misurati** sulle stesse 95 sezioni, e il valore è registrato come riferimento che il banco difende. I campi senza etichetta sono **zero**: vedi il riquadro in `docs/09` §4. Restano fuori il testo dentro gli SVG e quello su una vera immagine di sfondo, contati a ogni esecuzione (D-09)
PERF-01	Tempo di primo disegno su dispositivo di fascia bassa	prova manuale cross-device (`docs/02` §4)
SCI-11	Strutture di 2 voci che non sono molecole singole (erano 6)	**non rappresentabili**: Ivermectina è una miscela di omologhi, Coartem un'associazione di due principi attivi. Le altre quattro sono state chiuse riprendendo la struttura da ChEMBL, vedi `docs/06` §2.4
PERF-02	Copertura di codice dei banchi di prova	**misurata parzialmente**: 49,89 % di istruzioni sul percorso più ampio (`audit_copertura`). Resta non misurata la copertura dell'intera batteria e quella di rami; vedi §8

ID	Requisito	Copertura attuale
UI-07	Comportamento su Firefox e WebKit	**non verificato** — la batteria gira solo su Chromium; vedi §8

7. Copertura complessiva

Categoria	Requisiti	Verificati da banco	Copertura
Scientifici (SCI-01…36)	35	35	100 %
Agente AI (AI-01…10)	10	10	100 %
Stabilità (STA-01…08)	8	8	100 %
Interfaccia (UI-01…15)	13	13	100 %
Sicurezza (SEC-01…07)	5	5	100 %
Architettura (ING-01…02)	2	2	100 %
Totale automatizzato	**73**	**73**	**100 %**
Non automatizzati (§6)	5	0	0 %
Totale dichiarato	**78**	**73**	**94 %**

La copertura è calcolata sui requisiti **dichiarati in questo documento** e non va confusa con una copertura di codice: misura quanti requisiti hanno un banco che li verifica, non quante righe vengono eseguite durante i test.

> **Perché due totali.** Riportare solo il 100 % dei requisiti automatizzati sarebbe vero e fuorviante insieme: è il 100 % di ciò che si è scelto di automatizzare. Il numero che conta per chi valuta è il secondo — **54 requisiti verificati su 59 dichiarati** — e i cinque che mancano sono elencati per nome in §6, non riassunti in una percentuale.
>
> Questa tabella è controllata da `tools/verifica-documenti.js`, che conta gli identificativi realmente presenti nel documento e fallisce se i totali qui scritti non corrispondono. È un controllo nato da un errore reale: i totali dicevano 36 su 9 categorie scientifiche mentre gli identificativi definiti erano 10, e `SCI-10` compariva due volte — una come requisito verificato e una come requisito non verificato.

8. Lacune di copertura dichiarate

Non sono difetti: sono cose che **non sono state misurate**, e che quindi non possono essere affermate.

Lacuna	Che cosa comporta	Perché è così
Copertura di codice parziale	Misurata: **49,89 %** di istruzioni sul percorso più ampio che un banco compie. Non è la copertura dell'intera batteria, ed è di istruzioni, non di rami	Il profilatore di Chromium la raccoglie nel motore, senza build e senza riscrivere il sorgente: l'ostacolo era dello strumento (c8, istanbul), non del problema. Valore registrato, `audit_copertura` fallisce se scende
Solo Chromium	Il comportamento su Firefox e WebKit è verificato a mano, non da banco	La batteria usa `playwright-core`, che scarica un motore solo. Nell'ambiente di verifica la CDN degli altri motori risponde **403** alla politica di rete: non sono installabili lì
Nessuna regressione visiva	Un cambiamento grafico involontario non verrebbe intercettato	Gli spettri sono deterministici e confrontabili byte a byte (SCI-05): il confronto esiste sulle tracce, non sull'intera pagina
2 voci senza struttura	Due voci su 178 non hanno una struttura da mostrare: Ivermectina è una miscela di omologhi, Coartem un'associazione di due principi attivi. Erano sei	Vedi `06-Scientific-Accuracy-Data-Provenance.md` §2.4
Prestazioni su dispositivi lenti	Il tempo di primo disegno non è misurato su hardware di fascia bassa	Richiede dispositivi fisici; la prova è manuale

---

Documento aggiornato alla versione `bsi-v200`.

Dichiarazione di conformità di rilascio — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Versione	`bsi-v200`
Autore e responsabile del rilascio	Samuele Pio Provenzano
Repository	`github.com/samupropio1-ship-it/BioSpecInfo-v11`
Distribuzione	GitHub Pages — `samupropio1-ship-it.github.io/BioSpecInfo-v11/`
Natura del software	Progressive Web App client-side, senza backend

1. Oggetto della dichiarazione

Il sottoscritto dichiara che la versione `bsi-v200` di BioSpecInfo è stata sottoposta alla procedura di verifica descritta in `02-Verification-Validation-Report.md` e che gli esiti sono quelli riportati, senza selezione, in `evidence/RAPPORTO-VERIFICA.md` — documento generato automaticamente da `tools/genera-evidenza.js`, che trascrive l'uscita dei banchi senza intervento manuale.

Questa dichiarazione **non** attesta l'assenza di difetti. Attesta che:

1. esiste una procedura di verifica automatizzata e riproducibile da terzi;
2. l'esito riportato è quello realmente prodotto, compresi i fallimenti;
3. i limiti noti sono dichiarati in modo esplicito e non presentati come funzionalità.

2. Destinazione d'uso e limitazioni

2.1 Destinazione dichiarata

BioSpecInfo è uno strumento **didattico e di supporto allo studio** per chimica, biochimica, farmacologia e astrochimica. È progettato per studenti universitari e per la preparazione di esami.

2.2 Ciò per cui il software NON è destinato

Queste esclusioni sono parte integrante della dichiarazione.

Ambito	Esclusione
Uso clinico	Non è un dispositivo medico ai sensi del Regolamento (UE) 2017/745. Non supporta diagnosi, prescrizione, dosaggio né decisioni terapeutiche. I dati farmacologici hanno finalità esclusivamente didattica e l'applicazione lo dichiara in apertura della sezione.
Uso analitico certificato	Gli spettri sono **predizioni** da tabelle di gruppi funzionali, non misure né simulazioni quantomeccaniche. Non sostituiscono un database sperimentale (SDBS, NIST) né una determinazione strumentale, e non sono utilizzabili per identificazione formale di sostanze.
Ambiente regolamentato GxP	Non è validato per uso in ambiente GLP/GMP. L'allineamento a GAMP 5 descritto in `docs/03` è metodologico, non una qualifica formale.
Controllo di accesso	Il File Manager protegge i contenuti con un deterrente lato client. Su un sito statico non esiste controllo di accesso effettivo: chi legge il sorgente lo aggira. Non va usato per materiale riservato.

3. Sintesi degli esiti di verifica

Gli esiti integrali, con l'uscita testuale di ogni banco, l'ambiente di esecuzione, il commit e le impronte SHA-256, sono in `evidence/RAPPORTO-VERIFICA.md`.

Famiglia	Cosa dimostra
Dati scientifici	I dati chimici sono verificati contro una fonte indipendente
Agente AI	L'assistente resta utilizzabile quando il fornitore esterno si guasta
Stabilità	L'app regge sessioni lunghe, memoria esaurita, rete degradata
Interfaccia	I pannelli rispondono come documentato
Coerenza documentazione/codice	Ciò che la documentazione promette esiste nel codice

4. Difformità note alla data del rilascio

Elencate per esteso. Nessuna è stata rimossa dalla verifica per farla passare.

#	Difformità	Impatto	Trattamento
D-01	**2 voci** (erano 6) che non sono molecole singole: Ivermectina, miscela di omologhi B1a/B1b, e Coartem, associazione di due principi attivi	Per queste due non è disponibile la rappresentazione 2D/3D né la predizione spettrale	Voci lasciate **senza SMILES**: i dati farmacologici restano, la struttura non è mostrata. Per Digossina, Vincristina e Tacrolimus (topico e sistemico) la struttura è stata ripresa da **ChEMBL** e supera il confronto col peso molecolare dichiarato: le quattro voci sono uscite dal registro. Le due che restano non hanno una struttura da mostrare, non una struttura mancante.
D-02	19 voci prive di SMILES per natura (anticorpi monoclonali, peptidi)	Nessuno: per queste molecole la notazione SMILES non è la rappresentazione appropriata	Non è una difformità sostanziale; elencata per completezza
D-03	~~SEC-02 verificato solo per interposta proprietà~~ — **sanata**: vedi S-10	—	`audit_rete` osserva il traffico durante un uso reale, con un valore spia seminato nei dati dell'utente
D-04	~~Password del File Manager nella cronologia git~~ — **sanata**: vedi S-12	—	La password è stata **cambiata** alla versione `bsi-v188`. Quella vecchia resta leggibile nella cronologia e non apre più niente
D-06	Copertura di codice **misurata ma parziale**: 49,89 % di istruzioni, non di rami	È la copertura del percorso più ampio che un banco compie (95 sezioni più le altre pagine), non della batteria intera; e un `if` entrato da un solo lato conta come coperto	`audit_copertura` la misura con il profilatore di Chromium, senza build e senza riscrivere il sorgente. Il valore è **registrato**: se scende, la batteria fallisce

#	Difformità	Impatto	Trattamento
D-07	Verifica su Chromium soltanto	Firefox e WebKit sono provati a mano	Dichiarato in `docs/08` §8. Nell'ambiente di verifica il motivo è verificabile: la CDN da cui `playwright-core` scarica gli altri motori risponde **403** alla politica di rete, quindi Firefox e WebKit non sono installabili lì. **Riverificato il 2026-10-08**: `npx playwright install firefox` fallisce con «Download failure», e `cdn.playwright.dev` risponde ancora 403 al tunnel. Non è una scelta, è un vincolo dell'ambiente, e resta tale
D-08	~~Traduzione inglese parziale~~ — **sanata**: vedi S-13	—	Tutti e **16 i documenti** sono in inglese. `verifica-documenti` e `verifica-affermazioni` leggono `docs/en/` come l'italiano: un disaccordo fra le due lingue fa fallire la batteria
D-09	Conformità WCAG 2.1 AA non verificabile integralmente in modo automatico	Restano fuori il testo dentro gli SVG (8 888 elementi), quello su una vera **immagine** di sfondo (108) e tutto ciò che richiede giudizio umano. Il testo su **gradiente** è entrato nella misura: 591 elementi, valutati sulla tappa peggiore del gradiente	Gli elementi saltati sono **contati** e riportati a ogni esecuzione

#	Difformità	Impatto	Trattamento
D-10	Il tag `bsi-v181` punta al commit sbagliato	Il workflow di pubblicazione, avviato senza compilare il campo della versione, ha usato il proprio valore predefinito — `bsi-v181` — e, essendo rieseguibile per progetto, ha **spostato quel tag** dal commit `cb0cf03` a quello della `bsi-v188`, allegandole anche i pacchetti della v187. Gli allegati estranei sono stati rimossi. **Il tag non è riportabile indietro da un'app**: GitHub rifiuta che un'app scriva un ref verso un commit il cui `.github/workflows/*` differisce da quello corrente, e a `cb0cf03` quel file differisce di 319 righe. Tre strade provate, tutte bloccate dalla stessa regola: `git push --force` («refusing to allow a GitHub App to create or update workflow»), `PATCH /git/refs` con `force` (403), e cancella-e-ricrea — dove il **403 arriva già sulla creazione di un tag di prova** sullo stesso commit, perciò il tag vero non è stato toccato. `GITHUB_TOKEN` non può ottenere il permesso `workflows`. **Riverificato il 2026-10-08** con il client GitHub della sessione, che usa una credenziale diversa: `PATCH /git/refs/tags/bsi-v181` risponde **403 «Write access to this GitHub API path is not permitted through this proxy»**. È un secondo blocco, indipendente dal primo: anche senza la regola sui file di workflow, la scrittura sui riferimenti git non passa. Il tag punta a `64e8c316`, dovrebbe puntare a `cb0cf03e`	I tre pacchetti allegati alla release sono quelli giusti; va ignorato l'archivio «Source code (zip)», e il codice della `bsi-v181` si scarica dal commit `cb0cf03`. Dichiarato nella voce `bsi-v181` del CHANGELOG e nel testo della release, con il comando che rimette il tag da credenziali personali. Il valore predefinito è stato togliato e il workflow ora **rifiuta** una versione che non coincida con quella di `sw.js` e con il nome dei pacchetti — provato nei due versi

Il contrasto, da 1 069 difetti a zero — come è emerso e come è stato chiuso

Fino alla versione `bsi-v168` questo documento dichiarava la conformità WCAG «verificata a campione» e il banco riportava **0 difetti**. Erano due affermazioni vere e insieme prive di significato: il banco ispezionava solo gli elementi **visibili**, e `index.html` mostra una sezione alla volta. Su 19 751 elementi di testo ne stava guardando **41**.

Percorrendo tutte e 87 le sezioni sono emersi **1 069** difetti di contrasto. Non erano comparsi: c'erano sempre stati.

Oggi sono **zero**, misurati con lo stesso banco su tutte le sezioni — che oggi sono 88. Non per sostituzione in blocco — quella strada, tentata una volta, aveva introdotto 997 difetti nuovi — ma risalendo ogni volta alla **causa comune** e rimisurando dopo ogni modifica. I **40 campi privi di etichetta** sono anch'essi scesi a **zero**.

Le cause più gravi non erano colori sbagliati, ma **token del tema che valevano lo stesso dello sfondo**:

Difetto strutturale	Effetto misurato
`--g800` vale `#0d1522`, **identico a** `--bg` — e la regola `a { color: var(--g800) }` lo applicava a ogni link	Ogni link senza colore proprio era a contrasto **1:1**: invisibile
`--g700` vale `#16263d`, quasi identico a `--white` (`#16273e`)	Testo a **1,01:1**
Pulsanti che cambiavano **sfondo** senza cambiare **colore del testo**	Lo stato selezionato (o quello deselezionato, a seconda del pannello) diventava illeggibile: **1,39:1**
`lightenColor(hex,pct){ return hex; }`	Una funzione chiamata «schiarisci» che restituiva il colore **invariato**: uno stub mai finito, probabilmente nato proprio per questo problema
Pannello di risposta del quiz: fondo chiaro, testo chiaro	Lo studente **non poteva leggere la risposta corretta**

Il rimedio strutturale è stato un token esplicito — `--testo-forte` — per il testo in evidenza su superficie scura, applicato ai soli usi come *colore del testo*: `--g800` e `--g700` restano dove servono come sfondo o bordo.

Causa	Difetti	Rimedio
Rete di sicurezza del tema scuro con specificità superiore a quella dei componenti: il testo destinato alle schede **bianche** veniva forzato a `#d4dce6`, contrasto 1,38:1	159	Riscritta con `:where()` (specificità zero): protegge il testo senza colore proprio e smette di combattere quello che ce l'ha
Accento per categoria usato come **testo** su intestazione scura, con ripiego `#0d1522` (quasi nero): 1,13:1	~400	`bsiAccentoLeggibile()` conserva la tinta e alza la luminosità quanto basta. Applicata nei 14 punti in cui l'accento viene scelto — non nei dati, così vale anche per le voci future

Causa	Difetti	Rimedio
Due tinte usate solo su fondo scuro, sotto soglia per poco (3,75:1 e 3,04:1)	139	Sostituite dopo aver **verificato** che non comparissero mai su fondo chiaro
Zebratura di tabella bianco/scurissimo con testo sempre chiaro: le righe bianche erano illeggibili	57	Due toni scuri, coerenti col tema
Celle della tavola periodica appena sotto soglia (4,46:1 e 3,88:1)	43	Testo a `#eceff4`, verificato su tutte e nove le categorie
`"#var(--text)"` — un `#` di troppo rende il colore invalido: in SVG ripiega sul nero, in canvas l'assegnazione viene **ignorata** e resta il colore precedente	8 punti	Sostituito col valore reale

Il verso della correzione dipende dalla superficie, non dal colore. Il primo tentativo schiariva sempre: ha tolto trenta difetti sull'intestazione scura e ne ha messi diciotto sul riquadro chiaro che usa lo stesso accento. La funzione ora scurisce sui fondi chiari e schiarisce su quelli scuri.

La coda finale: 80 → 0. Gli ultimi ottanta erano dispersi su 33 cause distinte e sembravano lavoro di rifinitura. Non lo erano: sotto ce n'erano ancora tre strutturali.

Causa finale	Rimedio
Il «correttore automatico di contrasto» produceva difetti invece di toglierli. Usava la luminosità *percepita* al posto della luminanza WCAG, quindi le sue soglie non corrispondevano a nessun rapporto reale; **ignorava l'alfa**, così `rgba(255,180,84,.08)` su fondo scuro gli sembrava «fondo chiaro» e scuriva il testo a `#16273e` — creando un difetto a 1,21:1 sopra un sorgente che era già corretto; e guardava solo gli elementi con uno sfondo **proprio**, cioè mai il caso più comune	Riscritto: risale agli antenati **componendo l'alfa** per ottenere il fondo effettivo, misura il rapporto WCAG vero, e delega la correzione a `bsiAccentoLeggibile()`
`--g900` vale `#0d1522` — **lo stesso di `--bg`**, esattamente come `--g800`: l'eredità di una scala di grigi nata per il tema *chiaro*, dove `--g900` era «il testo più scuro». Capovolto il tema, quel ruolo non esiste più	Gli usi come **colore del testo** (oggi 30 regole) passano a `--testo-forte`; i 16 usi come **sfondo** restano dove sono
Un colore scelto dai dati usato come **sfondo** con il testo fissato a `#fff` nel codice: funziona finché la tinta è scura e smette appena qualcuno aggiunge `#e65100`	`bsiEtichettaLeggibile()` — il duale della funzione precedente: sceglie il testo guardando il fondo e, se nessuno dei due estremi basta, **scurisce il fondo conservando la tinta**, perché un'etichetta deve restare riconoscibile per colore

I passaggi misurati col banco ufficiale, non stimati: **80 → 27 → 26 → 10 → 4 → 0**. Ogni scalino è una rimisura completa su tutte le sezioni dopo un gruppo di modifiche; nessuna cifra qui sopra è attribuita a una causa singola, perché le cause non sono state misurate una per una.

> Il numero resta **registrato come riferimento** in `evidence/accessibilita-riferimento.json`. Zero non è un traguardo da archiviare: è il valore che il banco difende. Se una modifica futura ne reintroduce anche uno solo, la batteria **fallisce**.
>
> Resta fuori ciò che l'automatismo non può giudicare: il testo dentro gli SVG e quello su fondo a gradiente (D-09). Quegli elementi sono **contati** e riportati a ogni esecuzione, non taciuti.

4-bis. Difformità sanate con questa versione

Sono elencate perché riguardavano **l'apparato di verifica stesso**, e una dichiarazione di conformità che taccia sui difetti della propria evidenza non vale la carta su cui è scritta.

#	Difformità sanata	Perché contava
S-01	32 dei 39 banchi non erano nel repository: stavano nell'area di lavoro della sessione in cui erano stati scritti	Rendeva **falsa** la frase su cui poggia questo documento — «chiunque può rieseguirli». Chi clonava ne trovava 7 su 39. Ora stanno in `tools/banchi/`
S-02	Un banco assente veniva contato ma non faceva fallire la batteria	Una batteria in cui mancavano *tutti* i banchi avrebbe dichiarato «CONFORME» con zero fallimenti. Ora un banco assente rende l'esito **NON CONFORME**
S-03	`package.json` e `package-lock.json` erano esclusi dal repository	`npm install`, comando documentato per riprodurre l'evidenza, non installava nulla: la batteria falliva al primo banco con *Cannot find module*
S-04	La verifica di accessibilità ispezionava la sola sezione visibile	Su `index.html` significava una sezione su 87. Il «0 difetti» valeva per la Dashboard, non per l'applicazione
S-05	`SCI-10` era definito due volte nella matrice, e i totali di copertura erano errati (36 dichiarati su 37 definiti)	Il numero che un valutatore legge per primo era sbagliato. Un banco ora lo verifica
S-06	50 collegamenti rotti su 127 nei pacchetti di consegna	Documentazione consegnata a terzi con riferimenti che non portavano da nessuna parte
S-07	15 PDF allegabili fermi ai documenti 00-05 e a una versione precedente	Un allegato obsoleto afferma cose false con l'aria di essere autorevole
S-08	Il banco sulle affermazioni numeriche guardava **solo i documenti italiani**	La serie inglese derivava indisturbata: `docs/en/05` dichiarava «84 sections» con 87 sezioni nell'app, ed è rimasto fermo alla versione `bsi-v146` mentre il codice era alla 171. Ora l'insieme inglese è confrontato con la misura come quello italiano, e un disaccordo fra le due lingue fa fallire il banco

#	Difformità sanata	Perché contava
S-13	La traduzione inglese si fermava a 10 documenti su 16	Un valutatore non italofono leggeva meno di due terzi della documentazione, e la parte mancante era quella operativa: modello dei dati, interfacce, deploy, specifiche funzionali, manuale. Tradotti tutti: l'insieme inglese è ora completo quanto l'italiano, e i due banchi sulla coerenza lo controllano allo stesso modo
S-12	La password del File Manager era rimasta in chiaro nella cronologia git	Toglierla dai file non la toglie dalla storia: `git log -p` la restituisce a chiunque, e finché quella password è in uso il deterrente non deterre nessuno. L'unico rimedio è **cambiarla**, ed è stato fatto: nel sorgente c'è solo la nuova impronta SHA-256, la vecchia password resta nella cronologia e non apre più niente. Verificato nel browser: la nuova apre, la vecchia no
S-10	SEC-02 — «nessun dato personale lascia il dispositivo» — era verificato controllando i due *meccanismi* di uscita, non l'uscita	Un controllo sui meccanismi dice «non vedo come potrebbe uscire», che non è «non è uscito». `audit_rete` semina un valore irripetibile in 71 depositi dei dati dell'utente, poi **usa** l'applicazione e ispeziona URL, intestazioni e corpo di ogni richiesta. Provato all'inverso con `BSI_PROVA_FUGA=1`, che provoca una fuga di proposito: il banco deve fallire, e fallisce
S-11	Il registro delle deviazioni non aveva la guardia opposta: una voce dichiarata che **non devia più** restava	Un'esenzione che nessuno revoca è un permesso acceso a vuoto, e domani coprirebbe in silenzio una struttura sbagliata messa al suo posto. Il banco ora fallisce anche per questo — verificato rimettendo una voce sanata
S-09	Il «correttore automatico di contrasto» **creava** i difetti che avrebbe dovuto togliere	Ignorava l'alfa: un fondo `rgba(255,180,84,.08)` su superficie scura gli sembrava chiaro, e scuriva con `!important` un testo che nel sorgente era già corretto. Uno strumento di rimedio che peggiora la cosa da rimediare è il difetto più difficile da vedere, perché si presenta come la soluzione

5. Criteri di accettazione applicati

Una versione è pubblicata solo se **tutti** i criteri seguenti sono soddisfatti.

- ☑ Nessun errore JavaScript non di rete su tutte le pagine
- ☑ `audit_farmaci` — nessun difetto; ogni voce senza struttura con la **ragione** dichiarata; la formula ricostruita dal grafo coincide con quella di ChEMBL per tutte le voci che dichiarano una provenienza
- ☑ `verifica_guida` — nessuna promessa della documentazione priva di riscontro nel codice
- ☑ Nessun marcatore di conflitto nei file tracciati
- ☑ Versione della cache del Service Worker incrementata e allineata a `BSI_APP_VERSION`
- ☑ Difformità note elencate al §4 di questo documento
- ☑ Rapporto di evidenza rigenerato sul commit pubblicato
- ☑ **Nessun banco assente** — un banco che non c'è non è un banco superato
- ☑ `tools/genera-pacchetti.js` chiuso con **0 collegamenti rotti**
- ☑ `tools/genera-pdf.js` rieseguito: gli allegati descrivono questa versione

6. Gestione delle modifiche

Aspetto	Procedura
Tracciabilità	Ogni modifica passa da una pull request con descrizione del difetto, della causa e della verifica
Versionamento	La versione applicativa coincide con la versione della cache del Service Worker (`bsi-vNNN`), incrementata a ogni pubblicazione
Distribuzione	Automatica da `main` via GitHub Pages; il Service Worker segnala la nuova versione alle sessioni aperte senza interromperle
Regressione	La batteria completa è rieseguita prima di ogni unione in `main`
Reversibilità	Ogni versione corrisponde a un commit; il ripristino è un `git revert`

---

7. Proprietà intellettuale e licenze

Tutti i componenti di terze parti sono distribuiti con licenze **permissive** (MIT, BSD-3-Clause, CC0, Public Domain), reciprocamente compatibili e prive di vincoli copyleft forti. La distinta completa, con impronte e obblighi residui, è in `07-SBOM.md` e nel formato CycloneDX `evidence/sbom.cdx.json`.

Il codice proprio del progetto è opera dell'autore.

---

8. Trattamento dei dati

L'applicazione è **local-first**: non esiste un backend, non vengono raccolti dati personali e non è presente alcun identificativo persistente. I dati generati dall'utente (note, chat, chiavi API, file caricati) restano nel `localStorage` e nell'IndexedDB del dispositivo e sono cancellabili dalla stessa interfaccia, in modo selettivo.

Le chiamate di rete sono opzionali e dirette a servizi pubblici documentati (§4 di `07-SBOM.md`). Se l'utente configura il proxy, le chiavi API risiedono sul Worker di sua proprietà e non sul dispositivo.

Dettaglio in `03-Security-Privacy-Compliance.md`.

---

9. Firma

Il presente documento è redatto dall'autore del software, che ne assume la responsabilità. Non costituisce certificazione da parte di un organismo terzo, e non viene presentato come tale.

Chiunque può verificare quanto dichiarato rieseguendo la procedura del §5 di `evidence/RAPPORTO-VERIFICA.md` sul commit indicato, e confrontando le impronte SHA-256 dei file.

Samuele Pio Provenzano _Versione `bsi-v200`._

Riferimento delle interfacce — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Versione descritta	`bsi-v200`
Scopo	Documentare le interfacce HTTP del progetto e le API esterne consumate dall'applicazione.

0. Premessa necessaria

BioSpecInfo non espone API. È una Progressive Web App interamente client-side, servita come file statici: non esiste un backend, non esiste un server applicativo, non esistono endpoint REST del prodotto.

Questo documento sarebbe quindi vuoto, se non fosse per due interfacce che esistono davvero e che un valutatore ha diritto di vedere documentate:

	Cosa è	Documentato in
Proxy Spectra	Un Cloudflare Worker **opzionale**, di proprietà dell'utente, che inoltra le chiamate ai fornitori AI. È l'unica interfaccia HTTP scritta in questo progetto.	§1, §2
API esterne consumate	Servizi pubblici di terzi che l'applicazione interroga, sempre in modo opzionale.	§3
API interne JavaScript	I punti di estensione del codice, per chi deve intervenirci.	§4

> Una specifica OpenAPI di endpoint inesistenti sarebbe documentazione plausibile e falsa. Qui è descritto soltanto ciò che esiste.

1. Proxy Spectra — panoramica

Il proxy risolve due problemi che dal browser non hanno altra soluzione:

1. **CORS.** Diversi fornitori AI non inviano le intestazioni che permettono a

una pagina web di chiamarli direttamente. Dal Worker la chiamata parte da un server e il vincolo non si applica.

1. **Custodia delle chiavi.** Con il proxy la chiave API risiede fra i segreti

del Worker e non sul dispositivo dell'utente.

È **opzionale**: senza proxy l'applicazione funziona usando le chiavi inserite localmente dall'utente.

Deploy

```
cd proxy
npx wrangler login
npx wrangler deploy
npx wrangler secret put GROQ_KEYS     # una o più chiavi, separate da virgola
```

Configurazione

Variabile	Tipo	Default	Descrizione
`ORIGINI`	segreto/variabile	*(vuoto = tutte)*	Elenco di origini ammesse, separate da virgola. **Lasciarlo vuoto rende il proxy aperto a chiunque:** sconsigliato.
`LIMITE_IP`	variabile	`20`	Richieste per singolo IP nella finestra temporale
`TETTO_GIORNO`	variabile	`2000`	Richieste totali giornaliere
`GROQ_KEYS`, `GEMINI_KEYS`, `ANTHROPIC_KEYS`, `OPENAI_KEYS`, `XAI_KEYS`, `NVIDIA_KEYS`, `ZAI_KEYS`, `DEEPSEEK_KEYS`	segreti	—	Chiavi API. Più chiavi separate da virgola: esaurita la quota della prima (429), il proxy passa alla successiva **nella stessa richiesta**.

Endpoint

Metodo	Percorso	Descrizione
`OPTIONS`	`/*`	Preflight CORS
`GET`	`/stato`	Quali fornitori hanno una chiave configurata
`GET` `POST`	`/{fornitore}/{percorso...}`	Inoltro al fornitore

Fornitori riconosciuti: `anthropic` · `gemini` · `groq` · `xai` · `nvidia` · `zai` · `openai` · `deepseek`

Codici di stato

Codice	Significato
`200`	Risposta del fornitore, inoltrata così com'è
`204`	Preflight CORS accettato
`400`	Corpo della richiesta illeggibile
`403`	Origine non ammessa dall'elenco `ORIGINI`
`404`	Fornitore sconosciuto
`405`	Metodo diverso da GET, POST o OPTIONS
`429`	Limite per IP o tetto giornaliero superato

Codice	Significato
`503`	Nessuna chiave configurata per quel fornitore

Scelte di sicurezza

- **Nessuna intestazione inoltrata ciecamente.** Passano solo

`anthropic-version` e `anthropic-beta`; qualunque credenziale inviata dal client resta fuori. Un proxy che inoltra tutto è un relay aperto.

- **Il parametro `key` non viene propagato.** Se un client lo mette in query

string, il proxy lo scarta: la chiave è quella del server.

- **Il corpo viene letto una volta sola e conservato**, perché per ritentare con

una chiave diversa occorre poterlo rimandare — e uno stream si consuma.

---

2. Specifica OpenAPI 3.0

```
openapi: 3.0.3
info:
  title: Proxy Spectra
  version: "1.0.0"
  description: |
    Proxy opzionale (Cloudflare Worker) per le chiamate ai fornitori di modelli
    linguistici. Elimina i blocchi CORS e mantiene le chiavi API sul server
    anziché sul dispositivo dell'utente.

    Non fa parte dell'applicazione BioSpecInfo: viene pubblicato dall'utente sul
    proprio account Cloudflare, e l'applicazione lo usa solo se ne conosce
    l'indirizzo.
  license:
    name: Proprietary — All rights reserved (Samuele Pio Provenzano)
servers:
  - url: https://{nomeworker}.{account}.workers.dev
    description: Istanza dell'utente
    variables:
      nomeworker: { default: spectra-proxy }
      account:    { default: esempio }

tags:
  - name: Stato
    description: Interrogazione della configurazione, senza rivelare segreti
  - name: Inoltro
    description: Inoltro delle richieste ai fornitori di modelli

paths:
  /stato:
    get:
      tags: [Stato]
      summary: Fornitori configurati
      description: |
        Elenca i fornitori per i quali il proxy possiede almeno una chiave.
        Non rivela alcuna chiave. L'applicazione lo usa per mostrare soltanto i
        modelli che funzionano davvero, invece di farli fallire uno per uno.
      responses:
        "200":
          description: Elenco dei fornitori attivi
          content:
            application/json:
              schema:
                type: object
                required: [fornitori]
                properties:
                  fornitori:
                    type: array
                    items:
                      type: string
                      enum: [anthropic, gemini, groq, xai, nvidia, zai, openai, deepseek]
              examples:
                dueChiavi:
                  summary: Due fornitori configurati
                  value: { fornitori: [groq, gemini] }
                nessuna:
                  summary: Worker pubblicato ma senza segreti
                  value: { fornitori: [] }
```

```
      "403":
        $ref: "#/components/responses/OrigineNonAmmessa"

  /{fornitore}/{percorso}:
    parameters:
      - name: fornitore
        in: path
        required: true
        description: Fornitore di destinazione
        schema:
          type: string
          enum: [anthropic, gemini, groq, xai, nvidia, zai, openai, deepseek]
      - name: percorso
        in: path
        required: true
        description: |
          Percorso dell'endpoint presso il fornitore, ricomposto così com'è.
          Esempi: `openai/v1/chat/completions` per Groq,
          `v1beta/models/gemini-3.6-flash:streamGenerateContent` per Gemini.
        schema: { type: string }
        example: openai/v1/chat/completions
    post:
      tags: [Inoltro]
      summary: Inoltra una richiesta al fornitore
      description: |
        Il corpo viene trasmesso invariato. Il proxy aggiunge la chiave nel modo
        richiesto da quel fornitore (intestazione `Authorization: Bearer`,
        `x-api-key`, oppure parametro in query) e, se la chiamata fallisce per
        quota esaurita, ritenta con la chiave successiva nella stessa richiesta.
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              description: Corpo previsto dall'API del fornitore, trasmesso invariato
            examples:
              groq:
                summary: Completamento in streaming su Groq
                value:
                  model: llama-3.3-70b-versatile
                  stream: true
                  messages:
                    - role: user
                      content: Quali bande IR identificano un acido carbossilico?
      responses:
        "200":
          description: |
            Risposta del fornitore, inoltrata senza modifiche (tipicamente
            `text/event-stream` quando `stream: true`).
          content:
            application/json:
              schema: { type: object }
            text/event-stream:
              schema: { type: string }
        "400": { $ref: "#/components/responses/CorpoIlleggibile" }
        "403": { $ref: "#/components/responses/OrigineNonAmmessa" }
        "404": { $ref: "#/components/responses/FornitoreSconosciuto" }
        "429": { $ref: "#/components/responses/TroppeRichieste" }
        "503": { $ref: "#/components/responses/ChiaveAssente" }
```

```
    get:
      tags: [Inoltro]
      summary: Inoltra una richiesta in lettura al fornitore
      description: |
        Serve, fra l'altro, alla risoluzione del modello Gemini, che interroga
        `ListModels` in GET e senza chiave nel browser non potrebbe passare.
      responses:
        "200":
          description: Risposta del fornitore
          content:
            application/json:
              schema: { type: object }
        "403": { $ref: "#/components/responses/OrigineNonAmmessa" }
        "404": { $ref: "#/components/responses/FornitoreSconosciuto" }
        "429": { $ref: "#/components/responses/TroppeRichieste" }
        "503": { $ref: "#/components/responses/ChiaveAssente" }

components:
  schemas:
    Errore:
      type: object
      required: [errore]
      properties:
        errore:
          type: string
          description: Messaggio in italiano, pensato per essere mostrato all'utente
  responses:
    OrigineNonAmmessa:
      description: L'origine della richiesta non è nell'elenco `ORIGINI`
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Origine non ammessa da questo proxy." }
    FornitoreSconosciuto:
      description: Il primo segmento del percorso non corrisponde a un fornitore noto
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Fornitore sconosciuto: pippo" }
    CorpoIlleggibile:
      description: Il corpo della richiesta non è stato leggibile
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Corpo della richiesta illeggibile." }
    TroppeRichieste:
      description: Superato il limite per IP o il tetto giornaliero
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example: { errore: "Troppe richieste da questo indirizzo: riprova fra un minuto." }
    ChiaveAssente:
      description: Il proxy non ha una chiave configurata per quel fornitore
      content:
        application/json:
          schema: { $ref: "#/components/schemas/Errore" }
          example:
            errore: "Il proxy non ha una chiave per groq. Impostala con: wrangler secret put
GROQ_KEYS"
```

```
security: []
# L'autenticazione del CLIENT è volutamente assente: il controllo di accesso è
# l'elenco ORIGINI più i limiti per IP. Aggiungere un token lato client
# significherebbe scriverlo in una pagina statica, cioè renderlo pubblico.
```

3. API esterne consumate dall'applicazione

Tutte **opzionali**: senza rete l'applicazione resta pienamente funzionante sui dati locali, e lo dichiara all'utente.

Servizio	Endpoint usati	A cosa serve	Se non risponde

PubChem PUG-REST	`/compound/name/{nome}/property/...`
	 `/compound/smiles/{smiles}/SDF?record_type=3d`
	 `/compound/cid/{cid}/PNG`
		Nome IUPAC, formula, CAS, pittogrammi GHS, conformeri 3D, immagini 2D	L'app usa i dati tabulati locali e lo segnala

RCSB PDB	`https://files.rcsb.org/download/{pdb}.pdb`	Strutture proteiche per il visore 3D	Messaggio esplicito nel visore
NASA / ESA	Immagini e dati pubblici	Modulo di astrochimica	Il modulo mostra i dati tabulati
Fornitori AI	A scelta dell'utente, o via proxy	Risposte dell'assistente	L'assistente lo dichiara e propone un'alternativa raggiungibile

Nessuna di queste chiamate trasmette dati personali. Non sono presenti identificativi persistenti né telemetria attiva (`BSI_TELEMETRY_URL` è vuoto).

4. API interne JavaScript

Punti di estensione per chi deve intervenire sul codice. Sono globali sul `window` perché l'applicazione è un insieme di script classici, non moduli ES.

`window.BSISpettri` — motore spettrale

Funzione	Firma	Descrizione
`gruppi`	`(smiles, callback)`	Riconosce i gruppi funzionali via SMARTS/RDKit. Richiama con `null` se RDKit non è disponibile: il chiamante decide se ripiegare o dichiarare che non può calcolare.
`gruppiSync`	`(smiles) → oggetto | null`	Versione sincrona: risponde solo se la molecola è già in cache
`bandeDaGruppi`	`(gruppi) → banda[]`	Tabella delle bande IR `{n, pk, w, i, f}`

Funzione	Firma	Descrizione
`svgIR`	`(bande, opzioni) → string`	Spettro IR in trasmittanza, SVG, deterministico
`legendaIR`	`(bande) → string`	Assegnazioni in HTML
`quandoPronto`	`(callback)`	Notifica quando l'analisi di una molecola è disponibile, per ridisegnare
`precarica`	`()`	Scalda RDKit senza che nessuno stia aspettando

`window.bsiProxy` — stato del proxy

Funzione	Descrizione
`url()`	Indirizzo configurato, stringa vuota se assente
`stato()`	Promessa: interroga `/stato` e memorizza i fornitori coperti
`copre(idFornitore)`	Se quel fornitore è coperto dal proxy
`ricarica()`	Rilegge la configurazione

Altri punti notevoli

Simbolo	Descrizione
`window.bsiLoadRDKit(cb)`	Carica RDKit una volta sola e richiama con l'istanza
`window.irBandList(smiles)`	Bande IR — sostituita dal motore nuovo, mantiene la firma originale
`window.makeIRsvg(bande, colore)`	Disegno IR — idem
`window.nmrPeakList(smiles)`	Picchi ^{1}H con integrazione
`window.bsiCancellaDati(gruppi)`	Cancellazione selettiva dei dati salvati
`window.bsiOpenUpdate()`	Apre la finestra degli aggiornamenti

Documento aggiornato alla versione `bsi-v200`.

Modello dei dati — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Versione descritta	`bsi-v200`
Scopo	Documentare dove risiedono i dati, con quale struttura, e quale ciclo di vita hanno.

---

0. Premessa necessaria

BioSpecInfo non ha un database su server. Non esiste un backend, quindi non esistono tabelle relazionali gestite centralmente, migrazioni, o utenti memorizzati da qualche parte.

Esiste però un modello dei dati reale, e un valutatore ha diritto di vederlo documentato. Si articola su tre livelli:

Livello	Tecnologia	Cosa contiene	Dove vive
Preferenze e progressi	`localStorage`	Impostazioni, chiavi API, cronologia AI, avanzamento nello studio	Browser dell'utente
Archivio documenti	IndexedDB	Foto, documenti, sezioni HTML, note, registro attività	Browser dell'utente
Dataset didattici	SQLite via `sql.js`	Tabelle di esercitazione, create in memoria a ogni sessione	Memoria, non persistito

Tutti i dati restano sul dispositivo. Non vengono trasmessi, non esiste un identificativo utente lato server, e l'utente può cancellarli selettivamente dall'interfaccia.

---

1. Diagramma delle entità

```
erDiagram
    DISPOSITIVO ||--|| PREFERENZE : "possiede"
    DISPOSITIVO ||--|| ARCHIVIO : "possiede"
    DISPOSITIVO ||--o{ SESSIONE_SQL : "apre"

    PREFERENZE {
        string bsi_ai_provider "fornitore AI scelto"
        json   bsi_api_keys "chiavi per fornitore"
        string bsi_proxy_url "indirizzo del proxy"
        json   bsi_ai_threads "conversazioni salvate"
        json   bsi_ai_memory "memoria dell-agente"
        json   bsi_prov_ko "fornitori irraggiungibili"
        json   bsi_modelli_ko "modelli ritirati"
        json   bsi_tetti "tetti di token appresi"
        json   bsi_srs "ripetizione spaziata"
        json   bsi_quiz_progress "avanzamento quiz"
        string bsi_section "ultima sezione aperta"
    }

    ARCHIVIO ||--o{ PHOTOS : contiene
    ARCHIVIO ||--o{ DOCS : contiene
    ARCHIVIO ||--o{ SECTIONS : contiene
    ARCHIVIO ||--o{ NOTES : contiene
    ARCHIVIO ||--o{ ACTIVITY : registra

    PHOTOS {
        int    id PK "autoIncrement"
        string name
        string type "MIME"
        int    size "byte"
        string data "data URL"
        int    added "epoch ms"
    }
    DOCS {
        int    id PK "autoIncrement"
        string name
        string type
        int    size
        string data "data URL"
        int    date "epoch ms"
    }
    SECTIONS {
        int    id PK "autoIncrement"
        string name
        string icon
        string html "contenuto"
        int    size "caratteri"
        int    added "epoch ms"
    }
    NOTES {
        string key PK "sempre main"
        string text
        int    updated "epoch ms"
    }
    ACTIVITY {
        int    id PK "autoIncrement"
        string icon
```

```
        string msg
        string time "data locale"
    }
```

---

2. Preferenze e stato — `localStorage`

2.1 Assistente AI

Chiave	Tipo	Contenuto	Ciclo di vita
`bsi_ai_provider`	stringa	Identificativo del fornitore selezionato	Finché l'utente non cambia
`bsi_api_keys`	JSON	`{ idFornitore: chiave }`	Fino a cancellazione esplicita
`bsi_api_key`	stringa	Chiave singola, formato storico	Migrata a `bsi_api_keys`
`bsi_proxy_url`	stringa	Indirizzo del proxy	Fino a «Scollega»
`bsi_ai_threads`	JSON	`{ threads: [...], activeId }` — max **30 chat**, **100 messaggi** ciascuna	Potatura automatica
`bsi_ai_memory`	JSON	Fatti che l'agente ricorda fra le sessioni	Cancellabile
`bsi_ai_history`	JSON	Cronologia sintetica	Cancellabile
`bsi_nucleo`	stringa	Modalità «nucleo» attiva	—

2.2 Ciò che l'assistente impara sui fornitori

Queste tre chiavi sono il motivo per cui l'assistente migliora con l'uso invece di ripetere gli stessi errori. Hanno una scadenza perché un guasto di oggi non deve essere una condanna definitiva.

Chiave	Struttura	Scadenza	A cosa serve
`bsi_prov_ko`	`{ idFornitore: { t: epoch, n: conteggio } }`	**24 ore**	Fornitori che da questo dispositivo non rispondono. Retrocessi nella scelta, mai rimossi.
`bsi_modelli_ko`	`{ impronta: { modelli: [...], t } }`	**7 giorni**	Modelli ritirati dal fornitore, per non riproporli. Indicizzati sull'impronta della chiave.
`bsi_tetti`	`{ idFornitore: tettoToken }`	Permanente	Tetto di token appreso dal messaggio d'errore, anziché codificato

2.3 Studio e progressi

Chiave	Contenuto
`bsi_srs`	Ripetizione spaziata: schede, intervalli, prossima revisione
`bsi_sm2`	Parametri dell'algoritmo SM-2
`bsi_quiz_progress`, `bsi_quiz_history`	Avanzamento e storico dei quiz

Chiave	Contenuto
`bsi_studypath`	Percorso di studio scelto
`bsi_notes`, `bsi_note_<sezione>`	Note personali, globali e per sezione
`bsi_ds_prog`, `bsi_ds_proj`	Avanzamento del modulo di data science
`bsi_section`	Ultima sezione aperta, per riprendere da lì
`bsi_guide_v3`	Guida introduttiva già vista

2.4 Identificativi e licenza

Chiave	Contenuto	Nota sulla riservatezza
`bsi_device_id`	Identificativo casuale locale	**Non viene trasmesso.** Serve solo a distinguere le installazioni sullo stesso browser.
`bsi_user_email`	Email, se inserita dall'utente	Inserita volontariamente, non trasmessa
`bsi_pro_license`, `bsi_trial_start`	Stato della modalità Pro	Locale
`bsi_rdkit_lib2`	Libreria molecole del laboratorio	Locale

---

3. Archivio documenti — IndexedDB

Nome database	`bsi_filemanager_v1`
Versione schema	`2`
Object store	`photos`, `docs`, `sections`, `notes`, `activity`

3.1 Struttura degli store

Store	Chiave	Campi	Note
`photos`	`id` (autoIncrement)	`name`, `type`, `size`, `data`, `added`	Immagini come data URL. Limite per file: **20 MB**
`docs`	`id` (autoIncrement)	`name`, `type`, `size`, `data`, `date`	Documenti generici
`sections`	`id` (autoIncrement)	`name`, `icon`, `html`, `size`, `added`	Pagine HTML caricate dall'utente
`notes`	`key`	`key` (`"main"`), `text`, `updated`	Voce unica
`activity`	`id` (autoIncrement)	`icon`, `msg`, `time`	Registro delle ultime operazioni

> Lo store `files`, presente in versioni precedenti dello schema, non è più creato dalla versione 2.

3.2 Apertura e indisponibilità

L'apertura di IndexedDB è **asincrona**, e in alcune configurazioni (Safari in navigazione privata, dati dei siti bloccati) **non avviene mai**. Il codice attende una promessa anziché leggere una variabile che potrebbe non essere ancora valorizzata, e distingue due casi:

- **lettura** con archivio assente → restituisce un insieme vuoto, la pagina

mostra «ancora niente»;

- **scrittura** con archivio assente → l'errore raggiunge chi ha premuto il

pulsante, e in cima alla pagina compare un avviso persistente che nulla verrà conservato.

Se un'altra scheda richiede un aggiornamento di versione, la connessione viene chiusa (`onversionchange`) per non bloccare entrambe.

---

4. Dataset didattici — SQLite in memoria

Il modulo di esercitazione sui dati crea un database SQLite **in memoria** tramite `sql.js` (SQLite compilato in WebAssembly). Le tabelle sono generate a ogni sessione dai dataset inclusi nel repository e **non vengono persistite**: chiudendo la scheda spariscono.

Questa scelta è deliberata: l'esercitazione deve partire da uno stato noto ogni volta, e nessun dato dell'utente finisce in un database che non ha creato.

---

5. Dati di riferimento — costanti del codice

I dati scientifici non sono in un database ma in strutture costanti dentro `index.html`, caricate con l'applicazione. Sono di sola lettura e versionate insieme al codice.

Struttura	Contenuto	Verifica
`FARM_DATA`	**263 farmaci**: nome, categoria, SMILES, peso molecolare, meccanismo d'azione, indicazioni, effetti avversi, classe; per 36 voci anche `chembl` e `formula` del record di provenienza	`audit_farmaci` — struttura ⟷ peso, duplicati, e formula ricostruita dal grafo contro quella di ChEMBL
`UV_DATA`	Cromofori, λmax, coefficienti di estinzione, regole di Woodward	`audit_dati`
`MS_DATA`	Frammentazioni caratteristiche per classe	`audit_dati`
`MOLECOLE_ESAME`	Molecole d'esame tabulate	`test_spettri`
`PHYS_SIMBOLI`	Costanti fisiche con simbolo e unità	`test_costanti`

Struttura	Contenuto	Verifica
`SMARTS`, `SMARTS_H`	Pattern per il riconoscimento dei gruppi funzionali	`test_spettri`

---

6. Ciclo di vita e cancellazione

L'utente può cancellare i dati **per gruppi**, dall'interfaccia, senza azzerare tutto. I gruppi sono definiti in `bsiCancellaDati()`.

Gruppo	Cosa rimuove
`chiavi`	Chiavi API, fornitore scelto, indirizzo del proxy, memoria sui fornitori irraggiungibili
`chat`	Conversazioni e memoria dell'assistente
`studio`	Ripetizione spaziata, quiz, percorso di studio, note
`tutto`	Ogni chiave `bsi_*` e l'intero archivio IndexedDB

La cancellazione dell'archivio prosegue anche se una singola voce fallisce, e riporta quante non sono state eliminate: un'operazione «elimina tutto» che si ferma a metà senza dirlo sarebbe peggio del problema che risolve.

---

7. Conservazione e limiti di spazio

Vincolo	Valore	Comportamento al superamento
`localStorage`	5–10 MB secondo il browser	Le scritture falliscono. L'app resta operativa: le chat vengono potate (30 chat, 100 messaggi), e ogni scrittura è protetta.
IndexedDB	Quota concessa dal browser	Errore riportato a chi ha premuto, con avviso persistente
Singola immagine	20 MB	Rifiutata con messaggio esplicito, il caricamento delle altre prosegue

Il comportamento a spazio esaurito è verificato dai banchi `audit_quota` e `test_filemanager`: dieci pagine su dieci restano utilizzabili con `localStorage.setItem` forzato a fallire.

Documento aggiornato alla versione `bsi-v200`.

Guida al deploy — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Versione descritta	`bsi-v200`
Scopo	Procedura operativa per pubblicare, aggiornare e ripristinare l'applicazione.

0. Cosa si pubblica, in pratica

BioSpecInfo è un insieme di **file statici**. Non c'è nulla da compilare, nessun processo da tenere acceso, nessun database da provvisionare.

Build	Nessuna. I file del repository *sono* l'applicazione.
Runtime server	Nessuno
Variabili d'ambiente	Nessuna per l'applicazione. Solo il proxy opzionale ne ha (§4).
Database	Nessuno lato server
Requisito del server	Servire file statici su HTTPS con i MIME corretti

L'unico requisito non ovvio: il server deve restituire `application/wasm` per i file `.wasm`, altrimenti RDKit e SQLite non si avviano. GitHub Pages, Netlify, Vercel e Cloudflare Pages lo fanno già.

1. Ambienti

Ambiente	Come si ottiene	A cosa serve
Sviluppo	`python3 -m http.server 8899` dalla radice	Lavoro quotidiano e banchi di prova
Anteprima	Un ramo pubblicato su Pages, o `npx serve`	Verifica prima dell'unione
Produzione	GitHub Pages dal ramo `main`	`samupropio1-ship-it.github.io/BioSpecInfo-v11/`

> **Perché serve un server anche in sviluppo.** Aprire `index.html` con un doppio clic (`file://`) non funziona: RDKit e SQLite sono moduli WebAssembly e richiedono un contesto HTTP. È il motivo per cui anche i banchi di prova girano contro un server locale.

2. Pubblicazione in produzione

2.1 Procedura

```
# 1. dal ramo di lavoro, batteria completa
python3 -m http.server 8899 &
node tools/genera-evidenza.js          # deve chiudersi con 0 falliti

# 2. incrementare la versione — con lo strumento, MAI a mano
node tools/porta-versione.js bsi-v<NNN+1> --prova   # prima si guarda
node tools/porta-versione.js bsi-v<NNN+1>           # poi si scrive

# 3. rigenerare SBOM, evidenza, PDF e pacchetti — IN QUEST'ORDINE
#    I pacchetti imbustano i PDF e l'evidenza: se si costruiscono prima,
#    portano l'impaginato e il rapporto della versione PRECEDENTE.
#    genera-pacchetti adesso se ne accorge e si ferma, ma meglio non arrivarci.
node tools/genera-sbom.js
node tools/genera-evidenza.js       # la batteria; serve un server su :8899
node tools/genera-pdf.js            # 36 PDF; ogni piè di pagina porta versione e commit
node tools/genera-pacchetti.js      # i tre pacchetti; fallisce su link rotti, PDF o evidenza
vecchi

# 4. unire in main
git add -A && git commit -m "…"
git push -u origin <ramo>
#    poi pull request e squash merge
```

GitHub Pages pubblica automaticamente da `main`: il workflow *pages build and deployment* impiega circa un minuto.

2.1-bis Pubblicare la release della versione

L'applicazione è già online appena `main` si aggiorna; la **release** serve a un'altra cosa: dare a chi valuta un punto fermo da scaricare — un tag, le note di versione e i pacchetti di consegna allegati.

Si pubblica dal repository, non da fuori:

> **Actions → release → Run workflow**, campo `versione`: `bsi-v<NNN>`.

Il workflow crea il tag sul commit corrente, apre la release e le allega i tre pacchetti di `consegna/`. L'archivio completo — `models/` incluse, circa 124 MB — lo genera GitHub da sé come *Source code (zip)*.

Perché dal repository e non da riga di comando. Le credenziali con cui una sessione remota lavora al repository sono autorizzate a scrivere sui rami e **non sui tag**: la `POST` a `git-receive-pack` per `refs/tags/…` risponde `403 Forbidden`, mentre un ramo nuovo passa. Il token di Actions, invece, scrive sul repository che lo ospita.

Quello che il workflow **si rifiuta** di fare:

Condizione	Esito
nel `CHANGELOG.md` non c'è una voce `## [bsi-v<NNN>]` con contenuto	fallisce: una release senza note sembra una versione e non dice che cosa cambia
in `consegna/` ci sono meno di tre pacchetti	fallisce: un allegato mancante renderebbe la release incompleta in silenzio
il tag non punta al commit da cui vengono i pacchetti	fallisce: una release deve avere una sola provenienza

L'ultima riga esiste per un difetto reale. Rieseguendo il workflow per aggiornare i pacchetti, `gh release edit` aggiornava note e allegati ma **non spostava il tag**: l'archivio *Source code (zip)*, che GitHub genera dal tag, restava al commit della prima pubblicazione mentre i pacchetti venivano da uno successivo. Due provenienze, e niente che lo dicesse. Ora il tag viene spostato, e un passo successivo non lo dà per buono: richiede a GitHub dove punta e confronta.

Il workflow è **rieseguibile**: se la release esiste già, aggiorna note, allegati e tag invece di fallire a metà. Spostare un tag già pubblicato non è gratis — chi l'ha già scaricato ha un contenuto diverso — quindi vale per una release appena pubblicata, non per una vecchia: per quella si pubblica una versione nuova.

2.1-ter Perché il cambio di versione non si fa con una sostituzione

Sembra l'operazione più innocua del mondo: si cerca `bsi-v<NNN>` e si scrive `bsi-v<NNN+1>`. **Per tre rilasci di fila è stato fatto così, e ogni volta ha riscritto delle frasi storiche** — frasi che raccontano un fatto accaduto a una versione passata. Quello che ne è uscito stava dentro una dichiarazione di conformità firmata:

nel documento c'era scritto	la verità
la password era stata **cambiata alla versione corrente**	era stata cambiata alla versione `bsi-v188`
il tag era stato **spostato al commit della versione corrente**	l'incidente era accaduto alla versione `bsi-v188`
le trentasei voci erano state **aggiunte nella versione corrente**	erano state aggiunte in `bsi-v188`
«incrementa `CACHE` (es. `bsi-vN` → `bsi-vN`)»	un esempio uguale a se stesso

Nessun controllo poteva accorgersene: ogni riga citava una versione che **esiste**, nel documento che descrive quella versione.

> Questa tabella è scritta con dei segnaposto e non con i numeri veri perché i due controlli di cui si parla qui sotto **la boccerebbero** — ed è la dimostrazione che funzionano.

`tools/porta-versione.js` tocca **solo** le righe che DICHIARANO la versione — intestazioni, piè di pagina, distintivo del README, nomi dei pacchetti, le due righe di codice — e tutto il resto lo **elenca** perché lo si legga. Con `--prova` non scrive niente.

E due controlli in `tools/verifica-documenti.js` chiudono la porta: nessuna frase al passato può nominare la versione corrente, e nessun esempio di aggiornamento può mostrare la stessa versione da entrambe le parti.

> **Scrivere nominando la cosa, non il numero.** «Dalle tabelle di Pretsch» invece di «dalla versione tale»: si legge meglio e non invecchia.

2.2 Le due righe della versione

`CACHE` in `sw.js` e `BSI_APP_VERSION` in `index.html` **devono coincidere**.

- `CACHE` determina se il Service Worker considera la cache obsoleta: senza

incremento, i dispositivi continuano a servire i file vecchi e la pubblicazione non arriva a nessuno.

- `BSI_APP_VERSION` è ciò che l'utente vede nella finestra «Aggiornamenti» e ciò

che il controllo confronta con il `sw.js` su GitHub.

Sono disallineate una volta in passato — la seconda era ferma da mesi — e il risultato è stato che il controllo aggiornamenti confrontava un numero sbagliato.

2.3 Come arriva l'aggiornamento agli utenti

1. Il browser rileva un `sw.js` diverso e installa il nuovo Service Worker.
2. `skipWaiting()` lo attiva subito; le cache vecchie vengono eliminate.
3. Il Service Worker **avvisa** le schede aperte (`BSI_SW_UPDATED`) — non le

ricarica d'ufficio.

1. La pagina decide: se non c'è lavoro in corso si ricarica da sola; se

l'assistente sta rispondendo o c'è testo non salvato, mostra «Aggiorna ora / Più tardi».

Chi apre l'app in quel momento riceve già la versione nuova.

---

3. Alternative di pubblicazione

L'applicazione non dipende da GitHub Pages. Qualunque host di file statici va bene, purché su HTTPS (il Service Worker lo richiede).

Piattaforma	Comando / configurazione	Note
GitHub Pages	Settings → Pages → branch `main` , cartella `/`	In uso. Gratuito, dominio personalizzabile
Netlify	`netlify deploy --prod --dir=.`	Nessun comando di build
Vercel	`vercel --prod`	Progetto statico, nessun framework
Cloudflare Pages	`npx wrangler pages deploy .`	Comodo se si usa già il proxy
VPS con nginx	Copia dei file in `/var/www/biospecinfo`	Vedi configurazione sotto

Configurazione nginx

```
server {
    listen 443 ssl http2;
    server_name biospecinfo.example.org;
    root /var/www/biospecinfo;

    # Senza questo tipo MIME, RDKit e SQLite non si avviano
    types { application/wasm wasm; }

    # sw.js non va messo in cache dal CDN: è ciò che annuncia le versioni nuove
    location = /sw.js {
        add_header Cache-Control "no-cache, no-store, must-revalidate";
    }

    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Regola valida ovunque: `sw.js` non va messo in cache

Se una CDN conserva `sw.js`, il browser continua a vedere la versione vecchia e **nessun aggiornamento arriva mai**, per quanto si pubblichi. Vale per ogni piattaforma.

---

4. Proxy opzionale — deploy separato

Il proxy è un componente **distinto**, di proprietà dell'utente, pubblicato sul suo account Cloudflare. Non è necessario per usare l'applicazione.

```
cd proxy
npx wrangler login
npx wrangler deploy
# l'indirizzo viene stampato: https://spectra-proxy.<account>.workers.dev

npx wrangler secret put GROQ_KEYS      # una o più chiavi, separate da virgola
npx wrangler secret put GEMINI_KEYS    # facoltativo
```

Poi si incolla l'indirizzo nel riquadro «Proxy» dell'assistente, dentro l'app. Gli stessi quattro passi sono disponibili **nell'applicazione**, con i comandi pronti da copiare.

Configurazione consigliata

```
# Senza questa variabile il proxy accetta richieste da qualunque origine
npx wrangler secret put ORIGINI
# valore: https://samupropio1-ship-it.github.io
```

Variabile	Default	Perché toccarla
`ORIGINI`	*(vuoto = tutte)*	**Impostarla sempre.** Un proxy aperto è utilizzabile da chiunque, a spese della tua quota.
`LIMITE_IP`	`20`	Richieste per IP nella finestra
`TETTO_GIORNO`	`2000`	Tetto complessivo giornaliero

> **Le chiavi non vanno mai in `wrangler.toml`.** Quel file è versionato: una chiave scritta lì finisce su GitHub e i bot che lo scandagliano la trovano in poche ore. Si usa `wrangler secret put`, che le tiene fuori dal repository.

---

5. Dominio e certificato

Aspetto	Situazione
HTTPS	Obbligatorio: senza, il Service Worker non si registra e l'app non funziona offline
Certificato	Fornito e rinnovato automaticamente da GitHub Pages
Dominio personalizzato	Settings → Pages → Custom domain, più un record `CNAME` verso `<utente>.github.io`

---

6. Monitoraggio

Non esiste un server da monitorare: non ci sono processi che possano cadere, né risorse che si esauriscano.

Cosa osservare	Come
Esito della pubblicazione	GitHub → Actions → *pages build and deployment*
Disponibilità	Un controllo HTTP su `index.html` (UptimeRobot o simili)
Errori lato client	Console del browser. La telemetria remota esiste ma è **disattivata** (`BSI_TELEMETRY_URL` vuoto): attivarla richiede una decisione consapevole sul trattamento dei dati.
Consumo del proxy	Cloudflare → Workers → Analytics

---

7. Ripristino e procedure d'emergenza

7.1 Tornare alla versione precedente

```
git revert <commit>
git push origin main
```

Pages ripubblica in circa un minuto. I dispositivi ricevono la versione ripristinata come un normale aggiornamento — **purché il `CACHE` cambi**: un revert che riporta anche il numero di versione all'indietro va bene, perché conta che il valore sia *diverso*, non che sia maggiore.

7.2 «Ho pubblicato ma non vedo le modifiche»

Nell'ordine:

1. **La versione è stata incrementata?** È la causa nel novanta per cento dei

casi. Senza, il Service Worker serve la cache.

1. **Pages ha concluso?** GitHub → Actions.
2. **Il `sw.js` è in cache da una CDN?** Deve rispondere `no-cache`.
3. Dall'app: menu ✨ → **Aggiornamenti** → *Controlla*, che forza un

`registration.update()`.

1. In ultima istanza: strumenti per sviluppatori → Application → Service Workers

→ *Unregister*, poi ricaricare.

7.3 «L'assistente non risponde più»

Non è un problema di pubblicazione: riguarda il fornitore AI. Nell'app, il pulsante 🔌 **Prova** misura in pochi secondi quali fornitori il dispositivo riesce davvero a contattare, senza bisogno di alcuna chiave.

7.4 Ripristino dei dati dell'utente

I dati risiedono sul dispositivo e **non esistono copie sul server**: non è possibile ripristinarli da remoto. È una conseguenza diretta dell'architettura local-first, ed è dichiarata all'utente. Le esportazioni (note, documenti) restano la sola forma di copia di sicurezza.

---

8. Lista di controllo prima di pubblicare

- ☐ `node tools/genera-evidenza.js` si chiude con **0 falliti**
- ☐ `CACHE` in `sw.js` e `BSI_APP_VERSION` in `index.html` incrementate e **uguali**
- ☐ `node tools/genera-sbom.js` rieseguito
- ☐ `node tools/genera-pdf.js` rieseguito — altrimenti gli allegati descrivono la versione precedente
- ☐ `node tools/genera-pacchetti.js` rieseguito e chiuso con **0 collegamenti rotti**
- ☐ Rapporto di evidenza rigenerato sul commit che si pubblica
- ☐ Eventuali difformità nuove annotate in `09-Release-Conformance-Statement.md` §4
- ☐ Nessun marcatore di conflitto (`verifica_guida` lo controlla su 58 file)
- ☐ Nessuna chiave API nei file versionati
- ☐ Il debito di accessibilità **non è cresciuto** — se è sceso, rigenerare il

riferimento con `node tools/verifica-accessibilita.js --aggiorna-riferimento` e registrarlo nel commit

- ☐ La copertura di codice **non è scesa** — `audit_copertura` fallisce se lo è

Documento aggiornato alla versione `bsi-v200`.

Specifiche funzionali — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Versione descritta	`bsi-v200`
Scopo	Descrivere cosa fa il prodotto, per chi, con quali regole e con quali limiti.

---

1. Descrizione del prodotto

BioSpecInfo è una piattaforma di **chemioinformatica interattiva** per lo studio universitario di chimica, biochimica, farmacologia e astrochimica.

L'intera logica scientifica — interpretazione delle strutture molecolari, predizione spettrale, calcoli, visualizzazione 2D e 3D — viene eseguita **nel browser dello studente**. Non esiste un server: l'applicazione si installa come app, funziona senza rete e non trasmette dati.

1.1 Obiettivi

Obiettivo	Come viene perseguito
Accessibilità totale	Nessuna registrazione, nessun costo, nessun server. Chi apre il collegamento la sta già usando.
Indipendenza dalla rete	Tutto il necessario è incluso e messo in cache: funziona in aula, in treno, in biblioteca senza campo.
Correttezza verificabile	I dati scientifici sono sottoposti a controlli automatici; i limiti dei modelli sono dichiarati anziché nascosti.
Riservatezza per costruzione	Non esiste un backend che possa raccogliere dati: l'assenza di raccolta non è una promessa, è una conseguenza dell'architettura.

1.2 Cosa il prodotto non è

Non è un dispositivo medico	Non supporta diagnosi, prescrizione né dosaggio (Reg. UE 2017/745)
Non è uno strumento analitico certificato	Gli spettri sono predizioni, non misure
Non è una piattaforma collaborativa	Nessun account, nessuna condivisione, nessuna sincronizzazione fra dispositivi

---

2. Attori

Attore	Descrizione	Cosa può fare
Studente	Utente principale. Studia per un esame universitario.	Tutte le funzioni: consultazione, calcolo, quiz, ripetizione spaziata, assistente AI
Docente	Usa l'app come supporto alla didattica	Le stesse funzioni; caricamento di materiale proprio nel File Manager
Valutatore tecnico	Azienda o commissione che esamina il lavoro	Consultazione della documentazione e riesecuzione dei banchi di verifica

Non esistono ruoli tecnici: non c'è autenticazione, quindi non c'è distinzione di permessi. Il File Manager ha una protezione locale che è un **deterrente**, non un controllo d'accesso (vedi §6).

---

3. Aree funzionali

L'applicazione conta **95 sezioni**, raggruppate per area di studio.

3.1 Spettroscopia

Funzione	Descrizione
Centro spettroscopico	Si inserisce una molecola (SMILES, nome, formula) e si ottengono IR, $^{1}H/^{13}C$ NMR, MS, UV-Vis, Raman, struttura 2D e 3D, gruppi funzionali
Spettri IR visivi	Bande caratteristiche per classe di composti
Spettro di massa	Schemi di frammentazione e regole diagnostiche
UV-Vis	Cromofori, λmax, regole di Woodward
Predizione NMR assegnata per atomo	Si disegna o si scrive una struttura e si ottengono ^{1}H e ^{13}C con l'indicazione di QUALI atomi producono ogni segnale; spettro interrogabile (zoom, intensità, clic sul picco), tabella di assegnazione, cronologia SMILES/InChI, uscite CSV/JCAMP-DX/PNG
NMR bidimensionale	COSY, HSQC editato e HMBC costruiti camminando il grafo; cliccando una macchia si illuminano gli atomi che la producono. Non è una simulazione dell'esperimento, e il pannello lo dichiara
Apri un file qualunque, e il programma decide	Un solo ingresso, senza filtro di estensione: si apre quello che si ha e il programma **dichiara che cosa ha deciso che sia**. Un documento (PDF, fotografia, Word, OpenDocument, PowerPoint, Excel, testo) si apre, se ne vede **tutto** il contenuto (ogni pagina disegnata, tutto il testo), i dati spettroscopici riconosciuti compaiono in tabella **con la loro provenienza**, e lo svolgimento procede passo per passo; un JCAMP-DX o due colonne di numeri diventano uno spettro misurato; un'immagine fa entrambe le cose. Quello che non si è potuto leggere è dichiarato

Funzione	Descrizione
Spettro da una figura (digitalizzazione)	Dalla fotografia di un registratore, dal ritaglio di un articolo o dallo schermo dello strumento si estrae la traccia dai **pixel**: la cornice degli assi viene riconosciuta e digitalizzata solo la parte interna, la posizione della linea è il **baricentro pesato sulla scurezza** (subpixel), la griglia viene aggirata per continuità e scurezza, i due assi si tarano in lineare o in logaritmica, la %T si converte in assorbanza con $A = -\log_{10} T$. Il grafico, i picchi e le assegnazioni compaiono da sé e si rifanno a ogni modifica della taratura; una diagnosi dice che cosa è stato deciso e perché. Uscite CSV e JCAMP-DX. Scarto misurato: medio 2,2 cm^{-1}, peggiore 7,3 su sette figure costruite
Modello 3D dalla struttura disegnata	Coordinate costruite dal grafo, con gli stessi indici del predittore: un picco cliccato illumina l'atomo anche in tre dimensioni
NOE e NMR 2D (tabelle)	Valori di riferimento per le correlazioni spaziali
Tabelle di riferimento	Valori tabulati per la consultazione rapida
Quiz spettri	Riconoscimento del gruppo funzionale da uno spettro

3.2 Chimica generale e fisica

Tavola periodica interattiva · calcolatore chimico · bilanciamento di reazioni · gradi di insaturazione · diagrammi di distribuzione · VSEPR · orbitali molecolari · termodinamica avanzata · elettrochimica (Nernst) · pKa e tamponi · calcolatore di laboratorio · costanti e formule.

3.2-bis Separazione — cromatografia

Sezione **Cromatografia**: cromatogramma calcolato dai parametri (piatti teorici, fattore di ritenzione, selettività, tempo morto) con vista d'insieme e vista espansa della coppia di picchi; curva di **van Deemter** con i tre termini separati e il minimo calcolato in forma chiusa; calcolatore dell'**indice di ritenzione di Kováts**; tabella delle grandezze con gli intervalli d'uso; confronto GC/HPLC; guida ai rivelatori; diagnostica dei difetti di forma dei picchi.

> **Regola applicata.** Il cromatogramma non è un'illustrazione: le gaussiane hanno σ = t_R/√N e la risoluzione mostrata è quella che si misurerebbe su quei picchi. La sezione dichiara esplicitamente ciò che **non** fa — prevedere il tempo di ritenzione di una molecola a partire dalla struttura, che richiederebbe parametri sperimentali della fase stazionaria non ricavabili dal solo SMILES.

3.2-ter Taratura e incertezza

Sezione **Taratura**: regressione ordinaria ai minimi quadrati su dati incollabili da un foglio di calcolo, con pendenza e intercetta **corredate dalla loro deviazione standard**; grafico della retta con banda di confidenza al 95 % e **grafico dei residui**; concentrazione dell'incognito

con intervallo di confidenza completo (le tre sorgenti 1/M, 1/N e distanza dal baricentro sono mostrate separate); **LOD e LOQ** secondo ICH Q2(R1); propagazione dell'incertezza con indicazione di quale contributo domina; regole delle cifre significative.

Due diagnostiche che un r^2 non fornisce:

Diagnostica	Metodo	Perché non basta l'r^2
Non linearità	Test F fra retta e parabola sugli stessi dati	Un r^2 di 0,984 accompagna una curva di saturazione evidente: l'r^2 misura quanta varianza la retta spiega, non se la retta sia il modello giusto
Punto anomalo	Residuo **cancellato**: la retta di giudizio è ricostruita senza il punto giudicato	Il criterio ingenuo `

> **La seconda riga è una trappola incontrata scrivendo la sezione**, non un caso di scuola: il criterio a 3 σ era già scritto, e l'esempio che doveva dimostrarlo non veniva segnalato. Il controllo passava senza vedere nulla.

3.2-quater Chemioinformatica e QSAR

Sezione **Chemioinformatica**: un banco di lavoro che va dall'insieme grezzo al verdetto sul modello, interamente nel browser. Si incollano SMILES — con nome e attività, se ci sono — e si ottiene in sei pannelli ciò che un gruppo di chemioinformatica produce in una giornata.

Pannello	Cosa fa
Descrittori	Standardizzazione (frammento maggiore, SMILES canonico, deduplicazione), 43 descrittori RDKit, regola dei 5 di Lipinski, filtro di Veber, indice QED ricalcolato — MinimalLib non lo espone
Similarità e gruppi	Impronte Morgan/ECFP4, RDKit, pattern e MACCS; Tanimoto e Dice; matrice di similarità; vicini più simili; **raggruppamento di Butina** e selezione **MaxMin** per la diversità
Spazio chimico	PCA sui descrittori standardizzati, con varianza spiegata per componente e i **carichi** che dicono quale descrittore muove quale asse; scheletri di **Bemis–Murcko** con il conteggio delle molecole per scheletro
Modello QSAR	Regressione kernel (kernel di Tanimoto), kNN su Tanimoto o regressione logistica; **divisione per scheletro**; **validazione incrociata raggruppata** a 3, 5 o 10 pieghe; **confronto fra modelli sulle stesse pieghe** con il riferimento banale in cima; iperparametri scelti per **validazione annidata**; **intervalli di predizione conformi**; **curva di apprendimento**; dominio di applicabilità
Ricerca	Ricerca per sottostruttura su tutto l'insieme con `SubstructLibrary` di RDKit, query SMARTS o SMILES, conteggio delle occorrenze, otto query pronte
Tabella SAR	Decomposizione in gruppi R attorno a un nucleo: una riga per molecola, una colonna per posizione di sostituzione, con l'attività mediana di ogni sostituente. Le molecole senza il nucleo sono **escluse**, non mostrate con celle vuote
Coppie corrispondenti	Coppie che differiscono per **una sola** sostituzione, raggruppate per trasformazione con mediana, intervallo e **numero di coppie concordanti**

Pannello	Cosa fa
Salti di attività	Coppie strutturalmente vicine con attività lontane, ordinate per **SALI**: sono le coppie su cui ogni modello sbaglia, ed è onesto mostrarle
Allarmi strutturali	PAINS e Brenk, con il frammento evidenziato e il motivo per cui è segnalato

Tre scelte che distinguono un banco di lavoro da una dimostrazione:

Scelta	Metodo	Perché
Divisione per scheletro	L'insieme di prova contiene **soltanto scheletri mai visti** in addestramento	La divisione casuale mette analoghi stretti da entrambe le parti: il modello riconosce, non predice, e l'R^2 che ne esce non sopravvive alla prima molecola nuova
Modello nullo per rimescolamento	Lo stesso modello viene riaddestrato su **etichette mescolate**, otto volte; il verdetto è il confronto	Un R^2 di 0,4 può nascere dal caso quando le molecole sono poche. Se i sosia casuali arrivano allo stesso punteggio, il modello non ha imparato niente — e il pannello **lo dice**, invece di mostrare solo il numero buono
Deduplicazione sul canonico	`OC(=O)C` e `CC(=O)O` sono la stessa molecola	La stessa molecola in addestramento e in prova è una fuga di informazione, e la stringa scritta non basta a vederla

> **Il verdetto è scritto a parole, non lasciato al lettore.** Il pannello QSAR non si ferma alle metriche: dichiara se il modello ha battuto tutti i suoi sosia casuali e di quanto. Misurato sull'esempio dei 28 inibitori: R^2 0,861 contro un massimo di 0,574 fra gli otto sosia — margine 0,287. Sulle stesse molecole con etichette casuali il modello **non** batte i sosia, e il pannello lo dichiara allo stesso modo.

I risultati si portano via

Due esportazioni, entrambe formate in memoria dal browser: non c'è nessuna richiesta di rete, ed è la ragione per cui la nota accanto ai pulsanti può dire che nessun dato esce dal dispositivo senza chiedere di crederci sulla parola.

Esportazione	Contenuto
Tabella CSV	Una riga per molecola: nome, SMILES canonico e originale, attività, scheletro, nove descrittori, QED, violazioni di Lipinski, esito di Veber, allarmi trovati. Con la validazione incrociata attiva si aggiungono **previsto** e **residuo** — e solo allora, perché la validazione incrociata è l'unica che dia a ogni molecola una predizione da un modello che non l'aveva vista
SDF	Le strutture con **coordinate 2D generate**, il nome nella prima riga di ogni blocco e i campi dati attaccati: è il formato con cui si scambiano insiemi fra gruppi e programmi. Verificato col round-trip
Rapporto di metodo (.md)	Ciò che serve a **rifare** la stessa analisi: versione di RDKit, fingerprint, tipo di divisione, numero di molecole e scheletri, **il seme**, le metriche, il verdetto del controllo nullo, le pieghe una per una, e i limiti dichiarati

> **Perché il seme è nel rapporto.** Senza di esso «divisione casuale» e «etichette rimescolate» non significano niente: due esecuzioni darebbero numeri diversi e nessuno potrebbe dire quale sia quello giusto. Il generatore è un mulberry32 seminato, e lo stesso seme dà le stesse pieghe.

Una misura che nessun altro banco poteva fare

`verifica-accessibilita` percorre le 95 sezioni e misura ciò che è **visibile**. I sei pannelli di questa sezione stanno dentro un contenitore che resta `display:none` finché l'analisi non è stata eseguita: quel banco non li ha mai visti, e il suo «contrasto 0» non parlava di loro.

`test_cheminfo` esegue l'analisi per davvero, apre i sei pannelli a turno e misura il contrasto WCAG su ogni testo che vi compare — **460 elementi, 0 difetti**. Le formule sono riscritte dentro il banco e non prese dall'applicazione: un banco che chiedesse al codice sotto esame quanto vale il proprio contrasto non misurerebbe niente.

Il motore sta in `bsi-cheminfo.js`, è esposto come `window.BSIChem` ed è verificato dal banco `test_cheminfo` (189 controlli) contro valori calcolabili a mano.

3.3 Chimica organica

Retrosintesi (98 esercizi) · sintesi e meccanismi con frecce elettroniche · meccanismi animati · editor di strutture · modulo *Chimica Organica 1+2* con 25 moduli.

3.4 Biochimica

Amminoacidi · biomolecole · vie metaboliche animate · cinetica enzimatica (Michaelis-Menten) · macromolecole 3D · strutture biologiche interattive · guida di biochimica · biochimica d'esame.

La **guida di biochimica** è un documento a sé (`Biochimica_Guida_Definitiva.html`, 8,2 MB) caricato in un riquadro dalla sezione: 12 capitoli più l'indice, 78 sottosezioni, 98 figure incorporate, e due capitoli dedicati all'esame — «Guida all'orale» e «Domande d'esame reali e risposte corrette». Non contiene **nessuno script e nessuna richiesta di rete**: non può generare errori JavaScript, e il testo alternativo delle 98 figure viene dalle didascalie scritte dall'autore, non da descrizioni inventate (vedi `docs/06` §3-ter).

Chiude il documento una sezione di **autoverifica con 12 domande a risposta multipla**, recuperata dalla guida precedente — dove era un quiz in JavaScript — e riscritta **senza un rigo di codice**: le quattro opzioni sono in chiaro, la soluzione sta in un `<details>` nativo che si apre con un clic. In stampa il `<details>` sparisce e compare una copia dedicata della soluzione, così la copia cartacea non perde le spiegazioni.

3.5 Farmacologia

Funzione	Descrizione
Atlante 3D	Si sceglie zona del corpo, malattia e farmaco: il modello evidenzia l'organo bersaglio

Funzione	Descrizione
Banca dati	**263 farmaci** con struttura, peso molecolare, meccanismo, indicazioni, effetti avversi, classe. Per **36** la voce porta l'identificativo ChEMBL e la formula del record, e la formula viene ricostruita dal grafo e confrontata (§5.1)
Interazioni	Verifica delle interazioni fra principi attivi
Farmacocinetica	Calcolatore di emivita, clearance, volume di distribuzione
Casi clinici	Percorsi diagnosi-terapia a scopo didattico

3.6 Astrochimica

Molecole del mezzo interstellare · esopianeti e dati JWST · nebulose · spettri stellari · nucleosintesi · chimica cometaria · quiz.

3.6-bis Lettore di spettri

Si carica un file dal proprio strumento, o si incolla un testo, e viene letto.

Funzione	Descrizione
Formati	**JCAMP-DX** (`.jdx` , `.dx`) con compressione **ASDF** completa — PMAI per le cifre, DIF per le differenze, DUP per le ripetizioni — oppure due colonne di numeri separate da spazi, virgole o tabulazioni
Controllo di integrità	Il formato JCAMP prevede che, dopo una serie di differenze, il valore scritto per intero coincida con quello calcolato. Qui viene verificato
Ricerca dei picchi	Per **prominenza**, con il rumore stimato dalla deviazione assoluta mediana (MAD × 1,4826) e non dalla deviazione standard, che un picco alto gonfia
Assegnazione IR	Per ogni banda vengono elencate **tutte** le assegnazioni compatibili fra 24 intervalli, non la prima
Spettri di massa	La lista di picchi viene riconosciuta come tale; le differenze fra ioni sono confrontate con 17 **perdite neutre**
NMR	Integrazione delle aree per regione di spostamento chimico
Disegno	L'IR con i numeri d'onda **decrescenti**, come lo si guarda

Quello che non fa, ed è scritto nella sezione: non deduce la struttura. Un insieme di bande è compatibile con molte molecole, e un programma che da tre picchi producesse un nome farebbe un'affermazione che i dati non reggono.

3.6-ter Lingue e linguaggi chimici

Funzione	Descrizione
Lingua dell'interfaccia	**14 lingue** con selettore e campo di ricerca: italiano, inglese, spagnolo, francese, tedesco, portoghese, olandese, polacco, romeno, greco, russo, cinese, giapponese, arabo. La copertura di ciascuna è **misurata**, non dichiarata
Verso di scrittura	L'arabo porta con sé `dir="rtl"` ; tutte le altre `ltr` , anche al ritorno

Funzione	Descrizione
Linguaggi chimici	**25 uscite** da uno SMILES: formula e composizione in massa, SMILES canonico/piano/con idrogeni, CXSMILES, InChI e chiave InChI, scheletro della chiave, scaffold di Murcko, tag stereochimici CIP, SMARTS e CXSMARTS, molfile V2000 (anche aromatico) e V3000, XYZ, PDB, CML, JSON, e le **sei impronte** (Morgan, MACCS, RDKit, coppie di atomi, torsioni topologiche, pattern)
Nomenclatura IUPAC	Su una **classe dichiarata** (aciclici, neutri, C H O N F Cl Br I, un solo tipo di gruppo principale, sostituenti lineari). Fuori da quella classe **rifiuta** invece di tentare

3.7 Strumenti di studio

Funzione	Descrizione
Ripetizione spaziata	Algoritmo SM-2 su schede generate dai contenuti
Quiz	Per materia, con punteggio e storico
Percorso di studio	Sequenza guidata di argomenti
Note	Globali e per sezione
Pomodoro	Timer di studio
File Manager	Archivio personale di foto, documenti e pagine HTML

3.8 Assistente «Spectra»

Agente AI con **35 strumenti** che può interrogare l'applicazione stessa: calcolare descrittori, generare spettri, cercare costanti, bilanciare reazioni, aprire sezioni. Supporta dieci fornitori, e resta utilizzabile quando uno di essi si guasta (§5.2).

---

4. Flussi principali

4.1 Analizzare una molecola

```
1. Si apre il Centro spettroscopico
2. Si inserisce la molecola:
     · SMILES             → uso diretto
     · nome o formula     → risoluzione via PubChem (richiede rete)
     · disegno nell'editor → SMILES generato
3. L'applicazione mostra: struttura 2D, gruppi funzionali, descrittori,
   spettri previsti (IR, NMR, MS, UV-Vis), drug-likeness
4. La struttura 3D si carica se c'è rete (conformero da PubChem)
```

Senza rete: i punti 3 restano completi per gli SMILES inseriti direttamente; la risoluzione per nome e la struttura 3D non sono disponibili, e l'applicazione lo dichiara invece di restare in attesa.

4.2 Chiedere all'assistente

```
1. Si tocca il pulsante «Spectra»
2. Al primo uso: si sceglie un fornitore e si inserisce una chiave API,
   oppure si collega un proxy (e allora nessuna chiave serve)
3. Si scrive la domanda
4. L'assistente risponde, usando i propri strumenti quando servono:
   ogni strumento usato è mostrato, non nascosto
```

Se il fornitore non risponde: l'assistente lo annota, lo retrocede nella scelta per 24 ore e passa a un altro fornitore configurato. Il pulsante 🔌 **Prova** misura in pochi secondi quali fornitori il dispositivo raggiunge davvero, **senza bisogno di alcuna chiave**.

4.3 Studiare con la ripetizione spaziata

```
1. Si apre la sezione di studio
2. Si genera un mazzo dai contenuti
3. Per ogni scheda si valuta quanto la si ricordava
4. L'algoritmo SM-2 calcola quando riproporla
5. Alla riapertura compaiono solo le schede in scadenza
```

4.4 Archiviare materiale proprio

```
1. File Manager → password locale
2. Caricamento di foto, documenti o pagine HTML
3. Il materiale resta NEL DISPOSITIVO (IndexedDB), non viene trasmesso
```

---

5. Regole di business

5.1 Dati scientifici

Regola	Motivazione
Una struttura molecolare entra in banca dati solo se il peso ricalcolato coincide con quello di letteratura (±0,6 u)	Un dato plausibile ma sbagliato è indistinguibile da uno corretto per l'utente
Se una struttura non supera la verifica, la voce resta **senza struttura**	Un dato sbagliato sostituito da un altro sbagliato non è un progresso
Ogni assenza deve essere **registrata con il motivo**	Un'eccezione non motivata diventa una scorciatoia
Gli spettri sono deterministici	Due spettri della stessa molecola devono essere confrontabili
Una costante fisica ambigua non viene risolta a caso	Un valore sbagliato dato con sicurezza è peggio di una domanda di chiarimento

5.2 Assistente AI

Regola	Comportamento
Un fornitore che non risponde viene ricordato **24 ore**	Poi si ritenta: un guasto di oggi non è una condanna
Un modello ritirato viene appreso dal messaggio d'errore	Le liste codificate a mano invecchiano
Un'attesa imposta superiore a **10 secondi** fa cambiare fornitore, se ce n'è uno pronto	Oltre quella soglia l'utente crede che l'app sia bloccata
Un turno alla volta	Due invii sovrapposti corrompono la cronologia
I fornitori a pagamento non vengono proposti come gratuiti	Senza fatturazione attiva si sbatte sul limite alla prima domanda

5.3 Dati dell'utente

Regola	Comportamento
Massimo 30 conversazioni, 100 messaggi ciascuna	Oltre, `localStorage` si riempie e **ogni** salvataggio successivo fallisce
Immagini fino a 20 MB	Oltre, rifiuto esplicito; le altre proseguono
Cancellazione selettiva per gruppi	Azzerare tutto per liberare spazio è una perdita evitabile
Una scrittura fallita **deve** essere dichiarata	«✅ Salvato» quando non è vero è peggio di un errore

---

6. Limitazioni attuali

Dichiarate, non nascoste. Il dettaglio è in `09-Release-Conformance-Statement.md` §4.

#	Limitazione	Conseguenza pratica
1	**2 voci senza struttura da mostrare** (erano 6)	Ivermectina è una miscela di omologhi B1a/B1b e Coartem un'associazione di due principi attivi: una sola notazione SMILES non le rappresenta, e per queste due mancano 2D/3D e spettri previsti. Digossina, vincristina e tacrolimus (topico e sistemico) hanno ora la struttura, ripresa da ChEMBL e verificata contro il peso molecolare
2	**Spettri predetti, non misurati**	Utili per riconoscere i gruppi funzionali; non per identificazione formale
3	**Nessuna sincronizzazione**	I dati non passano da un dispositivo all'altro, e non esistono copie sul server
4	**File Manager: deterrente, non sicurezza**	Su un sito statico chi legge il sorgente aggira qualunque controllo lato pagina. Nel sorgente c'è solo l'impronta SHA-256 della password, mai la password
5	**Funzioni che richiedono rete**	Nome IUPAC, CAS, GHS, conformeri 3D, assistente AI

#	Limitazione	Conseguenza pratica
6	**Accessibilità automatizzata, non completa**	Contrasto, nomi accessibili, etichette, gerarchia dei titoli su 13 pagine e tutte le 95 sezioni: **0 difetti**, compreso il testo su gradiente (valutato sulla tappa peggiore). Restano fuori il testo dentro gli SVG e quello su una vera immagine di sfondo, contati a ogni esecuzione, e tutto ciò che richiede giudizio umano
7	**Cromatografia: nessuna previsione di ritenzione**	La sezione calcola risoluzione, efficienza e indici **dati** k, α e N; non prevede k da una struttura, che richiederebbe parametri sperimentali della fase stazionaria
8	**Copertura di codice parziale**	Misurata: **49,89 %** di istruzioni sul percorso più ampio che un banco compie. Non è la copertura dell'intera batteria, ed è di istruzioni, non di rami; vedi `08-Traceability-Matrix.md` §8
9	**Verifica su Chromium soltanto**	Firefox e WebKit sono provati a mano, non da banco

---

7. Sviluppi possibili

Non impegni: direzioni coerenti con l'architettura.

Area	Proposta	Perché
Dati	Le 2 voci residue non hanno una struttura da mostrare	Non è una lacuna da chiudere: ivermectina è una miscela, Coartem un'associazione. Semmai si potrebbero mostrare i **componenti**, dichiarandoli tali
Spettri	Confronto con database sperimentali (SDBS/NIST) dentro l'app	Renderebbe visibile la distanza fra predizione e misura
Accessibilità	Affiancare `axe-core` per le regole che il banco non implementa	Ruoli ARIA, ordine di tabulazione e gestione del fuoco restano fuori dalla misura attuale
Esportazione	Esportazione completa dei dati utente	Unico rimedio possibile all'assenza di copie di sicurezza
Assistente	Proxy preconfigurato per chi non vuole pubblicarne uno	Toglierebbe l'ultima barriera d'accesso all'AI

Documento aggiornato alla versione `bsi-v200`.

Guida per chi la usa — BioSpecInfo

Questa guida è scritta per chi studia, non per chi programma. Non serve sapere nulla di informatica.

Che cos'è

BioSpecInfo è un'applicazione per studiare chimica, biochimica, farmacologia e astrochimica. Si apre come un sito, ma si può **installare** sul telefono o sul computer e funziona anche **senza internet**.

Tre cose da sapere subito:

- **Non serve registrarsi.** Nessun account, nessuna email, nessuna password.
- **Non costa nulla.**
- **I tuoi dati restano tuoi.** Appunti, file e conversazioni non escono dal tuo

dispositivo: non c'è nessun server a cui possano arrivare.

👉 **samupropio1-ship-it.github.io/BioSpecInfo-v11**

Installarla (facoltativo, ma conviene)

Installandola ottieni un'icona sullo schermo, l'apertura a schermo intero e il funzionamento offline.

Dispositivo	Come si fa
Android / Chrome	Menu ⋮ → *Installa app* (o *Aggiungi a schermata Home*)
iPhone / iPad	Pulsante Condividi → *Aggiungi a Home*
Computer	Icona di installazione nella barra degli indirizzi, a destra

Dopo l'installazione, apri l'app una volta con la rete attiva: scaricherà tutto quello che le serve. Da quel momento funziona anche senza campo.

Muoversi nell'app

Elemento	Dove si trova	A cosa serve
🔍 **Cerca**	In alto	Cerca in tutta l'app. Scorciatoia: `Ctrl+K`
Menu delle sezioni	Barra di navigazione	Le 95 sezioni, raggruppate per materia
✨	In basso a destra	Strumenti, impostazioni e aggiornamenti
Spectra	In basso a destra	L'assistente che risponde alle domande

---

Le cose che farai più spesso

Analizzare una molecola

1. Apri **Centro spettroscopico**
2. Scrivi la molecola nella casella. Puoi usare:

- il **nome** — `aspirina`, `caffeina`
- la **formula SMILES** — `CC(=O)Oc1ccccc1C(=O)O`
- oppure **disegnarla** con l'editor

1. Premi Invio

Ottieni la struttura in 2D e 3D, i gruppi funzionali, il peso molecolare, gli spettri previsti (IR, NMR, massa, UV-Vis) e le proprietà farmacologiche.

> **Se sei senza rete:** la ricerca per *nome* non funziona, perché deve consultare una banca dati esterna. La formula SMILES invece funziona sempre.

Leggere uno spettro IR

Lo spettro è disegnato come su uno strumento vero: la scala in basso va da 4000 a 400 cm^{-1} (**da sinistra a destra i numeri calano**) e le bande puntano verso il **basso**.

La zona colorata sotto i 1500 cm^{-1} è l'**impronta digitale**: è lì che si riconosce la molecola specifica, non solo la sua classe. Sotto il grafico trovi l'elenco delle bande con la loro assegnazione.

> Gli spettri sono **previsti** dalla struttura, non misurati in laboratorio. Servono a riconoscere i gruppi funzionali e a capire il legame fra struttura e spettro. Per un dato sperimentale, l'app ti rimanda ai database ufficiali.

Tutto sugli spettri, in un posto solo

Le cose che si fanno con uno spettro stavano in **nove sezioni** sparse su due menù diversi. Ora c'è 🔬 **Centro spettri**, una sezione sola con dieci schede:

scheda	a che serve
Leggi il tuo spettro	un file, una fotografia, un PDF
Prevedi NMR	^{1}H e ^{13}C assegnati per atomo
NMR 2D e NOE	COSY, HSQC, HMBC
Massa	frammentazione e perdite neutre
IR	bande e gruppi funzionali
UV-Vis	cromofori e λmax
Tabelle di riferimento	^{1}H, ^{13}C, IR, costanti J

scheda	a che serve
Tutto su una molecola	editor, spettri, gruppi, confronto
Elucida una struttura	dai dati a una proposta
Allenati	riconosci il gruppo dallo spettro

I pannelli sono gli stessi di prima: sono stati **spostati**, non copiati — non esiste una seconda copia di niente. I vecchi collegamenti continuano a funzionare: se qualcosa nell'applicazione ti mandava a «Spettro MS», ora ti porta al Centro spettri aperto sulla scheda **Massa**.

> **Due tabelle delle costanti J, con numeri diversi.** Ce n'erano due: una dava il ^3J orto aromatico a 6–9 Hz, l'altra a 7–10. Chi cercava quel numero otteneva una risposta diversa a seconda di dove entrava. Ne resta **una**, quella dentro il Centro spettri, che sta nella sezione con la sua lista di riferimenti.

Dove si trovano le cose

Le sezioni stanno nella **barra di navigazione**, raggruppate per materia, con 🛠️ **Utility** fra loro: laboratorio, GHS, mappa concettuale, confronto molecole, ricerca molecole, atlante, risorse, materiali, note, file manager, statistiche, data science, pomodoro, calcolatrice di laboratorio, **articoli**, e le due della lingua.

Il menù ✨ tiene soltanto quello che *non* è una sezione: gli editor, i laboratori in sovrapposizione, e le azioni come esportare una figura, stampare, condividere, aggiornare. La regola è semplice e vale sempre: **se una cosa è nella barra, non è anche nel menù**. Prima non era così, e nel menù comparivano due «Data Science», due «File Manager», due «Articoli» e due «Aggiornamenti».

Due eccezioni, e hanno un motivo:

- **Spectra** non è nella barra: ha il suo pulsante dedicato, quello che pulsa

in basso. Tre strade per il copilota erano troppe.

- Il **lettore spettri** non è nella barra: è una scheda del Centro spettri.

> **Due File Manager, e sono due cose diverse.** Nella barra, 📁 **File Manager** serve a caricare e ordinare i tuoi appunti (PDF, immagini, Word). Nel menù ✨, 🔒 **File Manager protetto** è una pagina a parte con la password, per la galleria e le sezioni personalizzate. Avevano lo stesso nome e sembravano un doppione: ora si chiamano per quello che fanno.

Gli articoli e i Nobel

📰 **Articoli** (dal menù ✨) ha due schede.

Ultimi articoli mostra quello che è uscito negli ultimi due mesi, per argomento: spettroscopia, sintesi, catalisi, farmaci, materiali, biochimica, computazionale. Non è un elenco scritto a mano — sarebbe vecchio domani: arriva da **Crossref**, la banca dati dei DOI,

nel momento in cui apri la scheda. Ogni titolo porta al suo **DOI**, che è un indirizzo permanente, e accanto c'è il collegamento per **leggerlo in italiano**.

Nobel per la chimica elenca tutti i premi. Anni, premiati e motivazioni vengono dalla **Fondazione Nobel**, in diretta: non sono scritti dentro l'applicazione. La **spiegazione** in italiano invece sì — ed è abbinata all'*argomento* della motivazione ufficiale, non all'anno, così nessuna attribuzione viene da qui. Per ogni premio trovi la pagina ufficiale, il materiale scientifico e quello divulgativo della Fondazione, la versione tradotta in italiano e una ricerca su Wikipedia italiana — tutti indirizzi **costruiti** dall'anno, non scritti.

> **Se un premio non ha la spiegazione, te lo dico.** Preferisco scrivere «per questo premio non ho una spiegazione» e lasciarti la motivazione ufficiale e il materiale della Fondazione, che è la fonte, piuttosto che inventare.

> **Serve la rete.** Senza, non vedi una lista vecchia spacciata per nuova: vedi che cosa ho chiesto, a chi, e perché non è arrivato — con l'indirizzo della richiesta, che puoi aprire a mano.

Leggere uno spettro che hai misurato tu

La sezione **Lettore spettri** fa il contrario di quella sopra: non prevede uno spettro dalla struttura, legge quello che hai già.

Ci sono **due bottoni**, e non sono un doppione.

`📂 Apri un file` non filtra niente: qualunque cosa hai in cartella, aprila. `📷 Foto o immagine di uno spettro` apre invece la **galleria del telefono** e ti lascia anche scattare la foto sul momento — su un telefono sono due selettori diversi del sistema, non due filtri dello stesso, e senza il secondo non c'è modo di dare una fotografia.

Sotto i bottoni compare una riga che **ti dice che cosa ha deciso di essere** il file che hai aperto:

Quello che apri	Che cosa succede
un **PDF**, un **Word**, un OpenDocument, un foglio di calcolo	si apre, te lo mostra per intero e **svolge** quello che contiene
una **fotografia** (`.png` , `.jpg` , …)	tutte e due le cose: ne estrae la **traccia** e la mostra anche come pagina, perché può essere la foto di uno spettro o di un foglio
un **JCAMP-DX** (`.jdx` , `.dx`), anche compresso	è uno spettro: lo dichiara il formato stesso
un **testo** o un `.csv`	si decide **dal contenuto**: se ci sono dei numeri che formano una curva è uno spettro, altrimenti è il **testo di un quesito** e lo svolge

Se preferisci, puoi **incollare** i dati o il testo del quesito nella casella e premere **Leggi e svolgi**: fa lo stesso ragionamento.

> Prima c'erano due schede, e quella degli spettri lasciava scegliere solo `.jdx` e `.csv`. Su un telefono quel filtro non restringeva, **nascondeva**: chi ci arrivava con il PDF di un compito vedeva una cartella vuota. Ora la porta è una e non filtra niente.

Se non hai un file sotto mano, i tre pulsanti **esempio IR**, **esempio MS** ed **esempio NMR** caricano uno spettro di prova, e **esempio: un quesito d'esame** carica una traccia completa.

Dentro la sezione c'è 📖 **Come si usa — guida completa**: sette capitoletti che spiegano passo per passo i due bottoni, la taratura, che cosa fa il programma da solo, che cosa non può fare, come riconoscere un risultato storto dal sintomo che vedi, e come portare via i numeri.

Uno spettro che hai solo come figura

La traccia viene estratta dai **pixel**, e il grafico compare subito. Poi va **tarata**, perché i numeri degli assi non stanno nell'immagine:

campo	che cosa ci scrivi
x a sinistra / x a destra	i valori ai due estremi del grafico — per un IR tipicamente 4000 e 400 **in quest'ordine**, perché l'IR si scrive con i numeri d'onda che decrescono
y in basso / y in alto	i valori dell'asse verticale; per una figura in %T, 0 in basso e 100 in alto
unità y	`%T`, `T`, `A` o intensità. Scegliendo `%T` l'assorbanza si ricava con **$A = -\log_{10} T$**, che è la definizione
x/y logaritmica	se un asse è logaritmico, dillo: applicargli una mappa lineare non dà un errore, dà numeri sbagliati dall'aspetto giusto
cornice	normalmente la trovo io; «tutta la figura» serve quando la figura è già ritagliata al solo grafico

Cambi un campo e il grafico si rifà da sé: non c'è niente da confermare. Sotto compare la **diagnosi** — quante colonne, se la cornice è stata trovata e su quali lati, sfondo e soglia, quante colonne erano griglia — perché un numero senza il suo come non si può contestare.

Che cosa faccio da solo: trovo la **cornice degli assi** e digitalizzo solo quello che c'è dentro (i numeri scritti sugli assi sono pixel scuri come la traccia); **sopravvivo alla griglia**, scegliendo in ogni colonna il tratto vicino a dove la traccia stava un pixel prima e abbastanza scuro da essere traccia; leggo a **subpixel**, perché il centro di una linea spessa due pixel sta in mezzo.

> **Quanto sbaglio.** Misurato su sette figure costruite di cui si conoscono i centri delle bande: scarto **medio 2,2 cm^{-1}**, **peggiore 7,3**. Una colonna di una figura larga 900 pixel su 3600 cm^{-1} vale circa 4 cm^{-1}: sotto quel valore non c'è niente da guadagnare, è il limite della figura.

> **Se la taratura è sbagliata**, i picchi escono a numeri sbagliati pur stando nel punto giusto della figura: la forma si recupera dai pixel, la taratura no. E una foto presa di sbieco porta una deformazione che non raddrizzo — inquadra la figura di fronte.

Con **scarica CSV** e **scarica JCAMP-DX** i numeri escono da qui: il JCAMP lo apre qualunque programma di spettroscopia, e questo stesso lettore lo rilegge identico. Dentro c'è scritto che la taratura è stata data a mano.

Sotto il grafico trovi i picchi trovati con il loro rapporto segnale-rumore, e per ogni banda **tutte** le assegnazioni compatibili. Se a 1715 cm^{-1} leggi due righe, non è un difetto: a quel numero d'onda un chetone e un acido carbossilico sono entrambi possibili, e dirne uno solo sarebbe inventare.

Aprire un quesito e fartelo svolgere

Lo stesso bottone, `📂 Apri un file`. Accetta il PDF della prova, la fotografia del foglio, un file Word, un testo — e ti mostra **tutto** quello che c'è dentro: ogni pagina disegnata (cliccala per ingrandirla) e il testo per intero.

Poi, se nel testo ci sono dati spettroscopici, li riconosce e li mette in tabella **con scritto accanto da dove li ha presi**. Quella tabella sta sopra lo svolgimento apposta: guardala prima. Se ha scambiato una massa per una banda IR, lo svolgimento che segue sarà impeccabile e la risposta sbagliata, e te ne accorgi lì.

Lo svolgimento elenca i passaggi con la **prova** di ognuno e quanto è certo: i gradi di insaturazione sono aritmetica, il numero di carboni dal picco M+1 è una stima con un margine, una banda IR è *compatibile con* più cose.

> **Non ti dà la struttura.** Un insieme di dati spettroscopici è compatibile con più di una molecola, e un programma che ne sputasse fuori una sola darebbe una certezza che i dati non contengono. Quello che puoi fare è il contrario: scrivi tu una struttura e il programma la **confronta**, segnale per segnale, dicendo dove casca.

> **Se il documento è una scansione**, il testo non si legge: una pagina scansionata contiene pixel, non lettere, e non c'è riconoscimento ottico dei caratteri. L'app te lo dice invece di far finta che il foglio fosse vuoto. Le pagine restano visibili, e se una è la fotografia di uno spettro la traccia si estrae con **Apri un'immagine**.

> Il lettore **non ti dice che molecola è**. Un insieme di bande è compatibile con molte molecole diverse: il passo dallo spettro alla struttura lo fai tu, con il resto di quello che sai sul campione.

Scegliere la lingua

Nella sezione **Lingue** ci sono **quattordici** lingue, con un campo di ricerca per trovarle in fretta: italiano, inglese, spagnolo, francese, tedesco, portoghese, olandese, polacco, romeno, greco, russo, cinese, giapponese e arabo. Scegliendo l'arabo cambia anche il **verso** della pagina, da destra a sinistra.

Accanto c'è **Linguaggi**, che è un'altra cosa: converte una molecola fra le venticinque notazioni con cui la chimica la scrive — SMILES, InChI, SMARTS, molfile, XYZ, PDB, CML, le impronte digitali — e, per una classe dichiarata di molecole semplici, ne scrive il **nome IUPAC**. Fuori da quella classe lo dice, invece di tentare un nome sbagliato.

Studiare con le schede

1. Apri la sezione di studio
2. Genera un mazzo di schede
3. Per ogni scheda, dopo aver risposto, dichiara quanto la ricordavi

L'app calcola **quando** riproportela: le cose che sai bene tornano di rado, quelle incerte tornano presto. Riaprendo trovi solo le schede in scadenza.

Consultare un farmaco

Apri **Farmacologia**. Per ognuno dei 263 farmaci trovi struttura, peso molecolare, meccanismo d'azione, indicazioni, effetti avversi e classe.

> ⚕ **Solo a scopo didattico.** Queste informazioni non sostituiscono il parere di un medico o di un farmacista, e non servono per decidere una terapia.

Spectra, l'assistente

Spectra risponde alle domande di chimica e può usare gli strumenti dell'app: calcolare, generare spettri, cercare costanti, bilanciare reazioni.

Farlo funzionare

Spectra si appoggia a un servizio di intelligenza artificiale esterno, e ti serve una **chiave** gratuita. Due strade:

La più semplice — una chiave tua

1. Vai su console.groq.com e crea una chiave

(gratuita, bastano due minuti)

1. Aprila Spectra, incolla la chiave nel riquadro 🔑
2. Fatto

La definitiva — un proxy

Se ti capita spesso che i servizi non rispondano, o non vuoi tenere la chiave sul telefono, puoi pubblicare un *proxy*: un programmino gratuito che tiene la chiave al sicuro e fa funzionare anche i servizi che dal browser sono bloccati.

Nel riquadro **Proxy** dell'assistente trovi **«Come si attiva, in 4 passi»**, con i comandi già pronti da copiare.

Quando Spectra non risponde

Quasi sempre non è colpa dell'app, ma del servizio esterno. Premi 🔌 **Prova** nel pannello di Spectra: in pochi secondi ti dice **quali servizi il tuo dispositivo riesce davvero a contattare** — e non serve nessuna chiave per saperlo.

Se un servizio smette di rispondere, Spectra se lo ricorda per 24 ore e lo sposta in fondo all'elenco, sotto la voce *«Non hanno risposto da questo dispositivo»*. Non lo cancella: domani potrebbe tornare a funzionare.

Il File Manager

Un archivio personale per foto, documenti e pagine HTML, protetto da una password.

> ⚠️ **Una cosa che devi sapere.** Questa protezione è un **deterrente**, non una vera sicurezza: chi sa dove guardare può aggirarla. Non metterci documenti riservati.

I file restano sul tuo dispositivo. **Non esiste una copia da nessun'altra parte:** se cancelli i dati del browser o cambi telefono, spariscono. Usa l'esportazione per le cose a cui tieni.

Domande frequenti

Devo pagare qualcosa? No. L'app è gratuita. Se usi Spectra, la chiave gratuita del servizio AI ha dei limiti giornalieri, ma non si paga nulla.

Funziona davvero senza internet? Sì, dopo la prima apertura. Restano fuori solo le cose che per loro natura devono chiedere a qualcun altro: la ricerca di molecole per nome, le strutture 3D scaricate al momento, e Spectra.

I miei appunti vengono salvati da qualche parte? Solo sul tuo dispositivo. Non c'è nessun server: nessuno può leggerli, ma nemmeno recuperarli se li perdi.

Come cancello i miei dati? Menu ✨ → Impostazioni → cancellazione dati. Puoi scegliere **cosa**: solo le chiavi, solo le conversazioni, solo lo studio, oppure tutto.

Ho aggiornato ma non vedo niente di nuovo. Menu ✨ → **Aggiornamenti** → *Controlla*. Lì vedi anche quale versione stai usando.

Posso fidarmi degli spettri per una relazione? Per riconoscere i gruppi funzionali e capire la relazione struttura-spettro, sì. Per un dato da citare come sperimentale, no: sono previsioni. L'app ti rimanda ai database ufficiali (NIST, SDBS) con collegamenti diretti.

Ho trovato un errore in un dato chimico. Segnalalo: apri una *issue* sul repository GitHub. I dati sono sottoposti a controlli automatici, ma nessun controllo trova tutto.

Se qualcosa non va

Problema	Cosa provare
L'app non si apre	Ricaricala. Se persiste, menu ✨ → Aggiornamenti → Controlla
Non vedo le novità	La versione è nel menu ✨ → Aggiornamenti
Spectra non risponde	Pulsante 🔌 **Prova**: ti dice quali servizi raggiungi
«Spazio esaurito»	Menu ✨ → Impostazioni → cancella le conversazioni vecchie
Una sezione resta vuota	Controlla la rete: alcune sezioni caricano dati da internet
I grafici sono sfocati	Ricarica la pagina; i grafici si ridisegnano alla risoluzione dello schermo

---

Contatti

Autore: Samuele Pio Provenzano **Repository e segnalazioni:** github.com/samupropio1-ship-it/BioSpecInfo-v11

Guida aggiornata alla versione `bsi-v200`.

Documentazione dei test — BioSpecInfo

Campo	Valore
Software	BioSpecInfo
Versione descritta	`bsi-v200`
Scopo	Descrivere come sono organizzati i test, come eseguirli, che cosa coprono e dove restano scoperti.

---

1. Tipologie presenti

Tipo	Presente	Note
End-to-end (E2E)	✅	Playwright su Chromium headless contro un server HTTP locale. È la forma prevalente.
Verifica dei dati	✅	Confronto dei dati scientifici con una fonte indipendente
Prove di stabilità	✅	Sessioni lunghe, memoria esaurita, rete degradata
Coerenza documentazione/codice	✅	Verifica che ciò che la documentazione promette esista davvero
Unit test isolati	❌	Vedi §6
Copertura di codice strumentata	✅	`audit_copertura`, con il profilatore di Chromium: nessuna build, nessun sorgente riscritto. Vedi §6

Perché E2E e non unit test

L'applicazione è composta da script classici che operano direttamente sul DOM, senza moduli né iniezione delle dipendenze: non esistono unità isolabili senza una riscrittura profonda. I difetti che si sono effettivamente verificati — un canvas sfocato, un pulsante irraggiungibile perché il contenitore è nascosto, un fornitore AI che smette di rispondere — **non sarebbero comunque emersi** da un test unitario: vivono nell'integrazione fra codice, DOM e browser.

La scelta è quindi motivata, non subita. Ne restano i limiti, dichiarati al §6.

---

2. Come eseguire i test

Requisiti

```
npm install                      # solo playwright-core
python3 -m http.server 8899 &    # RDKit WASM richiede contesto HTTP
```

> Senza server i test falliscono tutti: aprire i file con `file://` impedisce il caricamento dei moduli WebAssembly.

Tutta la batteria, con rapporto

```
node tools/genera-evidenza.js
```

Esegue ogni banco, ne trascrive l'uscita integrale e produce `docs/evidence/RAPPORTO-VERIFICA.md` con ambiente, commit e impronte SHA-256. Esce con codice `0` solo se nessun banco fallisce.

```
node tools/genera-evidenza.js --veloce   # salta i banchi con browser
```

L'opzione `--veloce` è utile per un controllo rapido ma **non è sufficiente per un'attestazione**: i banchi saltati sono proprio quelli che misurano il comportamento reale.

Un singolo banco

```
node tools/verifica-farmaci.js           # strumento a sé stante
node tools/verifica-farmaci.js --json    # uscita leggibile da programma

cd tools/banchi && node test_spettri.js  # un banco E2E
```

I banchi E2E stanno in **`tools/banchi/`**, versionati come il resto del repository; la variabile `BSI_BANCHI` permette di indicare un'altra cartella.

> **Non è sempre stato così, ed è bene dirlo.** Fino alla versione `bsi-v168` la cartella predefinita era l'area di lavoro della sessione in cui i banchi erano stati scritti: **32 dei 39 banchi non erano nel repository**. Chi clonava e lanciava il comando documentato ne trovava sette, e non riceveva alcun avviso — perché un banco assente veniva contato ma non faceva fallire la batteria, che chiudeva con «CONFORME» e zero fallimenti.
>
> Entrambe le cose sono state corrette: i banchi sono versionati, e **un banco assente rende l'esito NON CONFORME**. Un banco che non c'è non è un banco superato.

---

3. Composizione della batteria

63 banchi, raggruppati per ciò che dimostrano.

3.1 Dati scientifici

Banco	Verifica	Controlli
`audit_farmaci`	Struttura ⟷ peso, duplicati, e la **formula ricostruita dal grafo** contro quella dichiarata da ChEMBL per ogni voce con provenienza	263 voci, 36 con fonte

Banco	Verifica	Controlli
`test_spettri`	Riconoscimento dei gruppi funzionali su molecole di riferimento	36
`test_spettri_ui`	Innesto del motore nei punti dell'app che disegnano spettri	16
`test_assi`	Convenzioni degli assi negli spettri SVG	13
`test_assi_canvas`	Convenzioni degli assi negli spettri su canvas	6
`test_costanti`	Ricerca delle costanti fisiche e rifiuto delle ambiguità	45
`audit_dati`	Coerenza dei dati tabulati	29
`test_farm_ui`	Che l'Atlante Farmaci **disegni** ogni farmaco che ha in memoria, categoria per categoria, e che una categoria **mai etichettata** compaia comunque	11
`test_astro`	I dati di astrochimica: peso molecolare ⟷ formula delle 30 molecole interstellari contro i pesi atomici IUPAC, nessun corpo celeste ripetuto, ogni domanda del quiz con una risposta valida fra le sue opzioni	13
`test_datasci`	La matematica che la sezione Data Science **mostra**: R^2 su una relazione esatta, pesi standardizzati nel rapporto vero, ROC su valori calcolabili a mano, il comportamento su rumore puro, e che **nessuna sezione resti irraggiungibile**	23
`test_simmetria`	Gruppi puntuali, modi normali, regole di selezione IR/Raman	26
`test_cheminfo`	Il motore di chemioinformatica: Tanimoto e Dice su valori calcolabili a mano, impronte di Morgan e MACCS, raggruppamento di Butina, scheletri di Bemis–Murcko, PCA, regressione kernel, divisione per scheletro, **modello nullo per rimescolamento delle etichette**, validazione incrociata raggruppata, esportazioni CSV e SDF, **contrasto dei nove pannelli**, accordo fra laboratorio e motore, frammentazione, **coppie corrispondenti su effetti noti**, ricerca per sottostruttura, tabella SAR, confronto fra modelli nei due versi, arricchimento (EF e BEDROC) su casi calcolabili a mano, intervalli conformi e il loro **rifiuto sulla classificazione**	189
`test_spettrolettore`	Il lettore di spettri e i modelli che ne seguono: ASDF simbolo per simbolo, JCAMP esplicito e compresso in DIF/DUP, picchi per prominenza su spettri **costruiti** e **su rumore puro**, curva contro lista di picchi, perdite neutre sul toluene, ESOL contro quattro valori sperimentali, quattro filtri di drug-likeness separati, foresta casuale nei due versi, **una sola porta d'ingresso** (un bottone, `accept` vuoto, instradamento provato su PDF/ `.docx` / `.txt` / `.jdx` / `.csv` /binario travestito/immagine) e il distintivo di versione confrontato con `BSI_APP_VERSION`	63
`test_nmr`	La predizione NMR assegnata per atomo: due insiemi separati di molecole (taratura e **validazione**, quest'ultima mai usata per scegliere i parametri), il conteggio dei segnali su molecole di cui si sa quanti ne danno, i rifiuti su uno SMILES illeggibile e su una stringa vuota, la **cronologia** con il riconoscimento dei doppioni per chiave InChI, e la sincronia **picco ↔ struttura ↔ 3D** verificata sugli indici, non sul numero di atomi illuminati	64

Banco	Verifica	Controlli
`test_nmr2d`	Le mappe di correlazione contate sul grafo: HSQC con il segno invertito sui CH_2, COSY fra protoni non equivalenti, HMBC a due e tre legami. **Nei due versi**: l'esafluorobenzene non dà macchie HSQC, il benzene **nessuna** macchia COSY fuori diagonale, il metano **nessuna** macchia HMBC pur avendo quattro protoni	40
`test_digitalizza`	Uno spettro estratto da una **figura**, su sette immagini costruite di cui si conoscono i centri delle gaussiane al cm^{-1}: figura nuda, con cornice ed etichette, con griglia grigia, con griglia nera, con cornice **e** griglia, tratteggiata, in %T. Scarto **medio 2,2 cm^{-1}**, **peggiore 7,3** — una colonna vale ~4 cm^{-1}, che è il limite della figura e non del metodo. **Nei due versi**: da un foglio bianco e dall'icona dell'applicazione non esce **nessuna** curva; il ripiego grezzo sulla stessa figura con la cornice trova **un** picco invece di tre; su asse logaritmico il punto di mezzo è la media geometrica e su asse lineare quella aritmetica. Conversione %T→A sui valori esatti (50 %T → A = 0,301) e **giro completo** del JCAMP esportato, riletto dal lettore stesso	45
`test_articoli`	Che nessuna funzione abbia due porte d'ingresso, e che gli articoli non siano scritti a mano. Dieci schede dentro un solo Centro spettri, con ogni identificatore presente nel documento **una volta sola**; nessun id ripetuto, nessun pulsante doppio, nessuna sezione raggiungibile da due strade; Spectra fuori dal menù e con il suo pulsante dedicato. Poi gli articoli: i link provati come **funzioni pure** dell'identificatore, le spiegazioni dei Nobel abbinate all'argomento della motivazione e **nessuna** inventata quando non si riconosce, e la prova decisiva — una risposta **iniettata** con un nome inesistente, che deve comparire sullo schermo. **Tre guasti distinti**: richiesta fallita, risposta vuota, forma non riconosciuta	58
`test_geom3d`	La geometria 3D costruita dal grafo, misurata contro i valori di letteratura: legami, angoli di valenza, linearità degli alchini. **Nei due versi**: il benzene esce piano e il cicloesano **non** piano. E la prova che rende lecito il collegamento picco↔atomo: ogni atomo pesante ha **lo stesso indice e lo stesso elemento** del predittore	29
`test_documento`	L'apertura di qualunque documento e lo svolgimento del quesito che contiene: PDF, Word e testo aperti davvero e il contenuto verificato; i dati riconosciuti **con la loro provenienza**. **Nei due versi**: un'immagine dichiara che non c'è riconoscimento ottico e non inventa uno svolgimento, un formato sconosciuto viene rifiutato invece che indovinato, $CDCl_3$ non diventa la formula del composto, senza etichetta MS non ci sono masse	47

Un banco che verifica un modello deve verificarlo anche quando il modello deve fallire. `test_cheminfo` prova il modello nullo nei due versi: su un segnale apprendibile l'R^2 vero (0,394) batte tutti e otto i sosia con etichette rimescolate, con un margine di 0,287; sulle **stesse molecole con etichette casuali** l'R^2 (−0,199) **non** li batte. Un controllo che verifica solo il primo caso passerebbe anche se la guardia fosse cablata su «vero».

Un «contrasto 0» che non copriva la superficie nuova. `verifica-accessibilita` misura ciò che è visibile, e i sei pannelli della sezione Chemioinformatica restano nascosti finché l'analisi non è stata eseguita: non li aveva mai visti. Ora `test_cheminfo` esegue l'analisi, apre i pannelli a turno e misura il contrasto su **460 elementi di testo**, con le formule WCAG riscritte dentro il banco — chiedere al codice sotto esame quanto vale il proprio contrasto non è misurare. Il banco pretende inoltre di averne misurati più di 250: se i pannelli non si aprissero, lo zero sarebbe vuoto e il controllo fallirebbe.

Tre picchi trovati non dimostrano nulla da soli. Un rilevatore troppo sensibile ne trova trenta dove ce ne sono tre; uno troppo rigido non ne trova nessuno. In nessuno dei due casi compare un errore. `test_spettrolettore` parte da tre gaussiane **costruite** a 1715, 2950 e 3400 cm^{-1} sotto rumore e pretende che il lettore ne trovi tre — 1716, 2952, 3402 — e poi, sullo **stesso rumore senza gaussiane**, che ne trovi **zero**. Senza il secondo controllo, il primo passerebbe anche con un rilevatore che segna ogni oscillazione. Lo stesso vale per la foresta casuale: R^2 fuori sacco 0,90 sulla relazione vera e **−0,16 su puro rumore**.

Un predittore non si rompe mai: produce sempre dei numeri. Se sono sbagliati di venti ppm, lo spettro si disegna lo stesso e sembra uno spettro. L'unico modo di sapere se serve a qualcosa è confrontarlo con valori misurati da altri — e c'è una trappola che rende quel confronto inutile: tarare i parametri sulle stesse molecole su cui poi si misura l'errore. Il numero che esce dice allora quanto bene lo schema ricorda i propri esempi. `test_nmr` tiene due insiemi separati: **taratura** (9 molecole, scarto 0,56 ppm) e **validazione** (22 molecole mai usate per quello, **0,92 ppm**). Il valore dichiarato nel pannello è il secondo. E il banco pretende che la validazione resti **peggiore** della taratura: se un giorno fossero uguali, vorrebbe dire che qualcuno ha spostato una molecola da un insieme all'altro, e il numero non direbbe più niente.

Il difetto era nel mio contatore, non nei dati. La prima esecuzione di `verifica_farmaci_v187` ha dato **0 formule su 36**: ogni molecola risultava `C<n>`, tutti carboni e nessun idrogeno. Il formato `get_json` di RDKit è **commonchem**, dove un atomo porta soltanto i campi che **differiscono** dai valori predefiniti — `z` vale 6 e `impHs` vale 0 — così che un atomo scritto `{}` è un carbonio. Leggevo `a.element`, che in quel formato non esiste. Trentasei dati che sembrano sbagliati tutti insieme sono quasi sempre un contatore sbagliato.

3.2 Agente AI

Banco	Condizione simulata
`test_404`	Il fornitore ha ritirato il modello
`test_503`	Sovraccarico temporaneo

Banco	Condizione simulata
`test_ko` · `browser_ko`	Fornitore irraggiungibile dal browser
`test_firma`	Firme di ragionamento opache da restituire
`test_tetto`	Tetto di token appreso dall'errore
`test_attesa` · `test_attesalunga`	Timeout e attese imposte
`test_doppioinvio`	Due invii sovrapposti
`browser_prova`	Misura di raggiungibilità senza chiave
`test_nucleo` · `browser_prov`	Ciclo agentico e selezione del fornitore
`caccia_ai`	Risposte malformate, stream troncato, 502 con HTML, Stop durante la risposta, pannello chiuso a metà, testo ostile

3.3 Stabilità

Banco	Cosa mette alla prova
`audit_stabilita`	95 sezioni aperte 5 volte, memoria piena, dati corrotti, raffiche di clic
`audit_promesse`	Promesse rifiutate e non gestite, con rete sana e con rete morta
`audit_quota`	`localStorage.setItem` forzato a fallire su 10 pagine
`test_sw`	Offline, rete degradata, aggiornamento durante il lavoro
`test_filemanager`	Archivio non disponibile, spazio esaurito, scritture fallite
`test_visore3d`	Consumo di contesti WebGL sfogliando molecole

3.4 Interfaccia

`browser_reset` · `browser_proxy` · `browser_proxyui` · `browser_rdkit` · `browser_lab` · `browser_frontiera` · `test_aggiorna` · `test_guidaproxy` · `test_fluidita` · `test_lingue` · `test_mol3d` · `test_spettrolettore`

`test_fluidita` — 15 controlli. Misura quanto la pagina resta *bloccata* a ogni cambio di sezione, su tutte le 91. Ha trovato tre difetti che nessun altro banco vedeva: il «Viewer 3D PRO» che fermava la pagina **797 ms** al primo click — il profilatore li attribuisce al primo `render()` di 3Dmol, che compila gli shader del contesto WebGL; le **296 figure** di «Sintesi» (5 351 nodi di SVG) disegnate tutte prima di mostrare qualunque cosa, mentre all'apertura se ne vedono tre; e **296 timer a 40 ms di distanza** programmati all'apertura della stessa sezione — undici secondi e mezzo di risvegli che chiedevano `getTotalLength()` su ogni tratto di ogni figura, comprese quelle fuori dallo schermo.

Dopo le correzioni: cambio mediano **16 ms**, peggiore **87 ms**, **nessuna sezione oltre i 100 ms** (erano due, la peggiore a 1 166 ms).

Alla versione `bsi-v188` lo stesso banco ha intercettato due cose. La prima: con 263 farmaci invece di 233, «Farmacologia» bloccava la pagina **159 ms** e le sezioni oltre i 100 ms tornavano a essere **tre** — la batteria era passata per un soffio la volta prima, e un limite

superato a intermittenza non protegge niente. Le carte ora si disegnano a fette da 8 ms: **55 ms**, zero sezioni oltre i 100.

La seconda riguarda il banco stesso. La soglia sul visore 3D era **80 ms assoluti**; con lo stesso codice pubblicato, su container diversi, lo stesso click ha misurato 14, 22, 98, 105 e 192 ms, perché WebGL qui è **SwiftShader** e il costo è la compilazione degli shader. Una soglia così misura la macchina: fallisce a codice identico e passerebbe a codice peggiorato. Anche un **rapporto** su una calibrazione nella stessa esecuzione non regge — quando il banco arriva al visore, WebGL è già caldo (2,9 ms) e il visore paga il freddo: il rapporto oscillava fra 14× e 36×. L'affermazione è diventata **strutturale**, che è anche il difetto originario: subito dopo il ritorno del gestore del click il contesto WebGL **non deve esistere**, poco dopo **deve**. Misurato 0 e 1; e la misura non è vacua, perché al secondo click — quando la tela c'è già — restituisce 1.

`test_lingue` **— 64 controlli.** Sorveglia due funzioni nuove e due modi diversi di mentire senza accorgersene.

La lingua. Un interruttore che «traduce l'applicazione» è facile da scrivere e difficile da mantenere onesto: basta che qualcuno cambi un'etichetta italiana e la sua traduzione non si trovi più. Il difetto non si vede — l'etichetta resta in italiano in mezzo all'inglese — a meno che qualcuno conti. Il banco conta: **160 elementi di scheletro, 160 tradotti, zero rimasti**. Pretende anche le due prove contrarie: che passando all'inglese il testo **cambi davvero** (un dizionario vuoto passerebbe «tutto tradotto» senza muovere una lettera) e che tornando all'italiano il testo sia **identico carattere per carattere**.

I linguaggi delle molecole. Le conversioni si verificano contro fatti esterni — sette chiavi InChI di letteratura **scritte dentro il banco** — e si pretende il giro completo SMILES → molfile → SMILES. Le formule si contano a mano.

Il nominatore IUPAC. 26 nomi scritti a mano in due lingue, e sette molecole che **devono essere rifiutate**: un nominatore che non rifiuta mai è un nominatore che inventa.

Provato nei due versi: rimettendo il `trim()` che rompe i molfile e togliendo una traduzione, il banco fallisce tre controlli su venti.

Le porte. Dalla v184 il banco verifica anche che le due funzioni siano **raggiungibili**: pulsante di navigazione, sezione che si disegna davvero, e — controllo nato da un difetto reale — che nessuna delle due mostri il cartello «🚧 Sezione in costruzione», che il ripiego per le sezioni vuote stampa 800 ms dopo un click in qualunque sezione lasciata vuota. Verifica inoltre che la voce del menu ✨ **porti** alla sezione e non apra una finestra sovrapposta, e che non esista alcun identificativo duplicato con entrambe le sezioni disegnate.

Le sei lingue. Per ognuna si pretendono due cose insieme: copertura piena dello scheletro (**162 su 162**) e che il testo **cambi davvero** — un dizionario vuoto passerebbe «tutto tradotto» senza muovere una lettera.

`test_mol3d` **— 46 controlli.** Sorveglia l'animazione di formazione della molecola e la misura degli angoli di legame, due aggiunte che hanno due modi diversi di sembrare giuste senza esserlo.

L'animazione. «C'è» o «non c'è» non si vede ispezionando il codice: un ciclo che gira e ridisegna sempre la stessa cosa supera qualunque controllo sull'esistenza della funzione. Il banco guarda i **pixel**: a 130 ms la tela deve essere quasi vuota — polvere — e a 2,3 s deve esserci una molecola. Misurato: **532 pixel accesi → 2 334**. Se i due numeri fossero uguali l'animazione non ci sarebbe, comunque sia scritto il programma. Verifica anche che il ciclo **si fermi** quando lo si ferma: una tela rimossa con il ciclo ancora acceso è una perdita che si paga in batteria.

Gli angoli. Un numero con un «°» accanto sembra una misura. Si verificano su geometrie costruite a mano, il cui valore è noto **per costruzione** e non preso da una tabella: tetraedro regolare **109,4712°**, acqua 104,47°, CO_2 180°, BF_3 120°, ammoniaca 106,13°. Tolleranza 0,05°.

Il rifiuto. Su una struttura **piatta** ($z = 0$) il pulsante degli angoli deve essere spento e la ragione scritta: su coordinate 2D un angolo di legame non è un angolo di legame, e mostrarlo sarebbe l'errore peggiore dei due perché somiglia a un dato. Provato nei due versi: una struttura con $z \neq 0$ non deve essere scambiata per piatta, altrimenti nessun angolo si vedrebbe mai.

La prima stesura di questo banco leggeva la barra dei comandi **dopo** aver fermato il ciclo, che la rimuove: trovava zero pulsanti e poi dichiarava «ognuno ha un nome accessibile» su un insieme vuoto. Un controllo che passa perché non ha niente da guardare è esattamente il difetto che questo progetto insegue; ora la verifica pretende due pulsanti **e** due nomi.

La selezione. L'angolo lo sceglie chi guarda, cliccando sugli atomi: uno mostra tutti gli angoli che insistono su di lui, due la lunghezza del legame, tre l'angolo con il vertice nel secondo cliccato, quattro il **diedro**. Il banco verifica tutti e quattro i casi sull'etanolo — C–C 1,509 Å, C–C–O 111,9°, H–C–C–O 60,7° — e pretende che due atomi **non legati** siano dichiarati tali invece di essere spacciati per un legame: una distanza non è un legame.

I diedri. Verificati su quattro conformazioni costruite a mano: eclissata 0°, gauche 60°, ortogonale 90°, anti 180°. Anche qui la tavola scritta a mano ha corretto un mio errore, non del programma: con le coordinate che avevo scelto il diedro vale 120° e non 60°, perché guardando lungo il legame centrale il vettore proiettato sta a (−0,5; 0,866).

L'ordine d'arrivo. «Prima lo scheletro, poi gli idrogeni» è una proprietà **misurabile**, non un'intenzione: il banco legge l'avanzamento di ogni atomo a metà formazione e pretende che ogni atomo pesante sia davanti a ogni idrogeno. Questo controllo ha trovato un difetto reale: la funzione elastica che fa il rimbalzo non è monotòna, e usandola anche come avanzamento un atomo partito prima poteva risultare «meno arrivato» di uno partito dopo. Ora le due grandezze sono separate — l'avanzamento è monotòno, il fattore elastico fa solo il rimbalzo — e il rimbalzo, che prima veniva tagliato via, si vede.

Metà dei controlli sono le *prove contrarie*, perché una pagina che non costruisce niente è fulminea: la tela WebGL deve comparire comunque, le 296 figure devono esistere tutte poco dopo, nessuna cornice deve restare vuota, la stampa non deve uscire muta e la ricerca deve filtrare anche le carte la cui figura non è ancora nata. E il banco conta le sezioni che ha attraversato: su dieci invece di 89 i tempi sarebbero ottimi e la misura priva di valore.

3.5 Sicurezza e accessibilità

Banco	Verifica
`verifica-sicurezza`	Chiavi API nei file tracciati, password in chiaro, segreti nel `wrangler.toml`, telemetria, script da domini esterni
`audit_storia`	Che nessuna credenziale sia MAI entrata nel repository: **1 551 versioni distinte di file di testo** su **390 commit**, con 9 schemi. Si rifiuta di passare su un clone superficiale
`verifica-accessibilita`	Contrasto WCAG AA, nomi accessibili, etichette dei campi, testo alternativo, gerarchia dei titoli, attributo `lang` — su 13 pagine
`audit_mobile`	Che a **390 px** la pagina non scorra in orizzontale, su tutte le 95 sezioni — **e** che nessuna sezione ritagli contenuto rendendolo irraggiungibile: un elemento più largo della sua sezione, senza un antenato che scorra, è testo presente e non leggibile. Una tabella larga dentro un contenitore scorrevole non viene segnalata
`audit_rete`	Che **nessun dato dell'utente lasci il dispositivo**: un valore spia seminato in 71 depositi, l'app usata su 6 pagine e 95 sezioni, URL, intestazioni e corpo di ogni richiesta ispezionati
`audit_copertura`	Quanti byte di JavaScript vengono **davvero eseguiti** percorrendo l'applicazione: 49,89 %, registrato e difeso

`audit_storia` — 6 controlli. `verifica-sicurezza` esamina i file *tracciati*, cioè lo stato attuale: risponde a «oggi nel repository non c'è nessuna chiave» e non alla domanda che conta su un repository pubblico. Una chiave messa in un commit e togliata nel successivo passa `verifica-sicurezza` per sempre e resta leggibile a chiunque cloni, perché git non dimentica. Se succede, il rimedio non è un commit di correzione: è revocare la chiave.

Questo banco esamina ogni *versione* di ogni file di testo mai entrata nel repository. Esito: **zero credenziali in 1 551 versioni su 390 commit**.

Due difese contro sé stesso, perché qui un numero perfetto può nascere da meno superficie o da uno schema cieco:

- **Il clone non deve essere superficiale.** Un `--depth 1` contiene una

frazione della storia: il banco girerebbe, non troverebbe niente e direbbe «nessuna credenziale in tutta la storia» avendone vista un decimo. Se trova `.git/shallow`, fallisce e dice quale comando eseguire.

- **Gli schemi si provano prima di fidarsi.** Uno schema sbagliato non trova

niente ed è indistinguibile da un repository pulito: ognuno dei 9 deve riconoscere un esempio costruito e deve *rifiutare* un quasi-esempio, ed entrambi vengono provati dentro una coda di rumore con cifre, come sarebbe in un file vero.

La seconda difesa è nata da un errore in questo stesso banco. Lo schema AWS chiedeva una cifra con `(?=.*\d)`, condizione sempre soddisfatta in un file grande perché la cifra sta più avanti: trovava 53 corrispondenze, tutte la stessa stringa `AKIAAAAAAAAAAAAAAAA` — sedici «A», la zona di zeri di un'immagine in base64. Il quasi-esempio non se ne accorgeva perché era troppo corto per contenere cifre. Ora l'identificativo AWS si giudica dalla *varietà* dei suoi sedici caratteri, e i quasi-esempi portano la loro coda.

Provato anche nell'altro verso: con un commit locale che contiene una chiave AWS di forma valida il banco fallisce nominando il blob; rimosso il commit, torna a passare.

> **L'ostacolo era dello strumento, non del problema.** Per tre versioni la copertura di codice è stata dichiarata non misurabile, con questa motivazione: «strumentare richiederebbe introdurre una build, che l'applicazione non ha». Era vero per `c8` e `istanbul`, che riscrivono il sorgente prima di eseguirlo. Ma **Chromium la raccoglie da solo**, dentro il motore, senza toccare un byte del file servito: `Profiler.startPreciseCoverage`, che Playwright espone come `page.coverage.startJSCoverage()`. Non serviva una build: serviva accorgersi che il vincolo apparteneva agli strumenti scelti.
>
> **Il primo risultato è stato 100 % su ogni file**, compresi 3,6 MB di `index.html`. Era il banco che non misurava niente, nella forma più insidiosa: un numero perfetto. V8 emette per ogni funzione un intervallo esterno con il conteggio degli ingressi, e dentro quello gli intervalli a `count: 0` per ciò che non è stato eseguito. Sommando gli intervalli positivi si somma l'intero file. Si fa il contrario: si parte dal totale e si sottrae l'unione dei vuoti.
>
> Il numero vero è **49,89 %**, e regge lo stesso patto del debito di accessibilità: registrato, e il banco fallisce se scende. Una soglia assoluta («80 %») sarebbe una cifra inventata; «non deve peggiorare» è una promessa mantenibile.

> **Perché una spia e non un elenco di richieste.** SEC-02 — «nessun dato personale lascia il dispositivo» — era verificato controllando i due *meccanismi* di uscita: variabile di telemetria vuota, nessuno script da un dominio esterno. È un'evidenza vera ma indiretta: dice «non vedo come potrebbe uscire», che non è «non è uscito». Guardare invece l'elenco degli ospiti contattati sarebbe altrettanto debole, perché conclude per assenza di prove.
>
> La spia rovescia l'onere. Si scrive una stringa irripetibile dentro i dati dell'utente — note, chat, chiavi API, impostazioni, progressi — poi si **usa** l'applicazione, e ogni richiesta che esce viene aperta e letta. Se la stringa non compare da nessuna parte, è perché davvero non è uscita.
>
> ```
> BSI_PROVA_FUGA=1 node tools/banchi/audit_rete.js   # deve FALLIRE
> ```
>
> Il banco porta con sé la prova di funzionare: con quella variabile provoca una fuga di proposito, verso un ospite **dichiarato** — contattarlo è lecito, mandargli i dati dell'utente no. Se in quel caso non fallisce, è lo strumento a essere guasto.

Perché un banco sul traboccamento orizzontale. UI-03 dice «l'app deve funzionare a 390 px», e la matrice lo dava per verificato da `audit_stabilita`, che però in viewport telefono guarda gli **errori JavaScript**: una pagina può non averne uno solo e uscire lo stesso di centosessantatré pixel dallo schermo. Era un requisito dichiarato coperto e non misurato. Il banco misura `scrollWidth` del documento — esattamente ciò che fa comparire la barra — e non un elemento più largo dello schermo, che dentro un contenitore fatto per scorrere è corretto e riempirebbe l'uscita di rumore finché nessuno la legge più.

Cosa non copre la verifica di accessibilità, e va detto: il testo dentro gli SVG (il colore viene da `fill`, lo sfondo è una forma disegnata) e il testo su sfondi a gradiente (non hanno *un* colore). Gli elementi saltati vengono contati e riportati, non nascosti.

Una sezione per volta. L'ispezione salta gli elementi non visibili — ed è corretto: un elemento nascosto non ha contrasto da misurare. Ma `index.html` alterna 95 sezioni e ne mostra una sola: su **19 751** elementi di testo il banco ne guardava **41**, e stampava «0 difetti». Non era un risultato falso, era un risultato su un campione che nessuno aveva dichiarato. Ora le sezioni vengono aperte una per una, e **il numero di sezioni percorse viene stampato**.

Quel conteggio si è guadagnato lo stipendio alla prima esecuzione: il primo tentativo ne percorreva **zero** — il clic di Playwright aspetta che l'elemento sia visibile, e i pulsanti stanno dentro gruppi di navigazione richiusi — e il banco l'ha detto («95 sezioni presenti, nessuna percorsa») invece di stampare un altro zero rassicurante.

3.5-bis Il debito di accessibilità, e il patto che non cresca

Percorrendo tutte le sezioni sono emersi **1 069** difetti di contrasto. Oggi sono **zero** — l'elenco delle cause e dei rimedi è in `docs/09` §4 — e i campi privi di etichetta sono scesi da 40 a **zero**.

Il meccanismo descritto qui sotto resta, e conta più del numero che custodisce. Pretendere zero *da subito* avrebbe lasciato due sole vie, entrambe cattive: la verifica rossa per sempre, oppure allentata finché torna verde. La terza via è **dichiarare il numero** e impedirgli di crescere, qualunque esso sia. È così che da 1 069 si è arrivati a zero: ogni scalino registrato, nessuno annullabile di nascosto.

File	Ruolo
`docs/evidence/accessibilita-riferimento.json`	Il conteggio registrato

Situazione	Esito del banco
Il conteggio **sale**	✗ FALLITO — è una regressione
Il conteggio **scende**	✓ passa, e chiede di aggiornare il riferimento
Il conteggio è **uguale**	✓ passa in silenzio: debito dichiarato, non cresciuto — oggi quel valore è **0**, e il banco lo difende

```
node tools/verifica-accessibilita.js --aggiorna-riferimento
```

Da usare **solo dopo aver ridotto** i difetti, mai per far tornare verde una regressione. Anche a zero non è conformità WCAG piena: restano fuori il testo dentro gli SVG e quello su fondo a gradiente, che l'automatismo non sa giudicare e che il banco **conta e riporta** a ogni esecuzione (`docs/09` D-09). È la misura onesta di ciò che si può misurare, con la garanzia che non peggiori di nascosto.

3.6 Coerenza

Banco	Verifica
`verifica_guida`	Che le promesse della documentazione esistano nel codice; assenza di marcatori di conflitto su 58 file
`verifica-documenti`	Collegamenti interni, allineamento delle versioni, coerenza dell'indice, motivazione delle deviazioni, **coerenza interna della matrice di tracciabilità** (identificativi non duplicati, totali di copertura pari agli identificativi realmente definiti)
`genera-pacchetti`	Riscrive i riferimenti dei documenti estratti e **verifica che nessun collegamento resti rotto** nei tre pacchetti di consegna. Verifica anche che **nessun PDF sia più vecchio del documento da cui viene** e che il rapporto di verifica imbustato dichiari **questa** versione: un impaginato o un'evidenza della versione precedente, in un pacchetto di adesso, è una bugia silenziosa
`genera-pdf`	Converte i documenti in PDF e fallisce se un file risulta sotto la soglia di plausibilità (conversione a vuoto)
`verifica-affermazioni`	Confronta ogni numero dichiarato nei documenti — sezioni, farmaci, malattie, tumori, strategie, moduli — con quello **misurato nell'applicazione in esecuzione**, in italiano *e* in inglese
`test_stack`	Che lo stack React/FastAPI esegua **gli stessi file** dell'applicazione: confronta le impronte SHA-256 che `GET /salute` dichiara con quelle dei file su disco, verifica che i numeri coincidano con quelli del browser, e che **nessun file `.ts` / `.tsx` contenga una tabella di spostamenti** — perché è lì che nascerebbe il secondo motore

Perché i numeri vanno misurati, non ricordati. Il dossier tecnico si apre con delle cifre: «63 malattie (23 tumori)», «46 strategie», «25 moduli». Sono la prima cosa che legge chi valuta, e bastano perché tutto il resto venga creduto o messo in dubbio. Nessuno le controllava: scritte una volta, restavano, mentre l'applicazione cambiava. E infatti `chimorga.html` dichiarava **«25 moduli»** in cima e **«17 moduli»** più sotto — due affermazioni sullo stesso oggetto, nello stesso file, una delle due falsa.

Una lezione pagata scrivendo il banco. Avevo già concluso che «63 malattie» fosse un'esagerazione: nel sorgente avevo trovato un array `DISEASES` con 12 voci, e stavo per «correggere» il documento. Misurando nel DOM ho visto che il menu ne elenca davvero 63 — l'array che avevo trovato era un altro, più piccolo. **Un conteggio sul sorgente non è una misura: è un indizio.** Per questo il banco apre la pagina, apre la sezione e conta.

Controlla anche che i documenti concordino **fra loro**: due documenti che dicono cose diverse sullo stesso oggetto sono un difetto anche quando uno dei due ha ragione. La traduzione inglese conta come uno di quei documenti — `docs/en/05` è rimasto a «84 sections» e fermo alla versione `bsi-v146` per tre rilasci, e nessun banco se ne accorgeva perché nessuno guardava l'insieme inglese.

Perché un controllo sulla matrice. La matrice dichiarava `SCI-10` due volte — una come requisito verificato al 100 %, una come requisito esplicitamente **non** verificato — e la tabella riassuntiva sommava 36 requisiti mentre nel documento ne erano definiti 37. Entrambi gli errori stavano nel numero che un valutatore legge per primo. Ora il banco conta gli identificativi realmente presenti e fallisce se i totali scritti non corrispondono: la prova che il controllo misura davvero è che, reintrodotti i due errori uno per volta, li segnala entrambi.

---

4. Casi di prova notevoli

Alcuni banchi meritano una menzione perché verificano proprietà che un test funzionale non osserva.

Caso	Metodo	Risultato misurato
Perdite di memoria	95 sezioni × 5 giri, nodi DOM contati a ogni giro	+26 876 al primo giro (costruzione), **+0** nei quattro successivi
Memoria esaurita	`setItem` sostituito con una funzione che lancia sempre	10 pagine su 10 restano operative
Rete degradata ≠ assente	Richieste sospese 20 s, intercettazione a livello di contesto	Risposta dalla cache in **3 507 ms** (soglia 3 500)
Riproducibilità degli spettri	Doppio disegno, confronto byte a byte	Identici

Caso	Metodo	Risultato misurato
Contesti WebGL	Costruzioni del visore su 10 molecole consecutive	Da **7 a 1**
Cronologia corrotta	Due `Invio` a 500 ms di distanza	`user,assistant` anziché `user,user,assistant,assistant`
Contrasto del testo	Formula WCAG su ogni elemento con testo proprio, 13 pagine **e 95 sezioni**	19 751 elementi esaminati (erano 41): **1 069** difetti emersi, **1 069 corretti**, **0 registrati** come valore che non può crescere
Annullamento immediato	Stop premuto a 1,5 s, stato campionato ogni secondo	pulsante Invia disponibile dal **1º** secondo (era il 10º)

5. Come aggiungere un banco

Struttura minima

```
/* Un commento che spiega COSA dimostra questo banco e perché serve. */
const { chromium } = require('playwright-core');
const CHROME = '/opt/pw-browsers/chromium-1194/chrome-linux/chrome';

(async () => {
  const b = await chromium.launch({ executablePath: CHROME, args: ['--no-sandbox'] });
  // serviceWorkers:'block' se il banco modifica i file: senza, il SW
  // potrebbe servire una copia in cache e si misurerebbe il file vecchio
  const ctx = await b.newContext({ serviceWorkers: 'block' });
  const pg = await ctx.newPage();
  const err = [];
  pg.on('pageerror', e => err.push(e.message));

  let ok = 0, ko = 0;
  const att = (d, atteso, avuto) => {
    if (String(atteso) === String(avuto)) { ok++; console.log('  ✓ ' + d + '  → ' + avuto); }
    else { ko++; console.log('  ✗ ' + d + '\n      atteso: ' + atteso + '\n      avuto:  ' +
avuto); }
  };

  await pg.goto('http://127.0.0.1:8899/index.html', { waitUntil: 'load', timeout: 60000 });
  await pg.waitForTimeout(2500);

  // … misure …

  att('nessun errore JS', 0, err.length);
  await b.close();
  console.log('\n' + (ko ? '✗ ' + ko + ' FALLITI, ' : '') + ok + ' passati');
  process.exit(ko ? 1 : 0);   // il codice di uscita è ciò che conta
})();
```

Poi si aggiunge il nome alla famiglia opportuna in `tools/genera-evidenza.js`.

La regola che conta

> **Un banco che non misura nulla passa.** È il modo più insidioso di ottenere una falsa garanzia: nessun allarme, nessun fallimento, e nessuna verifica.

Perciò **ogni banco che simula una condizione deve contare quante volte la simulazione è realmente scattata, e fallire se il conteggio è zero.**

Casi realmente incontrati durante lo sviluppo della batteria:

Trappola	Come si manifestava
`page.route` non intercetta le richieste del Service Worker (serve `context.route`)	La prova «rete degradata» misurava una rete sana e concludeva in 16 ms invece di 3 500
Sondare `localStorage` esaurito con una scrittura di un byte	Dopo la saturazione un byte trova sempre posto: il banco «passava»
Cercare `[data-tab]` dove la navigazione usa `[data-s]`	Zero schede trovate, quindi zero misure — e zero fallimenti
Le tacche di un asse su `<canvas>` non sono testo	Vanno raccolte intercettando `fillText`; e la riga più in basso è il *titolo* dell'asse, non le tacche
Sonda installata su una libreria caricata **dopo**	Viene sostituita insieme alla libreria: zero chiamate contate mentre il codice funzionava
Una `var` dentro una IIFE non è `window.qualcosa`	Assegnarla da fuori crea una variabile diversa

---

6. Lacune di copertura

Dichiarate. La copertura riportata in `08-Traceability-Matrix.md` misura **quanti requisiti hanno un banco**, non quante righe di codice vengono eseguite: sono due cose diverse e non vanno confuse.

Lacuna	Situazione attuale	Raccomandazione
Copertura di codice	Misurata: **49,89 %** di istruzioni sul percorso più ampio. Resta fuori la copertura dell'intera batteria e quella di **rami**: un `if` entrato da un solo lato conta come coperto	Estendere la raccolta a ogni banco, e passare dalla copertura di istruzioni a quella di rami
Accessibilità	Automatizzata su 13 pagine e tutte le 95 sezioni: contrasto WCAG, nomi accessibili, etichette, testo alternativo, gerarchia dei titoli. Restano fuori il testo negli SVG e quello su gradiente, **contati** a ogni esecuzione	Affiancare `axe-core` per le regole che questo banco non implementa (ruoli ARIA, ordine di tabulazione, gestione del fuoco)
Sicurezza	`verifica-sicurezza` esegue 10 controlli su 368 file tracciati e copre SEC-01, SEC-03, SEC-05, SEC-06; SEC-04 è coperto da `verifica_guida`. **SEC-02 resta indiretto**: vedi `docs/09` D-03	Osservare il traffico di rete durante un uso reale, l'unica verifica diretta di SEC-02

Lacuna	Situazione attuale	Raccomandazione
Browser diversi da Chromium	Nessuna prova automatica su Firefox o WebKit	Estendere i banchi principali a `webkit`, dove le differenze su IndexedDB e Service Worker sono maggiori
Prestazioni	Prove manuali cross-device	Misura automatica del tempo di primo disegno
Regressione visiva	Assente	Confronto di schermate per i grafici, che sono il cuore del prodotto

---

7. Criteri di accettazione

Una versione non viene pubblicata se uno solo di questi non è soddisfatto.

- ☐ `node tools/genera-evidenza.js` si chiude con **0 falliti**
- ☐ `tools/verifica-farmaci.js`: nessun difetto, ogni deviazione registrata con il motivo
- ☐ `verifica_guida`: nessuna promessa della documentazione priva di riscontro
- ☐ Nessun errore JavaScript non di rete su tutte le pagine
- ☐ Rapporto di evidenza rigenerato sul commit che si pubblica

---

Documento aggiornato alla versione `bsi-v200`.